

MANIPAL: **Mutation Detection and **A**nnotation in **N**ucleotide Sequences with an **I**ntegrated **P**ipeline for **A**utomation in **L**aboratories**

Thesis submitted in partial fulfillment of the requirements for the degree of
Masters of Science
In
Bioinformatics

By:

Anil Kanthi. Vittal
Reg. No.: 131706005
M.Sc. Bioinformatics
School of Life Sciences
Manipal University

Project Advisor:

Mr. P. Sandeep Mallya
Lecturer
Department of Biotechnology
School of Life Sciences, Manipal University

July 2015

Department of Biotechnology
School of Life Sciences
Manipal University, Manipal

CERTIFICATE

To,
The Director,
School of Life Sciences,
Manipal University,
Manipal

As thesis advisor for **Anil V Kanthi** (Reg. No. 131706005) for the dissertation work titled “**MANIPAL: Mutation detection and Annotation in Nucleotide sequences with an Integrated Pipeline for Automation in Laboratories**”, I certify that I have read this dissertation, find it satisfactory and in compliance with the University rules and regulations towards the partial fulfillment for the award of Master of Science (Bioinformatics) of Manipal University, Manipal.

Place:

Date:

Thesis Advisor's Signature

Thesis Advisor's Name and Address:

DECLARATION

I, Anil V Kanthi hereby declare that the project entitled “**MANIPAL: Mutation detection and Annotation in Nucleotide sequences with an Integrated Pipeline for Automation in Laboratories**” has been completed under the guidance of Mr. P. Sandeep Mallya, School of Life Sciences, Manipal University, Manipal. This is being submitted as a partial fulfillment of the M.Sc. Bioinformatics. I also declare that this project has not been submitted to any other University/Institution for the award of any other degree.

Place: Manipal

Date:

(Anil V Kanthi)

ACKNOWLEDGEMENTS

First and foremost I offer my sincerest gratitude to my supervisor, **Mr. P. Sandeep Mallya**, Lecturer, School of Life Sciences, Manipal University, Manipal who has supported me throughout my thesis with his patience and knowledge whilst allowing me the room to work in my own way.

I would like to thank **Dr. K. Satyamoorthy**, Director, School of Life Sciences, Manipal for giving me the opportunity to work on the project and for constant guidance, valuable suggestions, ideas and patience throughout the duration of the project.

I would like to thank **Dr. Saadi Abdul Vahab**, Associate Professor, School of Life Sciences, Manipal University, Manipal for making me understand the concepts of diagnostic testing and reporting and for verifying the results.

I would also like to thank **Dr. Sanjiban Chakrabarty**, Assistant Professor, School of Life Sciences, Manipal University, Manipal for guiding my research.

I would like to express my special gratitude to **Dr. Gopalakrishna Bhat**, Professor, School of Life Sciences, Manipal University, Manipal for his blessings.

I would like to thank **Rajesh Rao** for helping me with collecting the datasets.

I would like to thank all my friends for cheering me up and for always being there through the good times and bad.

Finally, I thank my parents and family for constantly encouraging and supporting me throughout my studies at Manipal University.

	Table of Contents	
Sl. No	Content	Pages
1	Introduction	1
1.1	<i>In Silico</i> Approach to Identify Mutations	2
1.1.1	Databases	3
1.1.2	Data Analysis	3
1.1.3	Challenges Facing Variant Calling	4
2	Review of Literature	5
3	Aims and Objectives	8
4	Materials and Methods	9
4.1	Operating System	9
4.2	File format of the input data	9
4.3	Software's	9
4.4	Software setup	11
4.5	Test Datasets	11
4.6	Development of the Local Gene Database	12
4.7	Pre-processing	13
4.8	Procedure	14
4.9	Protocol for integrating multiple genes into the MANIPAL System	18
4.9.1	ClinVar Database	18
4.9.1.1	Script to customize ANNOVAR's ClinVar database for integrating multiple genes into the Local Gene Database	19
4.9.2	dbSNP Database	20
4.9.2.1	Script to customize ANNOVAR's dbSNP database for integrating multiple genes into the Local Gene Database	21
4.9.3	RefGene Database	22
4.9.3.1	Script to customize ANNOVAR's RefSeqGene database for integrating multiple genes into the Local Gene Database	22

	Table of Contents	
Sl. No	Content	Pages
4.9.4	Reference sequences	25
4.9.4.1	Script to integrate the forward strand references of multiple genes into the Local Gene Database	26
4.9.4.2	Script to integrate the reverse complementary references of multiple genes into the Local Gene Database	27
4.10	Source Codes	28
4.10.1	Primary analysis	28
4.10.1.1	Determining the gene orientation for forward ABI file processing	28
4.10.1.2	Processing forward ABI file for the positive strand gene	29
4.10.1.3	Processing forward ABI file for the negative strand gene	35
4.10.1.4	Determining the gene orientation for reverse ABI file processing	43
4.10.1.5	Processing reverse ABI file for the positive strand gene	44
4.10.1.6	Processing reverse ABI file for the negative strand gene	50
4.10.2	Secondary analysis	57
4.10.2.1	Extracting essential elements from ANNOVAR output	57
4.10.2.2	Converting 1 letter amino acid abbreviations into full names	124
4.10.2.3	Web page template for generating the genomic report	131
5	Results	134
5.1	Intermediate files	134
5.2	Visualizing Heterozygous peaks in chromatogram	135
5.3	BLAST Alignment	135
5.4	ANNOVAR's annotated output	138
5.5	Genomic Report	139
5.6	The generated HTML code of the MANIPAL System report	142
6	Discussion	151
7	Conclusions	152
8	References	153

List of Figures		
Sl. No	Content	Pages
1	Building a custom Local Gene Database for the Diagnostic genes. A. List of genes considered for building the local database. B. and C. Two aspects of information for the list of genes selected in A.	13
2	Working of the MANIPAL System based on how the ABI files are analysed	14
3	Workflow of the MANIPAL System. A. Base calling is done for the input ABI file. B. FASTQ file is created from the output of Phred. C. FASTQ file is aligned with the reference using BWA and the resulting SAM file is converted into binary format and indexed and sorted using SAMtools	15
4	Workflow of the MANIPAL System. A. Variants are detected from the indexed BAM file by FreeBayes. B. For both the forward and reverse ABI file analysis, the resulting VCF file from FreeBayes contains the variants position, which is converted into its genomic position. C. The final VCF file is taken as input by the ANNOVAR tool for annotation	16
5	Workflow of the MANIPAL System. A. Generating chromatogram from ABI file. B. Creating genomic report in PDF format using shell scripting	17
6	FASTA sequence of the input ABI file	134
7	The quality values corresponding to each bases in the FASTA file	134
8	FASTQ file containing the FASTA sequence along with the quality values	135
9	The aligned data in SAM format	135
10	The chromatogram generated by the Bioconductor package called “sangerseqR” highlights the peak that is heterozygous	136
11	BLAST Validation	137
12	IGV BAM Visualization	137
13	ANNOVAR output for the forward and reverse ABI file of WISP3	138
14	MANIPAL System’s genomic report	141

List of Tables		
Sl. No	Content	Pages
1	List of software's used in the MANIPAL System	10
2	Features annotated by the ANNOVAR annotation software	10
3	The list of diagnostic genes analysed by the MANIPAL System and the corresponding conditions that the genes were diagnosed for	12

Chapter 1

INTRODUCTION

Single nucleotide variations in the DNA sequence occur when a single base (A, T, G or C) gets altered and are described as mutations as well as polymorphisms. A variant, which has a frequency of more than 1%, is called polymorphism whereas a mutation is defined as an irreversible alteration in the genomic sequence (Richards *et al.*, 2015). Every cell in the body relies on hundreds or thousands of proteins to work properly and carry out their functions. When a mutation occurs, it may block many of the proteins from performing their mechanisms correctly. A gene's normal function of producing a protein can be altered by a mutation, thus causing a flaw in the protein or even stop it from being produced. Thus, when a mutation changes a protein that has an important part to play in the body, it can damage the regular growth or can result in a disease. In few instances, the alterations are so dangerous that it may even terminate an embryos growth to remain alive till the birth (Genetics Home Reference, 2015). The crucial thing to consider is that the genes are not responsible for the cause of the disease, but the mutations that make the gene to behave abnormally are.

The completion of the Human Genome Project as well as new techniques to detect mutations has resulted in the rise of the number of mutations in the known genes. While the drop in the cost of sequencing along with the rapidly developing bioinformatics resources, has led to the establishment of Deoxyribonucleic acid (DNA) sequencing in the molecular diagnostic laboratories for the identification of disease causing variants, which is very essential in the context of genetic testing (Green *et al.*, 2013). Molecular genetic testing examines individual genes in order to discover the mutations or variations, which may cause a genetic condition. A variation can have huge or little impact on the chances of carrying a certain disorder. Diagnostic testing specifically detects the variants that leads to a disorder or dismisses the presence of a genetic condition. Detection of such variants assists in the diagnosis of individuals with a high chance of having a gene mutation. Single nucleotide polymorphisms (SNP's) and small insertions and deletions (INDEL's) are common genetic variants that are related to a lot of human genetic conditions. However, the words 'polymorphism' and 'mutation' are often used interchangeably which leads to uncertainty due to wrong deduction about their pathogenicity and benign effects. To avoid the confusion, the term 'variant' has been recommended for use as a standard by The American College of Medical Genetics and Genomics (ACMG). The clinical significance of a variant ranges from the variant being

definitely pathogenic to being benign (Richards *et al.*, 2015).

Sequencing techniques such as Sanger and next generation sequencing (NGS) are the popular methods to identify these variants. Despite the advent of next-generation sequencing technologies, Sanger sequencing still remains the most popular and robust method of automatic sequencers. It is established on capillary electrophoresis and involves the technique of incorporating chain-terminating dideoxynucleotides selectively by DNA polymerase during DNA replication, which is performed *in vitro*. It is also less susceptible to errors in sequencing whereas its sensitivity and accuracy makes the fluorescence-based sequencing as the ‘gold’ standard for detecting DNA variations in the gene of interest for medical as well as research applications (Schrijver *et al.*, 2012).

1.1 Insilico Approach to Identify Mutations

Along with the experimental approaches to classify the variants, there has been high-level research carried out in bioinformatics in order to understand the effects that these variants have at the molecular level. Furthermore to the experimental assays carried out in genomics, the bioinformatics research has concentrated on two common areas. First, the variation data have been annotated by the researchers along with the features at the molecular level, which are likely to influence the function. Second, in order to predict the mutations that are likely to have influence at the molecular level, statistical techniques have been developed.

All these efforts have led to the emergence of various web resources, databases and software’s for narrowing down to the candidate SNP’s or to hypothesize the reason for the genetic diseases at the molecular level. For example, the National Center for Biotechnology Information (NCBI), established in 1988 at the National Library of Medicine (NLM), stores the biological data submitted by researchers from all over the world and is now host to various databases such as Online Mendelian Inheritance in Man (OMIM, <http://www.ncbi.nlm.nih.gov/sites/entrez?db=omim>) (Hamosh *et al.*, 2005), ClinVar (<http://www.ncbi.nlm.nih.gov/clinvar/>) (Landrum *et al.*, 2014), Reference Sequence (RefSeq, <http://www.ncbi.nlm.nih.gov/refseq/>) (Pruitt *et al.*, 2007) and The Single Nucleotide Polymorphism database (dbSNP, <http://www.ncbi.nlm.nih.gov/SNP/>) (Sherry *et al.*, 2001), to name a few. Also tools such as Basic Local Alignment Search Tool (BLAST, blast.ncbi.nlm.nih.gov/BlastAlign.cgi) (Altschul *et al.*, 1990) and browsers like The University of California Santa Cruz Genome Browser (UCSC, <http://genome.ucsc.edu/>) (Karolchik *et al.*, 2007) have made it easier to analyse the data and gather information for undergoing scientific

research.

1.2 Databases

The RefSeqGene database of National Center for Biotechnology Information (NCBI), accessible at (<http://www.ncbi.nlm.nih.gov/refseq/rsg>), provides the sequences of well-annotated genes, which can be used as reference for analysis and serves as a basis for identifying the disease causing variants.

There are numerous databases such as dbSNP, OMIM and ClinVar that give us access to the variation data. These repositories, store the constantly increasing number of variants identified in the human genome and serve as a great resource for the classification and reporting of variations found in the diagnostic laboratories. Well-structured and comprehensive databases such as these provide us the details relating to the variations affecting an individual's gene.

The dbSNP database stores the genetic variations data. The OMIM repository contains information related to human genetic conditions while the ClinVar Database collects genomic variation data and provides information of the variants that are of importance to the health of the humans.

1.3 Data Analysis

With the explosion of data generation, there is also a need to automate the data processing. Currently, the analysis of data involves manual checking of the raw sequencing data (i.e., chromatograms) to identify variants and then annotate the same for each of the identified variants from various sources (review article). Sensitivity of such analysis requires accuracy and precision on the part of the analyser to confirm or rule out a possible genetic disorder. An elaborate manual procedure may lead to introduction of error in the analysis. Bioinformatics expertise can be used to automate the process of identifying and annotating the clinically relevant variants and currently, not many resources are available that can accomplish this task. Most of the software solutions presently available require user interference or provide modules that demand expertise in informatics skills to allow implementation in pipelines.

Experimental validation for the association of these variants to a disease is very time consuming and labor intensive, so there is a need to automate the process of variant analysis, classification and reporting. Hence, to reduce the amount of time consumed during the analysis and reporting of the clinically important variants of an individual, various methods currently exist to classify the identified SNP's on the basis of their pathogenicity such as SIFT (Sorting

Intolerant From Tolerant) (Ng and Henikoff, 2003), Polyphen2 (Polymorphism Phenotyping v2) (Adzhubei *et al.*, 2010), etc. Such programs predict the likely effect that a variant may have on the protein.

1.4 Challenges Facing Variant Calling

There are many popular programs that perform base calling, alignment, polymorphism identification, polymorphism annotation and variant assessment. These tools need to be configured separately and human interference is needed in all the steps to assess the variants. Currently, there are no software tools or packages that can accomplish the entire process of calling and annotating variants. There is a need for a fully automated workflow that could be used in a clinical setting as currently, there are none available.

Hence, for the purpose of clinical diagnosis, the MANIPAL (**M**utation detection and **A**nnotation in **N**ucleotide sequences with an **I**ntegrated **P**ipeline for **A**utomation in **L**aboratories) system, an automated framework for analyzing the ABI (Applied BioSystems) file format of the Sanger sequencing data, was developed to overcome these issues.

Chapter 2

REVIEW OF LITERATURE

Sequencing techniques are being accepted all over for pre-emptive measure for diagnosis and to identify biomarkers. As part of the MedSeq Project, which is a study published in 2015, genome sequencing was done in patients who have been declared by doctors with not having any serious clinical problem. After sequencing, variants were identified from the raw data files and classified based on the proof from the literature. After the classification, the report included the categorized variants. The report contained brief explanations of the significant identifications to the doctors. The starting section consists of the primary findings that may lead to a condition and is followed by more specific details for the detected variants. Classes were engaged for doctors who volunteered in the MedSeq Project and were introduced to the area of genomics. Assistance was provided to the doctors with a choice of communicating the relevant genetic consultant for help in understanding the patient's genome sequencing report. It was made sure that the information was distributed equally in the starting page and the succeeding pages. The report was generated using a software called "GeneInsight suite", in which the data of categorizing the variant is regularly updated to include the new discoveries. Whenever the classification of a variant that has been reported earlier changes, the concerned doctor gets notified about it. (Vassy *et al.*, 2015)

For the diagnosis of genetic conditions, Sanger sequencing is a widely used method. The data generated needs a bioinformatics expertise that will help in automating the entire process. Not many resources are available that can accomplish this task of automating the variant identification and annotation of the Sanger sequence data. The ODS (Online Diagnostic System) combines the tedious process of calling the bases, detecting the variants and annotating them by using a web server. It is accessible at <http://sunlab.lihs.cuhk.edu.hk/ODS>. The users will just have to upload the input ABI files and the ODS will process the raw data and provide the result in a very short period of time as compared to one day for a clinician to do it manually without any knowledge of the data analysis. It also allows to visualize the chromatograms and highlights the position of the mutation. The ABIF input files are base called using Phred, followed by Basic Local Alignment Search Tool (BLAST) to align the reads to the human reference genome of build NCBI 37. Using some algorithms that are in-house, the variants are called. The annotation process is performed individually for each of the resources. The exonic regions of RefSeq and Ensemble are checked for the location of the variants as well as the

possibility of the variants to be located in the CpG Islands. Next the variation databases such as dbSNP and HGMD are scanned for the presence of the variants. The functional effect of these variants are then assessed using the SIFT program and the information retrieved from OMIM database. The result provides summary about the trimmed bases, the quality the ABI files, the mapping details, variation report and the corresponding OMIM details if available for the variant. The ODS was demonstrated by processing the data of two genes of humans, namely XK and CASR and can be very useful for researchers to help them in the analysis of Sanger data. (Sun *et al.*, 2014)

After the diagnostic testing is completed in the laboratories, the report is generated. The reports that are brief but still clear in explanation are useful and valuable. The final report must contain all the steps that were performed during the test, clear explanation, procedure and references. The discovered variations should be displayed in a tabular format since the number of known variants identified in the laboratories is growing day by day. The explanation of the reported variants must contain all the proof in favor of the classified variant. All the variants reported in the results segment should be interpreted in substantially and may also contain extra details. The procedure by which the variant was discovered by the test must be noted along with the shortcomings of the assay in the report. (Richards *et al.*, 2015)

ANNOVAR (Annotate Variation) is an annotation software to shed light on the functional effect of the variations from the DNA sequencers. It also provides a pipeline for the reduction of the variants in order to point out specific set of variants that are highly likely to cause a disease. The major advantage of ANNOVAR is that its processing time for the annotation of variants is relatively fast. This is mainly because of the databases that have been pre-assembled without having to perform the MSA's (Multiple Sequence Alignments) every time nor having to wait for the response of the SQL databases that are remotely located. ANNOVAR also provides a feature that helps in filtering out the variants for the potential mutations in *Homo sapiens* genome which are non-synonymous through the functionally important scores such as SIFT, Polyphen which are pre-compiled. Three different kinds of annotations can be performed using ANNOVAR, namely the gene-based annotations, region-based annotations and the filter-based annotations. When ANNOVAR reads the input variant file, it processed it with the gene databases that are locally available, and then detects the variants located in the exon, intergenic region, intron, 3' or 5' UTR, upstream or downstream and at the splice sites. ANNOVAR reports the genes that are near the variants if they are present in the internecine region along with their distances. The changes in the amino acid and the stop

loss or stop gain mutations are reported if the variants are located in exotic regions, by processing the mRNA sequences that are annotated. On annotation tracks like the conserved elements as well as TFBS's, ANNOVAR is capable of carrying out annotation that is region based. Since it is a program that is operated on command line, ANNOVAR can be easily installed and operated as a standalone tool. (Wang *et al.*, 2010)

Chapter 3

AIM AND OBJECTIVES

Aim:

To develop an automated diagnostic framework for analyzing, assessing and reporting the clinically significant variants of individuals from the Sanger sequencing data in the most time efficient and reproducible manner.

Objectives:

1. To perform base calling and trimming of low quality reads.
2. To align the forward and reverse reads against the reference gene sequence.
3. To call and annotate the variants.
4. Customizing the MANIPAL pipeline for integrating the Diagnostic genes.
5. Visualizing heterozygous peaks in the chromatogram traces of the input ABI files.
6. Create a genomic report with all the significant mutations.

Chapter 4

MATERIALS AND METHODS

4.1 Operating System

The MANIPAL System, made for the automated data analysis of the Sanger sequence reads, was built and tested on Biolinux 7 (Field *et al.*, 2006) operating system and also configured on Red Hat Enterprise Linux 6 server operating system.

4.2 File format of the input data

The primary data on which the MANIPAL System starts its operation is a single ABI file. The variant calling program used in the further analysis has been configured to call the variants from an individual ABI file.

4.3 Software's

The commonly used open source tools were considered and assessed for the study, but the one's which provided the optimal results were implemented into the framework (**Table 1**).

- Phred (Ewing *et al.*, 1998) - A program to call the bases from the Sanger sequencing data.
- Burrows-Wheeler Aligner (BWA) (Li, 2013) - It is an alignment tool to map the forward and reverse reads to the reference.
- SAMtools (Li *et al.*, 2009) - A software that allows to index, merge and sort the aligned data to make it acceptable for processing by further programs.
- Freebayes (Garrison and Marth, 2012) - Tool that uses indexed and sorted aligned data along with the reference sequence to identify the position of the variants. For the objective of using single ABI file to identify the variants, an additional parameter of FreeBayes was added to change the number of observations needed to call a variant and was set to "1".
- ANNOVAR (Annotate Variation) (Wang *et al.*, 2010) - It is a software built with Perl programming language to annotate the functional effects of the discovered variants. Popularly used gene and variation databases along with databases of programs that

predict the consequences of a variant are provided by ANNOVAR.

- sangerseqR (Hill *et al.*, 2014) - It is a Bioconductor package that generates chromatogram traces and locates the heterozygous peaks.
- Wkhtmltopdf - It is a github repository which allows in converting HTML (HyperText Markup Language) page into a PDF (Portable Document Format).

Table 1: List of software's used in the MANIPAL System.

	Step	Program Name
1	Base-calling	Phred (v0.020425.c)
2	Alignment/Mapping	BWA (v0.7.10)
3	Alignment Post-Processing	SAMtools (v0.1.19)
3	Variant Calling	FreeBayes (v0.9.16)
4	Variant Annotation	ANNOVAR (2015Mar22)
5	Reporting	wkhtmltopdf (v0.12.2.1)
6	Chromatogram	sangerseqR (v1.4.0 on R version 3.1.3)

Table 2: Features annotated by the ANNOVAR annotation software.

	Features
1	Amino Acid Change in refGene
2	ClinVar Accession ID
3	Clinical Association
4	dbSNP rsID
5	Exonic Function in refGene
6	Function in refGene
7	Gene Reviews
8	MedGen ID
9	OMIM ID
10	Polyphen Score
11	Polyphen Prediction
12	SIFT Score
13	SIFT Prediction

All the software's were installed, configured and examined individually for their optimal results, before implementing them for automation.

4.4 Software setup

- Phred - Request for the Academic license of the software by sending the completed “ACADEMIC USER AGREEMENT” form to the corresponding individuals at: http://www.phrap.org/consed/academic_agreement.txt and download and install the program from the provided link by following the instructions in the documentation.
- BWA - Download and install bwa from <https://github.com/lh3/bwa>
- SAMtools - Download and install samtools from <https://github.com/samtools/samtools> along with its dependencies.
- FreeBayes - Download and install freebayes from <https://github.com/ekg/freebayes>
- ANNOVAR - Register for the ANNOVAR's license at: http://www.openbioinformatics.org/annovar/annovar_download_form.php and download the software from the provided link. Also download the required databases from <http://annovar.openbioinformatics.org/en/latest/user-guide/download/>
- Wkhtmltopdf - Download and install from <https://github.com/wkhtmltopdf/wkhtmltopdf>
- sangerseqR - Download R from <http://cran.r-project.org/>. In order to start downloading the sangerseqR package from Bioconductor, begin R by providing the command “R” on the terminal and issue the following commands once R console starts on the terminal:

```
> source("http://bioconductor.org/biocLite.R")  
> biocLite("sangerseqR")
```

This downloads and installs the package automatically along with its dependencies.

4.5 Test Datasets

ABI files of 11 diagnostic genes (**Table 3**), generated from the Applied BioSystems 3130 Genetic Analyzer at the School of Life Sciences, Manipal University were considered for the

study.

Table 3: The list of diagnostic genes analyzed by the MANIPAL System and the corresponding conditions that the genes were diagnosed for.

	Gene Name	Name of the Condition	NCBI Accession ID
1	ADRB2	Beta2 adrenoreceptor	NC_000005.9
2	ATP7B	Wilson's Disease	NC_000013.10
3	HBB	Beta-Thalassemia	NC_000011.9
4	GALNS	Morquio A syndrome	NC_000016.9
5	GJB2	Non-syndromic hearing loss	NC_000013.10
6	GLB1	Morquio B Syndrome and GM1 Gangliosidosis	NC_000003.11
7	p53	p53 mutations screen	NC_000017.10
8	TWIST2	Setleis syndrome	NC_000002.11
9	WISP3	Progressive Pseudorheumatoid Dysplasia	NC_000006.11
10	WNT7A	Fuhrmann syndrome	NC_000003.11
11	UGT1A1	Gilbert's syndrome	NG_033238.1

4.6 Development of the Local Gene Database

Instead of using the GRCh37 (Genomic Reference Consortium Human genome 37) *Homo sapiens* genome as the reference, the Local Gene Database, hereafter referred to as LGdb, is created with the reference gene sequences for the list of diagnostic genes that are being tested for (**Figure 1 A**).

The reference sequences as well as the reverse complementary reference for each of the genes are downloaded from the NCBI RefSeqGene Database (**Figure 1 B**). The start coordinate of each genes are added to the FASTA (Fast Alignment) header of reference, after the '>' symbol.

The data corresponding to the genes under consideration are extracted from the RefSeqGene, dbSNP and ClinVar databases of ANNOVAR with a script and added to the LGdb (**Figure 1 C**).

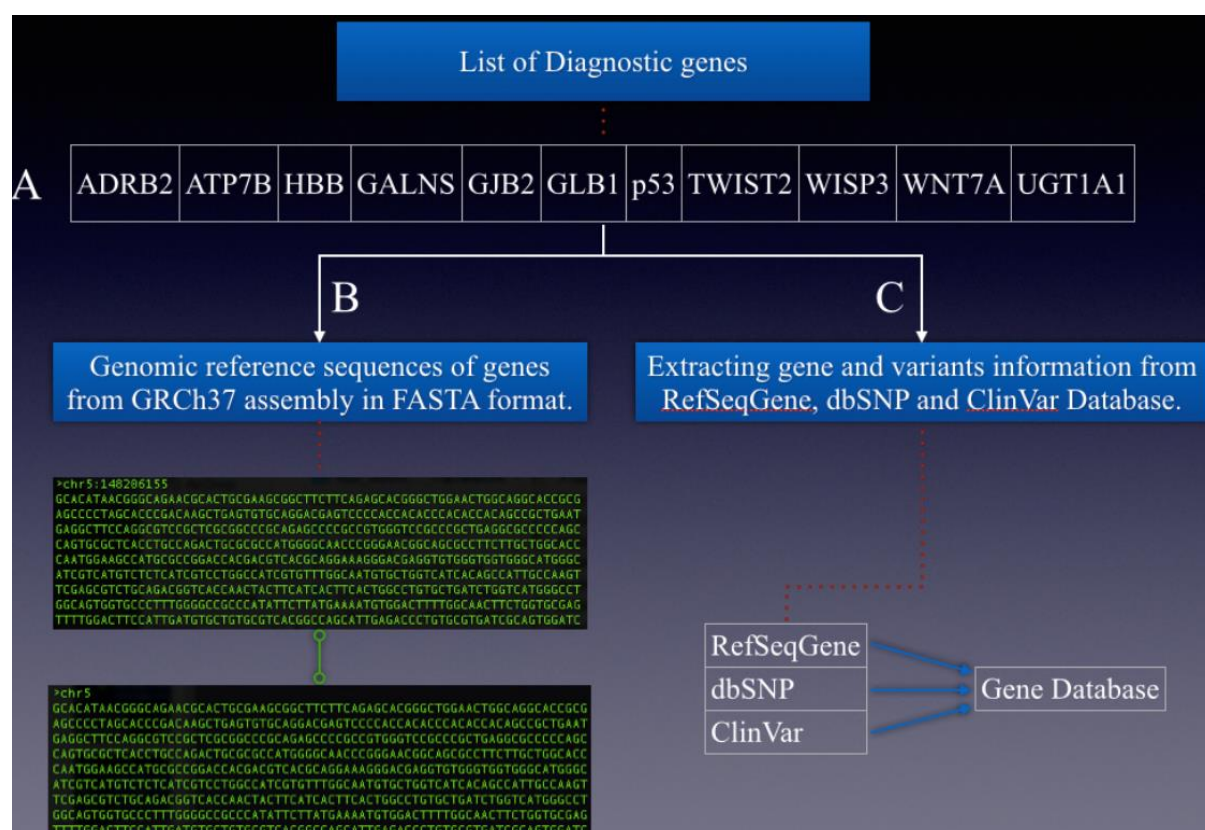


Figure 1: Building a custom Local Gene Database for the Diagnostic genes. **A.** List of genes considered for building the local database. **B.** and **C.** Two aspects of information for the list of genes selected in **A.**

4.7 Pre-processing

Before reading the ABI input file, the MANIPAL System determines the orientation of the gene by acquiring the genes name from the user and searching it in the LGdb. Based on whether the gene is on the positive or negative strand, the relevant script is executed to analyse the ABI file (**Figure 2**).

If the gene is present on the positive strand then the script that handles the execution of the forward ABI file on the plus strand is executed after providing the ABI file. If the gene is present on the negative strand then the script that handles the execution of the forward ABI file

on the minus strand is executed.

After the forward ABI file is analyzed, the orientation of the gene for the reverse ABI file processing is determined and the respective script that handles the reverse ABI file is executed.

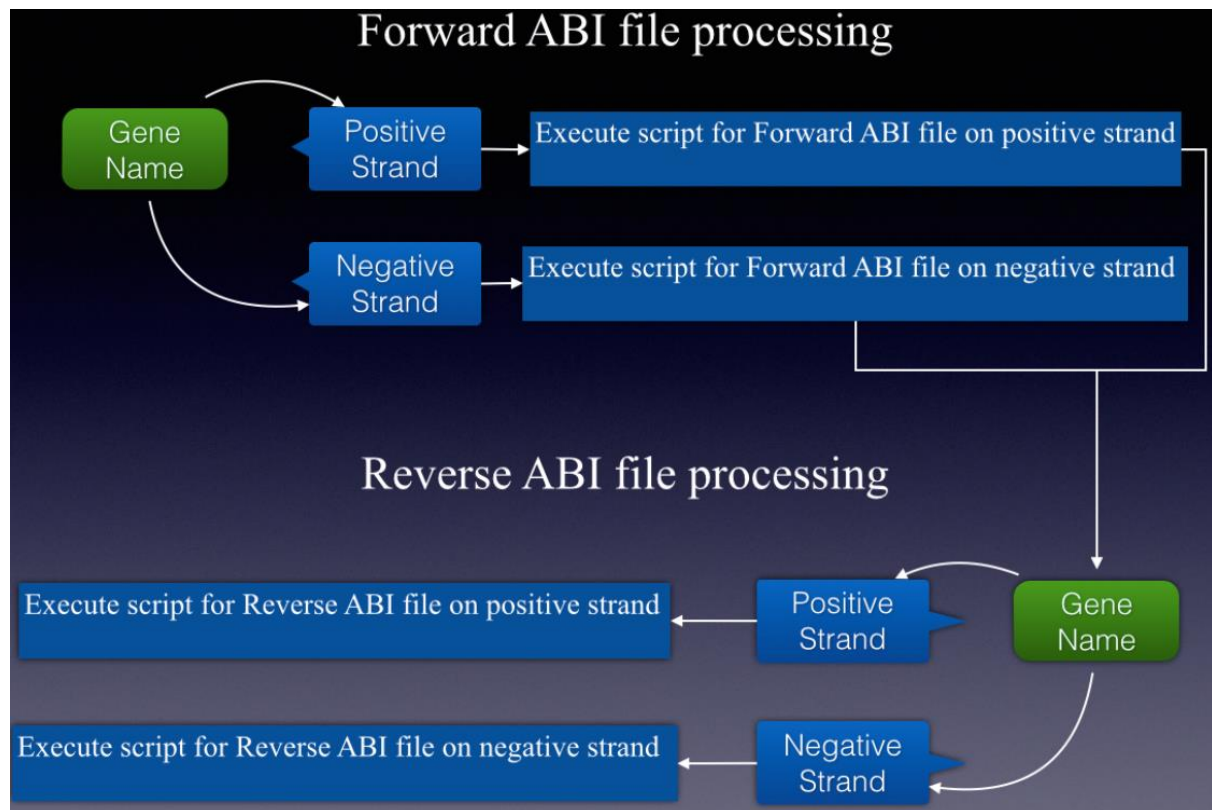


Figure 2: Working of the MANIPAL system based on how the ABI files are analyzed.

4.8 Procedure

The MANIPAL System (**Figure 3, Figure 4 and Figure 5**), consists of a series of steps starting from reading the ABI file as input to calling the variants to generating a report, with emphasis on the medically important variants. After the trimming of low quality bases following base calling, the FASTQ file is created and the single read is aligned to the reference sequence. Subsequent to indexing and sorting the aligned Binary Alignment/Map (BAM) file, the variant(s) are detected and then annotated to obtain a tabular file with all the annotated features of the observed variant(s). Using this resultant file as a foundation for reporting, the

essential fields are extracted, appended into a HTML page and subsequently converted into a PDF file.

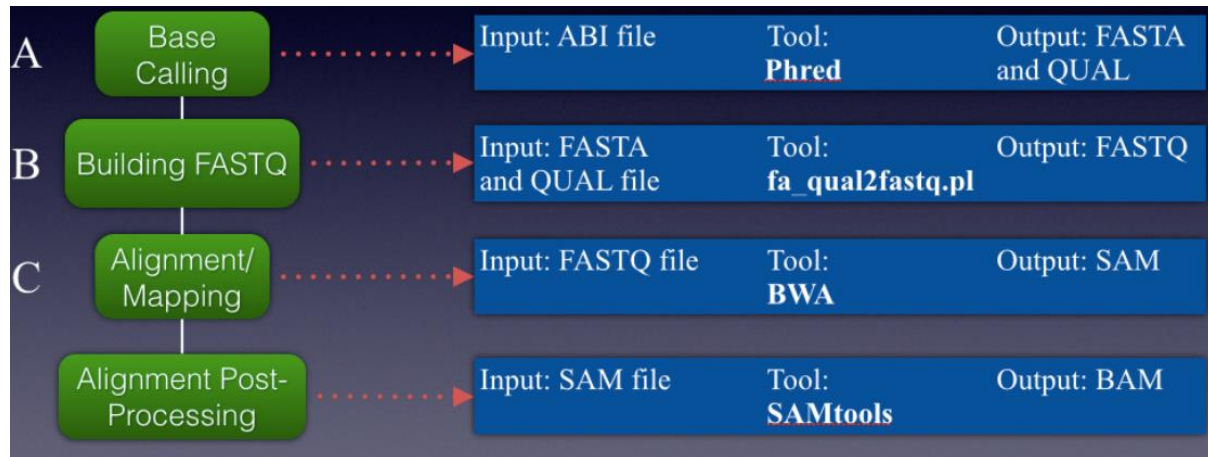


Figure 3: Workflow of the MANIPAL System. **A.** Base calling is done for the input ABI file. **B.** FASTQ file is created from the output of Phred. **C.** FASTQ file is aligned with the reference using BWA and the resulting SAM file is converted into binary format and indexed and sorted using SAMtools.

Step 1 | Base calling— The MANIPAL System takes ABI file as input and Phred performs base calling (**Figure 3 A**) by trimming out the low quality bases, resulting in FASTA and quality files.

Step 2 | Building FASTQ— A Bioperl (Stajich *et al.*, 2002) script combines the FASTA file and the corresponding quality values to create a FASTQ file (**Figure 3 B**).

Step 3 | Alignment— BWA aligner is used for mapping the FASTQ read to it's appropriate reference, resulting in a Sequence Alignment/Map (SAM) file format (**Figure 3 C**).

Alignment of gene on plus strand: The forward ABI file is aligned to the reference using an in-house script whereas the reverse ABI file is first reverse complemented by the “revseq” program of EMBOSS (Rice *et al.*, 2000) and then aligned to the reference.

Alignment of gene on minus strand: The forward ABI file is first reverse complemented by the “revseq” program and then aligned to the reference, whereas the reverse ABI file is aligned to the reference without reverse complementing.

Step 4 | Alignment Post-processing— In order to convert the human readable SAM file into a computer understandable binary file, the SAM file is converted into a binary BAM file by a tool named SAMtools (**Figure 3 C**).

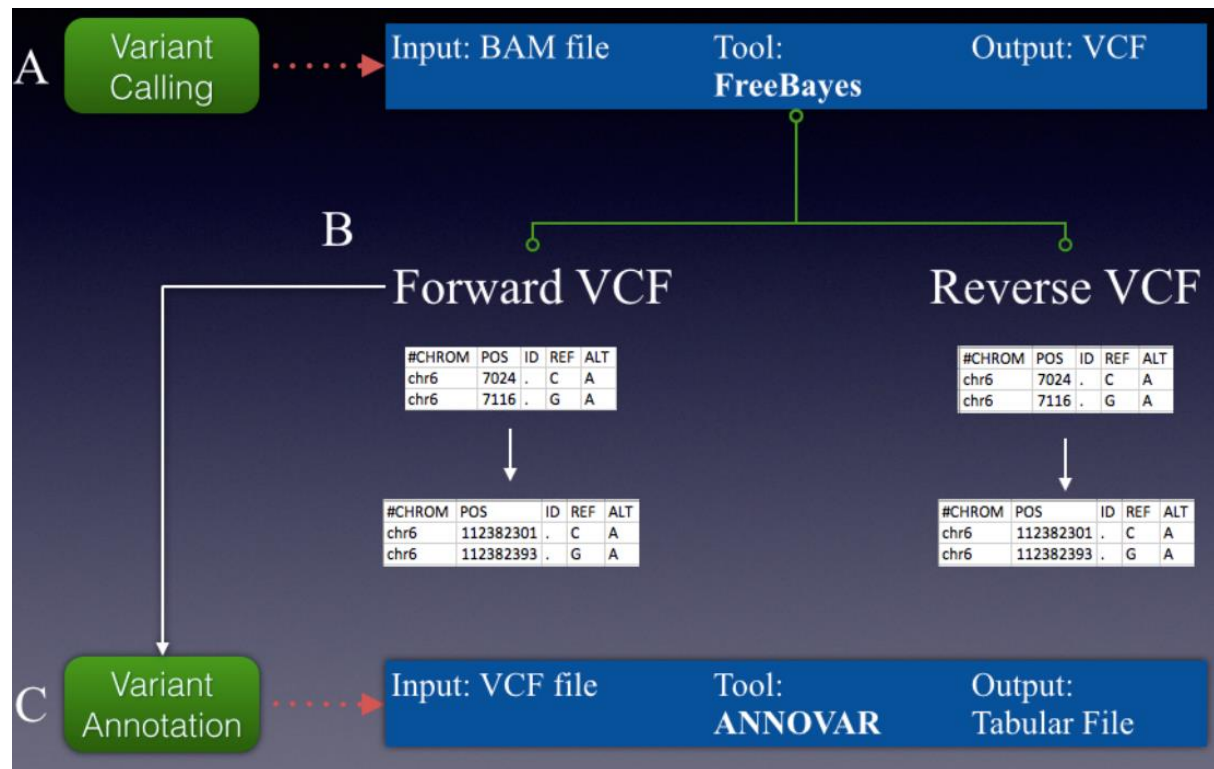


Figure 4: Workflow of the MANIPAL System. **A.** Variants are detected from the indexed BAM file by FreeBayes. **B.** For both the forward and reverse ABI file analysis, the resulting VCF file from FreeBayes contains the variants position, which is converted into its genomic position. **C.** The final VCF file is taken as input by the ANNOVAR tool for annotation.

Step 5 | Variant Calling— FreeBayes tool discovers the variants from the indexed BAM file (**Figure 4 A**). It generates a file in Variant Call Format (VCF) (Danecek *et al.*, 2011) containing the individual variant's position, which is in association to the reference sequence in the LGdb. For accurate annotation of the identified variants according to the genomic location of the variant, the start co-ordinate of the gene is extracted from the FASTA header of the corresponding reference in the LGdb and is added to each of the variant's position using AWK (Alfred Weinberger Kernighan) (Aho *et al.*, 1988), thus creating the correct VCF file for annotation (**Figure 4 B**).

Step 6 | Variant Annotation— The variants present in the VCF file are annotated by the ANNOVAR’s Perl script “table_annovar.pl” (**Figure 4 C**). The output is in a tab-delimited text format and contains annotations for all the called variant(s).

Step 7 | Visualizing the DNA sequence chromatogram of heterozygous peaks— With an ABI file as input, the “makeBaseCalls” function of a Bioconductor package called “sangerseqR”, is used to highlight the peaks that are heterozygous. The primary and secondary basecalls are similar for homozygous peaks but different for the heterozygous peaks (**Figure 5 A**).

Step 8 | Generating diagnostic report— A web page is created using HTML language and is used as a template for generating report. The essential elements are checked in the fields from each row of the ANNOVAR’s annotated file, and if present it is extracted and assigned as variables and placed in a temporary file. These Bash variables are then integrated between the HTML tags and the HTML document is converted into a PDF report (**Figure 5 B**).

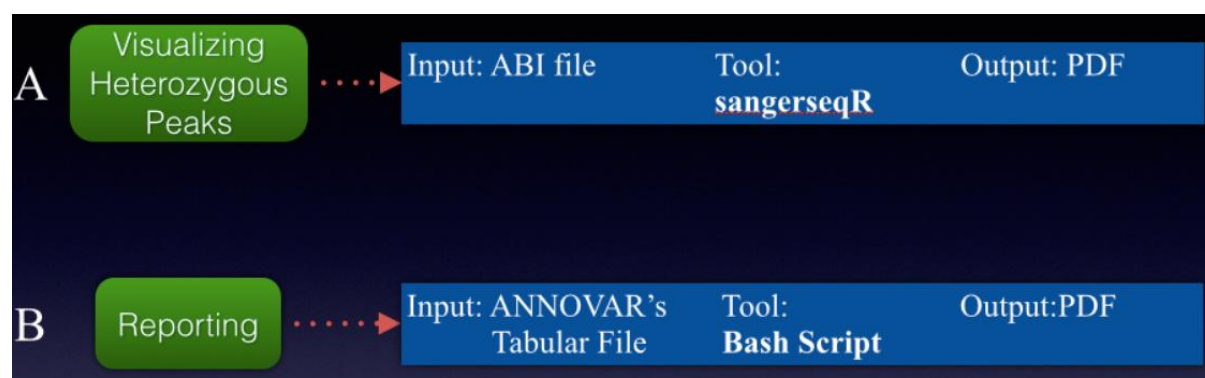


Figure 5: Workflow of the MANIPAL System. **A.** Generating chromatogram from ABI file. **B.** Creating genomic report in PDF format using shell scripting.

Automating the MANIPAL system— The automation of the entire running of the system was done using Bash (GNU bash, version 4.3.11) scripting by integrating the Perl scripts along with all the command line tools used for the data analysis. Four separate in-house Bash scripts (**Code 4.10.1.2**, **Code 4.10.1.3**, **Code 4.9.54.10.1.5**, and **Code 4.10.1.6**) are implemented in order to provide the choice for analyzing ABI files of the forward as well as the reverse strand.

4.9 Protocol for integrating multiple genes into the MANIPAL System

For each gene, there are three different local databases, which contain the genes data from RefSeqGene, dbSNP, and ClinVar databases. Once a genes name that needs to be analyzed is provided to the MANIPAL System, the relevant script checks for the identified variant's annotation in only three of these databases for the analyzed gene and provides the annotated result.

In order to add new genes to the existing MANIPAL System's LGdb, the gene specific data needs to be extracted from the three databases and the genes reference needs to be formatted to make it suitable for processing.

4.9.1 ClinVar Database

The ANNOVAR's database as well as data of new genes to be included, must be downloaded from the NCBI's Variation Viewer tool (version 1.3) by carrying out the following steps-

- a) Check for the latest ClinVar database of ANNOVAR and download using the ANNOVAR's Perl script "annotate_variation.pl" by the following command-

```
perl annotate_variation.pl -buildver hg19 -downdb -webfrom annovar  
clinvar20150330 humandb/
```

- b) Go to the homepage of NCBI's Variation Viewer tool (<http://www.ncbi.nlm.nih.gov/variation/view/>) and pick the GRCh37.p13 Assembly.
- c) Enter the gene name in the search box section and select the same from the displayed menu.
- d) In the Variation data section, tick "Yes" for the 'dnSNP in Source database' as well as the 'In ClinVar' option.
- e) Download the table in "TSV" file format.
- f) After downloading the data for each gene, rename the text files with the correct name of the gene following the "GENENAME.tsv" format.
- g) If required, change the path of the ANNOVAR's ClinVar Database on "Line 8" of the "clinvar_adding_MANIPAL.sh" script.
- h) Change directory [cd] to the folder on the terminal where all the ".tsv" files are present

and run the “clinvar_adding_MANIPAL.sh” script by providing the name of the gene to be integrated.

4.9.1.1 Script to customize ANNOVAR’s ClinVar database for integrating multiple genes into the Local Gene Database

```
#!/bin/bash

# Usage :
# 1> Change the path of the ANNOVAR's CLINVAR Database on "Line 8".
# 2> After Downloading data from NCBI Variation Viewer, name the files
with the correct name of the GENES [named in "GENENAME.tsv" format].
# 3> Change directory [cd] to the folder on the terminal where all the
.tsv files are present and run this script!

CLINVAR=/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB/DB/
hg19_clinvar_20150330.txt

# CONVERTS all filenames to Uppercase and keeping extensions the same!
perl -e 's/\.[^\.]*$/rename $_, uc($`) . $&/e for @ARGV' *.tsv

read -p "Enter the name of the Gene: " gene_name
if [ ! -z $gene_name ]
then
echo "The entered gene name is '$gene_name'. "
else
until [ -d "$dir" ] && [ "$dir" != " " ]
do
echo "The Gene name cannot be empty and is required by the MANIPAL System
to integrate new Genes."
echo "Please provide the Name of the Gene : "
read gene_name
done
fi
cat $gene_name.tsv | sed '/^#/d' | awk '{print $2}' | uniq >
clinvar_$gene_name.txt
mkdir LGdb
grep -w -F -f clinvar_$gene_name.txt $CLINVAR >
LGdb/hg19_clinvar_20150330$gene_name.txt

perl compileAnnnovarIndex.pl LGdb/hg19_clinvar_20150330$gene_name.txt 1000
> LGdb/hg19_clinvar_20150330$gene_name.txt.idx
rm -rf clinvar_$gene_name.txt

echo " Add the files in the LGdb folder to the following path :
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB "

# mv clinvar/hg19_clinvar_20150330$gene_name.txt
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/

while true; do
    read -p "Do you want to extract any more genes features from ClinVar
database, please enter with 'Yes' or else 'No' : " yn
    case $yn in
        [Yy]* ) bash clinvar_adding_MANIPAL.sh; break;;
        [Nn]* ) echo
```

```
"~~~~~ Thank You
for adding new Genes to the the MANIPAL system.
~~~~~"; exit;;
* ) echo "Please answer with 'Y' or 'N'. ";
esac
done
```

4.9.2 dbSNP Database

To obtain the data of all the dbSNP ID's for the additional genes, the corresponding data needs to be downloaded from The University of California Santa Cruz (UCSC) Table Browser by carrying out the following steps. The downloaded data, which contains the rsID's for the genes can then be intersected with the whole genome dbSNP database of ANNOVAR to create a LGdb's for each of the genes.

- a) Check for the latest dbSNP database of ANNOVAR and download using the ANNOVAR's perl script "annotate_variation.pl" by the following command-

```
perl annotate_variation.pl -buildver hg19 -downdb -webfrom annovar
avsnpl42 humandb/
```

- b) Go to the homepage of UCSC Table Browser (<https://genome.ucsc.edu/cgi-bin/hgTables>) and specify the organisms clade as "mammal", genome as "Human" and assembly as "Feb.2009 (GRCh37/hg19)".
- c) Select "Variation" as group and "All SNPs(142)" as the database in track to display the corresponding options in the table menu.
- d) Limit the query of the gene name to specific position by selecting the "position" radio button in the region control. "lookup" for the genes name after typing it in the text box to select the chromosomal position of the gene under the RefSeq Genes list.
- e) If a gene has more than one transcript then create a 3-field BED (Browser Extensible Data) file for all the transcripts, containing the chromosomal position, start and end coordinates of the gene and using the "define regions" button, the BED file can be entered. This step is necessary to include the variants of the gene form all its transcripts.
- f) Select the output format as "BED- browser extensible data" and specify the output format as "plain text" after entering the filename in the output text box with the correct name of the gene following the "GENENAME.txt" format.

- g) Submit the data query by clicking the “get output button”, select the “Whole Gene” radio button under the “Create one BED record per” section and download the BED file by clicking the “get BED” button.
- h) After downloading data from UCSC Table Browser, name the files with the correct name of the genes [named in "GENENAME.txt" format].
- i) If required, change the path of the ANNOVAR's dbSNP Database on "Line 8" of the “dbSNP_adding_MANIPAL.sh” script.
- j) Change directory [cd] to the folder on the terminal where all the “.txt” files are present and run the “dbSNP_adding_MANIPAL.sh” script by providing the name of the gene to be integrated.

4.9.2.1 Script to customize ANNOVAR's dbSNP database for integrating multiple genes into the Local Gene Database

```
#!/bin/bash

# Usage :
# 1> Change the path of the ANNOVAR's dbSNP Database on "Line 8".
# 2> After Downloading data from UCSC Table Browser, name the files
with the correct name of the GENES [named in "GENENAME.txt" format].
# 3> Change directory [cd] to the folder on the terminal where all the
.txt files are present and run this script!

DBSNP=/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB/DB/hg
19_avsnpl42.txt

# CONVERTS all filenames to Uppercase and keeping extensions the same!
perl -e 's/\.[^\.]*$/rename $_, uc($`) . $&/e for @ARGV' *.txt

read -p "Enter the name of the Gene: " gene_name
if [ ! -z $gene_name ]
then
echo "The entered gene name is '$gene_name'. "
else
until [ -d "$dir" ] && [ "$dir" != " " ]
do
echo "The Gene name cannot be empty and is required by the MANIPAL System
to integrate new Genes."
echo "Please provide the Name of the Gene : "
read gene_name
done
fi

cat $gene_name.txt | awk '{print $4}' > snpids_$gene_name.txt

mkdir LGdb
```

```

grep -w -F -f snpids_$gene_name.txt $DBSNP >
LGdb/hg19_avsnpl42$gene_name.txt

rm -rf snpids_$gene_name.txt

echo " Add the files in the LGdb folder to the following path :
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB "

while true; do
    read -p "Do you want to extract any more genes features from dbSNP
database, please enter with 'Yes' or else 'No' : " yn
    case $yn in
        [Yy]* ) bash dbSNP_adding_MANIPAL.sh; break;;
        [Nn]* ) echo
"~~~~~ Thank You
for adding new Genes to the the MANIPAL system.
~~~~~"; exit;;
        * ) echo "Please answer with 'Y' or 'N'. ";;
    esac
done

```

4.9.3 RefGene Database

By performing the following steps, the ANNOVAR's RefGene database needs to be downloaded and the new genes to be added should be searched in the corresponding databases for a match to extract its annotated features together with its mRNA sequence in FASTA format to create LGdb's for each of the genes.

- a) Check for the latest refGene database of ANNOVAR and download using the ANNOVAR's perl script "annotate_variation.pl" by the following command-

```

perl annotate_variation.pl -buildver hg19 -downdb -webfrom annovar
refGene humandb/

```

- b) If required, change the path of the ANNOVAR's refGene Databases on "Line 5" and "Line 7" of the "refGene_adding_MANIPAL.sh" script.
- c) Execute the "refGene_adding_MANIPAL.sh" script on the terminal by changing directory [cd] to the folder where the script is present and by providing the name of the gene to be integrated.

4.9.3.1 Script to customize ANNOVAR's RefSeqGene database for integrating multiple genes into the Local Gene Database

```

#!/bin/bash

# Usage :
# 1> Change the path of the ANNOVAR's REFGENE Databases on "Line 7"

```

```

and "Line 9".
# 2> Change directory [cd] to any folder of your wish on the terminal
and run this script!

REFGENE=/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB/DB/
hg19_refGene.txt

REFGENEMRNA=/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB
/DB/hg19_refGeneMrna.fa

read -p "Enter the name of the Gene: " gene_name
if [ ! -z $gene_name ]
then
    echo "The entered gene name is '$gene_name'. "
else
until [ -d "$dir" ] && [ "$dir" != " " ]
do
    echo "The Gene name cannot be empty and is required by the MANIPAL
System to integrate new Genes."
    echo "Please provide the Name of the Gene : "
    read gene_name
done
fi

# To create hg19_refgene.txt -->
cat $REFGENE | grep -w "$gene_name" > hg19_refGene$gene_name.txt

mkdir LGdb

mv hg19_refGene$gene_name.txt LGdb/

# To create hg19_refgeneMrna.txt -->

mkdir refgenemrna

cat $REFGENE | grep -w "$gene_name" | awk '{print $2}' > gene_ids.txt

while IFS="" read -r line; do printf -v line$((++i)) "%s" "$line"; done <
gene_ids.txt

if [ -z "$line1" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line1/,+1p" $REFGENEMRNA >
refgenemrna/line1.txt ; fi

if [ -z "$line2" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line2/,+1p" $REFGENEMRNA >
refgenemrna/line2.txt ; fi

if [ -z "$line3" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line3/,+1p" $REFGENEMRNA >
refgenemrna/line3.txt ; fi

if [ -z "$line4" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line4/,+1p" $REFGENEMRNA >
refgenemrna/line4.txt ; fi

if [ -z "$line5" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line5/,+1p" $REFGENEMRNA >
refgenemrna/line5.txt ; fi

```

```

if [ -z "$line6" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line6/,+1p" $REFGENEMRNA >
refgenemrna/line6.txt ; fi

if [ -z "$line7" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line7/,+1p" $REFGENEMRNA >
refgenemrna/line7.txt ; fi

if [ -z "$line8" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line8/,+1p" $REFGENEMRNA >
refgenemrna/line8.txt ; fi

if [ -z "$line9" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line9/,+1p" $REFGENEMRNA >
refgenemrna/line9.txt ; fi

if [ -z "$line10" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line10/,+1p" $REFGENEMRNA >
refgenemrna/line10.txt ; fi

if [ -z "$line11" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line11/,+1p" $REFGENEMRNA >
refgenemrna/line11.txt ; fi

if [ -z "$line12" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line12/,+1p" $REFGENEMRNA >
refgenemrna/line12.txt ; fi

if [ -z "$line13" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line13/,+1p" $REFGENEMRNA >
refgenemrna/line13.txt ; fi

if [ -z "$line14" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line14/,+1p" $REFGENEMRNA >
refgenemrna/line14.txt ; fi

if [ -z "$line15" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line15/,+1p" $REFGENEMRNA >
refgenemrna/line15.txt ; fi

if [ -z "$line16" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line16/,+1p" $REFGENEMRNA >
refgenemrna/line16.txt ; fi

if [ -z "$line17" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line17/,+1p" $REFGENEMRNA >
refgenemrna/line17.txt ; fi

if [ -z "$line18" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line18/,+1p" $REFGENEMRNA >
refgenemrna/line18.txt ; fi

if [ -z "$line19" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line19/,+1p" $REFGENEMRNA >
refgenemrna/line19.txt ; fi

if [ -z "$line20" ]; then echo "There is no Transcript available for the
$gene_name Gene "; else sed -n "/$line20/,+1p" $REFGENEMRNA >
refgenemrna/line20.txt ; fi

```



```

cd refgenemrna

cat *.txt > hg19_refGeneMrna$gene_name.fa

cd ..
mv refgenemrna/hg19_refGeneMrna$gene_name.fa LGdb/

rm -rf refgenemrna gene_ids.txt

echo " Add the files in the LGdb folder to the following path :
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB "

while true; do
    read -p "Do you want to extract any more genes features from refGene
database, please enter with 'Yes' or else 'No' : " yn
    case $yn in
        [Yy]* ) bash refGene_adding_MANIPAL.sh; break;;
        [Nn]* ) echo
"~~~~~ Thank You
for adding new Genes to the the MANIPAL system.
~~~~~"; exit;;
        * ) echo "Please answer with 'Y' or 'N'. ";;
    esac
done

```

4.9.4 Reference sequences

Download each genes reference sequence from the NCBI Gene Database by carrying out the following steps. In order to make the genes reference compatible with the running of MANIPAL System, the FASTA header of each reference sequence needs to be modified by changing the default header line and adding the chromosomal location followed by the start co-ordinate of the gene in the “>chr:start-position” format.

- a) Go to the homepage of NCBI’s Gene Database (<http://www.ncbi.nlm.nih.gov/gene/>), enter the genes symbol in the search box following the “GeneName[sym]” format and filter the results by taxon by selecting the organism as “*Homo sapiens*”.
- b) Under the “Genomic regions, transcripts, and products” section and change the assembly of the Genomic Sequence to “GRCh37.p13 Primary Assembly”.
- c) Under the “Customize view” section, uncheck the “Show reverse complement” display option to obtain the reference of the forward strand of the gene and download the FASTA sequence by navigating to the genomic FASTA sequence of the gene.
- d) To get the reverse complementary reference of the gene, check the “Show reverse complement” display option under the “Customize view” section and download the FASTA sequence by navigating to the genomic FASTA sequence of the gene.

- e) After downloading the sequences for all the genes individually, name each FASTA file with the correct name of the gene following the "GENENAME.fasta" format.
- f) Change the path of the ANNOVAR's refGene Database on "Line 8" of "gene_ForwardReference_adding_MANIPAL.sh" script.
- g) Change directory [cd] to the folder on the terminal where all the forward strand reference sequence [GENENAME.fasta] files are present and execute the "gene_ForwardReference_adding_MANIPAL.sh" script.
- h) Change the path of the ANNOVAR's refGene Database on "Line 8" of "gene_ComplementaryReference_adding_MANIPAL.sh" script.
- i) Change directory [cd] to the folder on the terminal where all the forward strand reference sequence [COMP_GENENAME.fasta] files are present and execute the "gene_ComplementaryReference_adding_MANIPAL.sh" script.

4.9.4.1 Script to integrate the forward strand references of multiple genes into the Local Gene Database

```
#!/bin/bash

# Usage :
# 1> Change the path of the ANNOVAR's REFGENE Database on "Line 8".
# 2> After Downloading data from NCBI's Gene Database, name the files
with the correct name of the GENES [named in "GENENAME.fasta" format].
# 3> Change directory [cd] to the folder on the terminal where all the
gene reference sequence [.fasta] files are present and run this script!

REFGENE=/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB/DB/
hg19_refGene.txt

# CONVERTS all filenames to Uppercase and keeping extensions the same!
perl -e 's/\.[^\.]*$/rename $_, uc($`) . $&/e for @ARGV' *.fasta

read -p "Enter the name of the Gene: " gene_name
if [ ! -z $gene_name ]
then
    echo "The entered gene name is '$gene_name'. "
else
until [ -d "$dir" ] && [ "$dir" != " " ]
do
    echo "The Gene name cannot be empty and is required by the MANIPAL
System to integrate new Genes."
    echo "Please provide the Name of the Gene : "
    read gene_name
done
fi
```

```

cat $REFGENE | grep -w "$gene_name" | awk '{print $3}' > gene_chr.txt

chr=$(cat gene_chr.txt | sed -n 1p)

mkdir LGdb

cat $REFGENE | grep -w "$gene_name" | awk '{print $5}' > gene_position.txt

position=$(cat gene_position.txt | sed -n 1p)

sed -i.bak '1 s/^.*$/>'$chr':'$position'/' $gene_name.fasta

mv $gene_name.fasta LGdb/

echo " Add the files in the LGdb folder to the following path :
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB "

rm gene_chr.txt gene_position.txt
find -maxdepth 1 -type f -name "*.bak" -delete
find -maxdepth 1 -type f -name "*.fasta~" -delete
find -maxdepth 1 -type f -name "*.sh~" -delete

while true; do
    read -p "Do you want to add any other genes reference sequence to the
    LGdb database, please enter with 'Yes' or else 'No' : " yn
    case $yn in
        [Yy]* ) bash gene_ForwardReference_adding_MANIPAL.sh; break;;
        [Nn]* ) echo
        "~~~~~ Thank You
        for adding new Genes to the the MANIPAL system.
        ~~~~~"; exit;;
        * ) echo "Please answer with 'Y' or 'N'. ";;
    esac
done

```

4.9.4.2 Script to integrate the reverse complementary references of multiple genes into the Local Gene Database

```

#!/bin/bash
# Usage :
# 1> Change the path of the ANNOVAR's REFGENE Database on "Line 8".
# 2> After Downloading data from NCBI's Gene Database, name the files
with the correct name of the GENES [named in "COMP_GENENAME.fasta" format].
# 3> Change directory [cd] to the folder on the terminal where all the
gene reference sequence [.fasta] files are present and run this script!

REFGENE=/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB/DB/
hg19_refGene.txt

# CONVERTS all filenames to Uppercase and keeping extensions the same!
perl -e 's/\.[^\.]*$/rename $_, uc($`) . $&/e for @ARGV' *.fasta

read -p "Enter the name of the Gene: " gene_name
if [ ! -z $gene_name ]
then
    echo "The entered gene name is '$gene_name'. "
else

```

```

until [ -d "$dir" ] && [ "$dir" != " " ]
do
    echo "The Gene name cannot be empty and is required by the MANIPAL
System to integrate new Genes."
    echo "Please provide the Name of the Gene : "
    read gene_name
done
fi

cat $REFGENE | grep -w "$gene_name" | awk '{print $3}' > gene_chr.txt

chr=$(cat gene_chr.txt | sed -n 1p)

mkdir LGdb

cat $REFGENE | grep -w "$gene_name" | awk '{print $5}' > gene_position.txt

position=$(cat gene_position.txt | sed -n 1p)

sed -i.bak '1 s/^.*$/>'$chr':'$position'/' COMP_$gene_name.fasta

mv COMP_$gene_name.fasta LGdb/

echo " Add the files in the LGdb folder to the following path :
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/geneDB "

rm gene_chr.txt gene_position.txt
find -maxdepth 1 -type f -name "*.bak" -delete
find -maxdepth 1 -type f -name "*.fasta~" -delete
find -maxdepth 1 -type f -name "*.sh~" -delete

while true; do
    read -p "Do you want to add any other genes reference sequence to the
LGdb database, please enter with 'Yes' or else 'No' : " yn
    case $yn in
        [Yy]* ) bash gene_ComplementaryReference_adding_MANIPAL.sh; break;;
        [Nn]* ) echo
"~~~~~ Thank You
for adding new Genes to the the MANIPAL system.
~~~~~"; exit;;
        * ) echo "Please answer with 'Y' or 'N'. ";;
    esac
done

```

4.10 Source Codes

4.10.1 Primary analysis

These scripts handle the steps beginning from finding the orientation of the gene to annotating the identified variants.

4.10.1.1 Determining the gene orientation for forward ABI file processing

```

#!/bin/bash

echo "::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::

```



```

fi

position=$(sed -n '/:/p' $reference | cut -d':' -f2 | sed -n 1p)
if [ ! -z "$position" ]
then
echo "The genomic start coordinate of the gene is '$position'."
else
until [ ! -z "$position" ]
do
echo "The genomic start coordinate of the Gene cannot be empty and is
required by the MANIPAL System."
read -p "Please provide the edited Gene Reference file [with .fasta
extension] : " reference
position=$(sed -n '/:/p' $reference | cut -d':' -f2 | sed -n 1p)
done
fi

cd ..

#-----
#-----
#-----

# Generating test code and condition name for reporting from the genes
name.

mkdir new/

if [ $gene_name = "ADRB2" ]
then
test_code="PCR028"
condition_name="Beta2 adrenoreceptor"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "ATP7B" ]
then
test_code="WIL006"
condition_name="Wilson's Disease"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "HBB" ]
then
test_code="PCR027"
condition_name="Beta Thalassemia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GALNS" ]
then
test_code="MOR004"
condition_name="Morquio A Syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GJB2" ]
then
test_code="CON101"
condition_name="Non syndromic hearing loss"
echo $test_code > new/test_code.txt

```

```

echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GLB1" ]
then
test_code="MOR005"
condition_name="Morquio B Syndrome and GM1 Gangliosidosis"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TP53" ]
then
test_code="P53001"
condition_name="P53 mutations screen"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TWIST2" ]
then
test_code="SET001"
condition_name="Settleis syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WISP3" ]
then
test_code="PRO058"
condition_name="Progressive Pseudorheumatoid Dysplasia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WNT7A" ]
then
test_code="FUH001"
condition_name="Fuhrmann syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "UGT1A1" ]
then
test_code="GIL001 and GIL002"
condition_name="Gilbert's syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

fi

#-----
#-----
#-----

# From the genes name, obtaining the gene description from the Local Gene
Database.

cp
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/summa
ry_$gene_name.txt new/

#-----
#-----
#-----

# Taking the ABI file input from the user.

```



```

read -p "Enter Name of the Forward .ab1 file you wish to process : " ABI
if [ -f "$ABI" ] && [ "$ABI" != " " ]
then
echo "The entered Forward ABI filename is $ABI."
else
until [ -f "$ABI" ] && [ "$ABI" != " " ]
do
echo "The ABI file cannot be empty and is required by the MANIPAL System."
echo "Please provide the correct Forward ABI file : "
read ABI
done
fi

#-----
# Executing sangerseqR package to get the chromatogram.

R < sangerseqR.R --no-save

echo "                                Preparing FASTA file to use
as reference ..."

sed 's/:.*//' geneDB/$reference > $gene_name.fa

#II -->
#-----
# Creating directories for the files created during processing.

mkdir ABI_file fastq phd_dir seq_dir qual_dir

mv $ABI ABI_file/

echo "                                Base Calling ..."

phred -id ABI_file/ -sd seq_dir -qd qual_dir -pd phd_dir -trim_fasta -
trim_phd -trim ""
mv seq_dir/*.seq seq_dir/seqs_fasta.fasta
mv qual_dir/*.qual qual_dir/seqs_fasta.qual

echo "                                Building FASTQ ..."

perl fasta_qual2fastq.pl seq_dir/seqs_fasta.fasta qual_dir/seqs_fasta.qual
> fastq/merged.fastq

#III -->
echo "                                Aligning the Reads ..."

bwa mem ALL fastq/merged.fastq > result.sam

echo "                                Alignment Post-Processing
..."

```

```

samtools view -Sb result.sam > pipelineMINE.bam

samtools sort pipelineMINE.bam endMINE

samtools index endMINE.bam

samtools faidx $gene_name.fa

echo "                                Calling the Variants ..."

freebayes -f $gene_name.fa -C 1 endMINE.bam > freebayes.vcf

echo "                                Annotating the Variants ..."

perl convert2annovar.pl freebayes_GRCh37_FINAL_header.vcf -format vcf4 >
annovar.human

perl table_annovar.pl -buildver hg19 annovar.human -protocol
refGene,avsnpl42,clinvar_20150330,ljb26_all,popfreq_all_20150413 -operation
g,f,f,f,f humandb/
mkdir Result
mv annovar.human.hg19_multianno.txt Annovar_Result
mv Annovar_Result Result

cd Result
mv Annovar_Result Annovar_Result_$ABI.txt

cd ..

#-----
#-----
# BLAST Alignment

# II) If gene is on Positive STRAND!
# A- BLAST for 'FORWARD ABI file'-->

makeblastdb -in $gene_name.fa -dbtype nucl -out $gene_name

blastn -query seq_dir/seqs_fasta.fasta -db $gene_name | sed '1,30d' | sed -
n '/Query/,/Sbjct/p' | sed '/Query/ i\ ' > Result/$ABI.blastn

#sed 's/| |||@|/g' Result/$ABI.blastn | sed 's/| ||-|/g' | sed 's/|
|/|---|/g' | sed 's/| |||---|/g' | sed 's/| |||-----|/g' | sed
's/| |||@@@@@|/g' | sed 's/| |||@@@@@@@@|/g' >
Result/$ABI.blastn

find . -name "annovar.human*" -exec rm {} \;
find -maxdepth 1 -type f -name "*.log" -delete
find -maxdepth 1 -type f -name "*.bam" -delete
find -maxdepth 1 -type f -name "*.bai" -delete
find -maxdepth 1 -type f -name "*.sam" -delete
find -maxdepth 1 -type f -name "*.vcf" -delete
find -maxdepth 1 -type f -name "*.fa" -delete
find -maxdepth 1 -type f -name "*.fai" -delete
find -maxdepth 1 -type f -name "*.amb" -delete
find -maxdepth 1 -type f -name "*.ann" -delete

```

```

find -maxdepth 1 -type f -name "*.bwt" -delete
find -maxdepth 1 -type f -name "*.pac" -delete
find -maxdepth 1 -type f -name "*.sa" -delete
find -maxdepth 1 -type f -name "*.gz" -delete
find -maxdepth 1 -type f -name "*.sh~" -delete
find -maxdepth 1 -type f -name "*.nsq" -delete
find -maxdepth 1 -type f -name "*.nin" -delete
find -maxdepth 1 -type f -name "*.nhr" -delete

cd ABI_file

mv $ABI ../

cd ..

rm -rf ABI_file fastq qual_dir strand.txt

# V -->

echo "                                     Result ..."

echo "The Annotated Result of the '$ABI' ABI file is present in the Result
folder. "

#-----
-----
# Transferring the variables to reporting scripts and executing it.

echo $ABI > new/ABI.txt

echo $gene_name > new/gene_name.txt

bash annovar_human.sh

#-----
-----
# Option to repeat the Forward ABI file processing.

while true; do
read -p "                                     Do you wish to process any other
forward ABI file? " yn
case $yn in
[Yy]* ) bash MANIPAL_Genome_positive_forward.sh; break;;
[Nn]* ) echo
"~~~~~ Finished
Forward ABI file processing!
~~~~~"; exit;;
* ) echo "Please answer with 'yes' or 'no' ";;
esac
done

```

4.10.1.3 Processing forward ABI file for the negative strand gene

```

#!/bin/bash

echo "::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::

```

```

MANIPAL System
:....."

#I -->
#-----
-----

# Obtaining gene name from the user:

echo "    01. ABCB1      02. ADRB2      03. ATP7B      04. CTSC      05. GJB2
[alias CX26]
        06. GALNS      07. GLB1      08. HBB      09. KRAS      10. ATXN3
[alias MJD]
        11. TWIST2      12. UGT1A      13. WNT7A      14. WISP3      15.
TP53 [alias p53]
        16. HPV"

read -p "Enter any one of the displayed Gene Names: " gene_name
if [ ! -z $gene_name ]
then
    echo "The entered gene name is '$gene_name'. "
else

until [ -d "$dir" ] && [ "$dir" != " " ]
do
    echo "The Gene name cannot be empty and is required by the MANIPAL
System."
    echo "Please provide the Name of the Gene : "
    read gene_name
done
fi

#-----
-----

# To check for the existence of reference sequence and to obtain the start
co-ordinate of the gene.

cd geneDB/

if [ -e $gene_name.fasta ]
then
reference=$gene_name.fasta
else
echo "The entered Gene Reference Name is not correct."
until [ -e "$gene_name.fasta" ]
do
echo "    01. ABCB1      02. ADRB2      03. ATP7B      04. CTSC      05. GJB2
[alias CX26]
        06. GALNS      07. GLB1      08. HBB      09. KRAS      10. ATXN3
[alias MJD]
        11. TWIST2      12. UGT1A      13. WNT7A      14. WISP3      15.
TP53 [alias p53]
        16. HPV"

read -p "Please provide the correct Gene Name from the above list : "
gene_name
reference=$gene_name.fasta
done
fi

```

```

position=$(sed -n '/:/p' $reference | cut -d':' -f2 | sed -n 1p)
if [ ! -z "$position" ]
then
echo "The genomic start coordinate of the gene is '$position'."
else
until [ ! -z "$position" ]
do
echo "The genomic start coordinate of the Gene cannot be empty and is
required by the MANIPAL System."
read -p "Please provide the edited Gene Reference file [with .fasta
extension] : " reference
position=$(sed -n '/:/p' $reference | cut -d':' -f2 | sed -n 1p)
done
fi

cd ..

#-----
#-----
# Generating test code and condition name for reporting from the genes
name.

mkdir new/

if [ $gene_name = "ADRB2" ]
then
test_code="PCR028"
condition_name="Beta2 adrenoreceptor"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "ATP7B" ]
then
test_code="WIL006"
condition_name="Wilson's Disease"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "HBB" ]
then
test_code="PCR027"
condition_name="Beta Thalassemia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GALNS" ]
then
test_code="MOR004"
condition_name="Morquio A Syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GJB2" ]
then
test_code="CON101"
condition_name="Non syndromic hearing loss"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

```

```

elif [ $gene_name = "GLB1" ]
then
test_code="MOR005"
condition_name="Morquio B Syndrome and GM1 Gangliosidosis"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TP53" ]
then
test_code="P53001"
condition_name="P53 mutations screen"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TWIST2" ]
then
test_code="SET001"
condition_name="Settleis syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WISP3" ]
then
test_code="PRO058"
condition_name="Progressive Pseudorheumatoid Dysplasia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WNT7A" ]
then
test_code="FUH001"
condition_name="Fuhrmann syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "UGT1A1" ]
then
test_code="GIL001 and GIL002"
condition_name="Gilbert's syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

fi

#-----
#-----
#-----
# From the genes name, obtaining the gene description from the Local Gene
Database.

cp
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/summa
ry_$gene_name.txt new/

#-----
#-----
#-----
# Taking the ABI file input from the user.

```

```

read -p "Enter Name of the Forward .ab1 file relevant to the $reference
Reference Gene file : " ABI
if [ -f "$ABI" ] && [ "$ABI" != " " ]
then
echo "The entered Forward ABI filename is $ABI."
else
until [ -f "$ABI" ] && [ "$ABI" != " " ]
do
echo "The ABI file cannot be empty and is required by the MANIPAL System."
echo "Please provide the correct Forward ABI file relevant to the
$reference Reference Gene file : "
read ABI
done
fi

#-----
# Executing sangerseqR package to get the chromatogram.

R < sangerseqR.R --no-save

echo "                                Preparing FASTA file to use
as reference ..."

sed 's/:.*//' geneDB/$reference > $gene_name.fa

#II -->
#-----
# Creating directories for the files created during processing.

mkdir ABI_file fastq phd_dir seq_dir qual_dir

mv $ABI ABI_file/

echo "                                Base Calling ..."

#ttuner -id ABI_file -pd phd_dir -sd seq_dir

phred -id ABI_file/ -sd seq_dir -qd qual_dir -pd phd_dir -trim_fasta -
trim_phd -trim ""
mv seq_dir/*.seq seq_dir/seqs_fasta.fasta
mv qual_dir/*.qual qual_dir/seqs_fasta.qual

revseq -sequence seq_dir/seqs_fasta.fasta -outseq seq_dir/seqs_fasta.fasta

echo "                                Building FASTQ ..."

perl fasta_qual2fastq.pl seq_dir/seqs_fasta.fasta qual_dir/seqs_fasta.qual
> fastq/merged.fastq

#III -->

echo "                                Aligning the Reads ..."

```

```

bwa index -p ALL $gene_name.fa

bwa mem ALL fastq/merged.fastq > result.sam

echo "                                " Alignment Post-Processing
...                                "

samtools view -Sb result.sam > pipelineMINE.bam

samtools sort pipelineMINE.bam endMINE

samtools index endMINE.bam

samtools faidx $gene_name.fa

echo "                                " Calling the Variants ...
...                                "

freebayes -f $gene_name.fa -C 1 endMINE.bam > freebayes.vcf

#IV -->

echo "                                " VCF file processing ...
...                                "

sed '/^##/d' freebayes.vcf > freebayes_GRCh37_tail.vcf

#For FORWARD strand
awk -v s=$position '{$2+=s}1' OFS='\t' freebayes_GRCh37_tail.vcf >
freebayes_GRCh37_FINAL.vcf

bgzip freebayes_GRCh37_FINAL.vcf

#CREATE HEADER FILE for VCF Merging

tabix -r header.txt freebayes_GRCh37_FINAL.vcf.gz >
freebayes_GRCh37_FINAL_header.vcf.gz

gunzip freebayes_GRCh37_FINAL_header.vcf.gz

echo "                                " Annotating the Variants ...
...                                "

cd gene_humanDB

cp hg19_refGene$gene_name.txt ../humandb
#cp hg19_refGene$gene_name.txt.idx ../humandb

cp hg19_refGeneMrna$gene_name.fa ../humandb

#cp hg19_snp138$gene_name.txt ../humandb
#cp hg19_snp138$gene_name.txt.idx ../humandb

cp hg19_avsnp142$gene_name.txt ../humandb
cp hg19_avsnp142$gene_name.txt.idx ../humandb

cp hg19_clinvar_20150330$gene_name.txt ../humandb
cp hg19_clinvar_20150330$gene_name.txt.idx ../humandb

cd ..

```



```

cd humandb

mv hg19_refGene$gene_name.txt hg19_refGene.txt
#mv hg19_refGene$gene_name.txt.idx hg19_refGene.txt.idx

mv hg19_refGeneMrna$gene_name.fa hg19_refGeneMrna.fa

#mv hg19_snp138$gene_name.txt hg19_snp138.txt
#mv hg19_snp138$gene_name.txt.idx hg19_snp138.txt.idx

mv hg19_avsnp142$gene_name.txt hg19_avsnp142.txt
mv hg19_avsnp142$gene_name.txt.idx hg19_avsnp142.txt.idx

mv hg19_clinvar_20150330$gene_name.txt hg19_clinvar_20150330.txt
mv hg19_clinvar_20150330$gene_name.txt.idx hg19_clinvar_20150330.txt.idx

cd ..

perl convert2annovar.pl freebayes_GRCh37_FINAL_header.vcf -format vcf4 >
annovar.human

perl table_annovar.pl -buildver hg19 annovar.human -protocol
refGene,avsnp142,clinvar_20150330,ljb26_all,popfreq_all_20150413 -operation
g,f,f,f,f humandb/

cd humandb
# rm hg19_snp138.txt hg19_snp138.txt.idx
rm hg19_clinvar_20150330.txt.idx hg19_clinvar_20150330.txt
hg19_avsnp142.txt hg19_avsnp142.txt.idx hg19_refGeneMrna.fa
hg19_refGene.txt

cd ..

mkdir Result
mv annovar.human.hg19_multianno.txt Annovar_Result
mv Annovar_Result Result

cd Result
mv Annovar_Result Annovar_Result_$ABI.txt

cd ..

#-----
# BLAST Alignment

# II) If gene is on NEGATIVE STRAND!

# A- BLAST for 'FORWARD ABI file'-->

makeblastdb -in geneDB/comp_$gene_name.fasta -dbtype nucl -out $gene_name

blastn -query seq_dir/seqs_fasta.fasta -db $gene_name | sed '1,30d' | sed -
n '/Query/,/Sbjct/p' | sed '/Query/ i\ ' > Result/$ABI.blastn

#sed 's/| | |@|/g' Result/$ABI.blastn | sed 's/| | |@@|/g' | sed 's/|
| | |@@@|/g' | sed 's/| | |@@@@|/g' | sed 's/| | |@@@@@|/g' | sed
's/| | |@@@@@@|/g' | sed 's/| | |@@@@@@@|/g' >
Result/$ABI.blastn

```

```

find -maxdepth 1 -type f -name "*.log" -delete
find -maxdepth 1 -type f -name "*.bam" -delete
find -maxdepth 1 -type f -name "*.bai" -delete
find -maxdepth 1 -type f -name "*.sam" -delete
find -maxdepth 1 -type f -name "*.vcf" -delete
find -maxdepth 1 -type f -name "*.fa" -delete
find -maxdepth 1 -type f -name "*.fai" -delete
find -maxdepth 1 -type f -name "*.amb" -delete
find -maxdepth 1 -type f -name "*.ann" -delete
find -maxdepth 1 -type f -name "*.bwt" -delete
find -maxdepth 1 -type f -name "*.pac" -delete
find -maxdepth 1 -type f -name "*.sa" -delete
find -maxdepth 1 -type f -name "*.gz" -delete
find -maxdepth 1 -type f -name "*.sh~" -delete
find -maxdepth 1 -type f -name "*.nsq" -delete
find -maxdepth 1 -type f -name "*.nin" -delete
find -maxdepth 1 -type f -name "*.nhr" -delete

cd ABI_file

mv $ABI ../

cd ..

rm -rf ABI_file fastq qual_dir strand.txt

# V -->

echo "                                     Result ..."

                                     "

echo "The Annotated Result of the '$ABI' ABI file is present in the Result
folder. "

#-----
-----
-----
# Transferring the variables to reporting scripts and executing it.

echo $ABI > new/ABI.txt

echo $gene_name > new/gene_name.txt

bash annovar_human.sh

#-----
-----
-----
# Option to repeat the Forward ABI file processing.

while true; do
    read -p "                                     Do you wish to process any
other ABI file? " yn
    case $yn in
        [Yy]* ) bash MANIPAL_Genome_negative_forward.sh; break;;
        [Nn]* ) echo
"~~~~~ Finished
Forward ABI file processing!

```

```
~~~~~"; exit;;
    * ) echo "Please answer with 'yes' or 'no' ";;
    esac
done
```

4.10.1.4 Determining the gene orientation for reverse ABI file processing

```
#!/bin/bash

echo "::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
MANIPAL System
::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::"

#I -->
# TAKING INPUT FROM USER:

echo "    01. ABCB1      02. ADRB2      03. ATP7B      04. CTSC      05. GJB2
[alias CX26]
    06. GALNS      07. GLB1      08. HBB      09. KRAS      10. ATXN3
[alias MJD]
    11. TWIST2      12. UGT1A      13. WNT7A      14. WISP3      15.
TP53 [alias p53]
    16. HPV"

read -p "Enter any one of the above displayed Gene Names or any other Gene
Name of your choice : " gene_name
if [ ! -z $gene_name ]
then
    echo "The entered gene name is '$gene_name'. "
else
    until [ -d "$dir" ] && [ "$dir" != " " ]
    do
        echo "The Gene name cannot be empty and is required by the MANIPAL
System."
        echo "Please provide the Name of the Gene : "
        read gene_name
    done
fi

#cat
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/hg19_
refGene$gene.txt | awk '{if ($4 == "+") print $4;}' > strand.txt

awk '{print $4}'
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/hg19_
refGene$gene_name.txt | sed -n '1p' > strand.txt

if grep -Fqx "+" strand.txt ; then bash MANIPAL_Genome_positive_reverse.sh
else bash MANIPAL_Genome_negative_reverse.sh
fi
```

4.10.1.5 Processing reverse ABI file for the positive strand gene

```
#!/bin/bash

echo "::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
MANIPAL System
::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::"

#I -->
#-----
-----

# Obtaining gene name from the user:

echo "    01. ABCB1      02. ADRB2      03. ATP7B      04. CTSC      05. GJB2
[alias CX26]
    06. GALNS      07. GLB1      08. HBB      09. KRAS      10. ATXN3
[alias MJD]
    11. TWIST2      12. UGT1A      13. WNT7A      14. WISP3      15.
TP53 [alias p53]
    16. HPV"

read -p "Enter any one of the above displayed Gene Names or any other Gene
Name of your choice : " gene_name
if [ ! -z $gene_name ]
then
    echo "The entered gene name is '$gene_name'. "
else

until [ -d "$dir" ] && [ "$dir" != " " ]
do
    echo "The Gene name cannot be empty and is required by the MANIPAL
System."
    echo "Please provide the Name of the Gene : "
    read gene_name
done
fi

#-----
-----

# To check for the existence of reference sequence and to obtain the start
co-ordinate of the gene.

cd geneDB/

if [ -e $gene_name.fasta ]
then
reference=$gene_name.fasta
else
echo "The entered Gene Reference Name is not correct."
until [ -e "$gene_name.fasta" ]
do
echo "    01. ABCB1      02. ADRB2      03. ATP7B      04. CTSC      05. GJB2
[alias CX26]
    06. GALNS      07. GLB1      08. HBB      09. KRAS      10. ATXN3
[alias MJD]
    11. TWIST2      12. UGT1A      13. WNT7A      14. WISP3      15.
```

```

TP53 [alias p53]
16. HPV"

read -p "Please provide the correct Gene Name from the above list : "
gene_name
reference=$gene_name.fasta
done
fi

position=$(sed -n '/:/p' $reference | cut -d':' -f2 | sed -n 1p)
if [ ! -z "$position" ]
then
echo "The genomic start coordinate of the gene is '$position'."
else
until [ ! -z "$position" ]
do
echo "The genomic start coordinate of the Gene cannot the empty and is
required by the MANIPAL System."
read -p "Please provide the edited Gene Reference file [with .fasta
extension] : " reference
position=$(sed -n '/:/p' $reference | cut -d':' -f2 | sed -n 1p)
done
fi

cd ..

#-----
#-----
# Generating test code and condition name for reporting from the genes
name.

mkdir new/

if [ $gene_name = "ADRB2" ]
then
test_code="PCR028"
condition_name="Beta2 adrenoreceptor"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "ATP7B" ]
then
test_code="WIL006"
condition_name="Wilson's Disease"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "HBB" ]
then
test_code="PCR027"
condition_name="Beta Thalassemia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GALNS" ]
then
test_code="MOR004"
condition_name="Morquio A Syndrome"
echo $test_code > new/test_code.txt

```

```

echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GJB2" ]
then
test_code="CON101"
condition_name="Non syndromic hearing loss"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GLB1" ]
then
test_code="MOR005"
condition_name="Morquio B Syndrome and GM1 Gangliosidosis"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TP53" ]
then
test_code="P53001"
condition_name="P53 mutations screen"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TWIST2" ]
then
test_code="SET001"
condition_name="Settleis syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WISP3" ]
then
test_code="PRO058"
condition_name="Progressive Pseudorheumatoid Dysplasia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WNT7A" ]
then
test_code="FUH001"
condition_name="Fuhrmann syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "UGT1A1" ]
then
test_code="GIL001 and GIL002"
condition_name="Gilbert's syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

fi

#-----
#-----
#-----
# From the genes name, obtaining the gene description from the Local Gene
Database.

cp

```

```

/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/summary_
$gene_name.txt new/
#-----
-----
# Taking the ABI file input from the user.

read -p "Enter Name of the Forward .ab1 file you wish to process : " ABI
if [ -f "$ABI" ] && [ "$ABI" != " " ]
then
echo "The entered Forward ABI filename is $ABI."
else
until [ -f "$ABI" ] && [ "$ABI" != " " ]
do
echo "The ABI file cannot be empty and is required by the MANIPAL System."
echo "Please provide the correct Forward ABI file : "
read ABI
done
fi

#-----
-----
# Executing sangerseqR package to get the chromatogram.

R < sangerseqR.R --no-save

echo "                                Preparing FASTA file to use
as reference ..."

sed 's/:.*//' geneDB/$reference > $gene_name.fa

#II -->
#-----
-----
# Creating directories for the files created during processing.

mkdir ABI_file fastq phd_dir seq_dir qual_dir

mv $ABI ABI_file/

echo "                                Base Calling ..."

#ttuner -id ABI_file -pd phd_dir -sd seq_dir

phred -id ABI_file/ -sd seq_dir -qd qual_dir -pd phd_dir -trim_fasta -
trim_phd -trim ""
mv seq_dir/*.seq seq_dir/seqs_fasta.fasta
mv qual_dir/*.qual qual_dir/seqs_fasta.qual

revseq -sequence seq_dir/seqs_fasta.fasta -outseq seq_dir/seqs_fasta.fasta

echo "                                Building FASTQ ..."

perl fasta_qual2fastq.pl seq_dir/seqs_fasta.fasta qual_dir/seqs_fasta.qual
> fastq/merged.fastq

```

```

#III -->

echo "                                " Aligning the Reads ...
bwa mem ALL fastq/merged.fastq > result.sam

echo "                                " Alignment Post-Processing
...                                "

samtools view -Sb result.sam > pipelineMINE.bam

samtools sort pipelineMINE.bam endMINE

samtools index endMINE.bam
echo "                                " Calling the Variants ...

freebayes -f $gene_name.fa -C 1 endMINE.bam > freebayes.vcf

echo "                                " Annotating the Variants ...

perl convert2annovar.pl freebayes_GRCh37_FINAL_header.vcf -format vcf4 >
annovar.human

perl table_annovar.pl -buildver hg19 annovar.human -protocol
refGene,avsnp142,clinvar_20150330,ljb26_all,popfreq_all_20150413 -operation
g,f,f,f,f humandb/

mkdir Result
mv annovar.human.hg19_multianno.txt Annovar_Result
mv Annovar_Result Result

cd Result
mv Annovar_Result Annovar_Result_${ABI}.txt

cd ..

#-----
#-----
# BLAST Alignment

# II) If gene is on Positive STRAND!

# B- BLAST for 'REVERSE ABI file'

makeblastdb -in geneDB/comp_$gene_name.fasta -dbtype nucl -out $gene_name

blastn -query seq_dir/seqs_fasta.fasta -db $gene_name | sed '1,30d' | sed -
n '/Query/,/Sbjct/p' | sed '/Query/ i\ ' > Result/${ABI}.blastn

find . -name "annovar.human*" -exec rm {} \;
find -maxdepth 1 -type f -name "*.log" -delete
find -maxdepth 1 -type f -name "*.bam" -delete
find -maxdepth 1 -type f -name "*.bai" -delete
find -maxdepth 1 -type f -name "*.sam" -delete

```



```

find -maxdepth 1 -type f -name "*.vcf" -delete
find -maxdepth 1 -type f -name "*.fa" -delete
find -maxdepth 1 -type f -name "*.fai" -delete
find -maxdepth 1 -type f -name "*.amb" -delete
find -maxdepth 1 -type f -name "*.ann" -delete
find -maxdepth 1 -type f -name "*.bwt" -delete
find -maxdepth 1 -type f -name "*.pac" -delete
find -maxdepth 1 -type f -name "*.sa" -delete
find -maxdepth 1 -type f -name "*.gz" -delete
find -maxdepth 1 -type f -name "*.sh~" -delete
find -maxdepth 1 -type f -name "*.nsq" -delete
find -maxdepth 1 -type f -name "*.nin" -delete
find -maxdepth 1 -type f -name "*.nhr" -delete

cd ABI_file

mv $ABI ../

cd ..

rm -rf ABI_file fastq qual_dir strand.txt

# V -->

echo "                                Result ..."

echo "The Annotated Result of the '$ABI' ABI file is present in the Result
folder. "

#-----
-----
-----
# Transferring the variables to reporting scripts and executing it.

echo $ABI > new/ABI.txt

echo $gene_name > new/gene_name.txt

bash annovar_human.sh

#-----
-----
-----
# Option to repeat the Forward ABI file processing.

while true; do
    read -p "                                Do you wish to process any
other ABI file? " yn
    case $yn in
        [Yy]* ) bash MANIPAL_Genome_positive_reverse.sh; break;;
        [Nn]* ) echo
"~~~~~ Finished
Reverse ABI file processing!
~~~~~"; exit;;
        * ) echo "Please answer with 'yes' or 'no' ";;
    esac
done

```

4.10.1.6 Processing reverse ABI file for the negative strand gene

```
#!/bin/bash

echo "::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
MANIPAL System
::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::"

#I -->
#-----
-----

# Obtaining gene name from the user:

echo "    01. ABCB1      02. ADRB2      03. ATP7B      04. CTSC      05. GJB2
[alias CX26]
    06. GALNS      07. GLB1      08. HBB      09. KRAS      10. ATXN3
[alias MJD]
    11. TWIST2      12. UGT1A      13. WNT7A      14. WISP3      15.
TP53 [alias p53]
    16. HPV"

read -p "Enter any one of the displayed Gene Names: " gene_name
if [ ! -z $gene_name ]
then
    echo "The entered gene name is '$gene_name'. "
else
    until [ -d "$dir" ] && [ "$dir" != " " ]
    do
        echo "The Gene name cannot be empty and is required by the MANIPAL
System."
        echo "Please provide the Name of the Gene : "
        read gene_name
    done
fi

#-----
-----

# To check for the existence of reference sequence and to obtain the start
co-ordinate of the gene.

cd geneDB/

if [ -e $gene_name.fasta ]
then
    reference=$gene_name.fasta
else
    echo "The entered Gene Reference Name is not correct."
    until [ -e "$gene_name.fasta" ]
    do
        echo "    01. ABCB1      02. ADRB2      03. ATP7B      04. CTSC      05. GJB2
[alias CX26]
    06. GALNS      07. GLB1      08. HBB      09. KRAS      10. ATXN3
[alias MJD]
    11. TWIST2      12. UGT1A      13. WNT7A      14. WISP3      15.
TP53 [alias p53]
    16. HPV"
```

```

read -p "Please provide the correct Gene Name from the above list : "
gene_name
reference=$gene_name.fasta
done
fi

position=$(sed -n ':/p' $reference | cut -d':' -f2 | sed -n 1p)
if [ ! -z "$position" ]
then
echo "The genomic start coordinate of the gene is '$position'."
else
until [ ! -z "$position" ]
do
echo "The genomic start coordinate of the Gene cannot be empty and is
required by the MANIPAL System."
read -p "Please provide the edited Gene Reference file [with .fasta
extension] : " reference
position=$(sed -n ':/p' $reference | cut -d':' -f2 | sed -n 1p)
done
fi

cd ..

#-----
#-----
# Generating test code and condition name for reporting from the genes
name.

mkdir new/

if [ $gene_name = "ADRB2" ]
then
test_code="PCR028"
condition_name="Beta2 adrenoreceptor"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "ATP7B" ]
then
test_code="WIL006"
condition_name="Wilson's Disease"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "HBB" ]
then
test_code="PCR027"
condition_name="Beta Thalassemia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GALNS" ]
then
test_code="MOR004"
condition_name="Morquio A Syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GJB2" ]

```

```

then
test_code="CON101"
condition_name="Non syndromic hearing loss"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "GLB1" ]
then
test_code="MOR005"
condition_name="Morquio B Syndrome and GM1 Gangliosidosis"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TP53" ]
then
test_code="P53001"
condition_name="P53 mutations screen"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "TWIST2" ]
then
test_code="SET001"
condition_name="Settleis syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WISP3" ]
then
test_code="PRO058"
condition_name="Progressive Pseudorheumatoid Dysplasia"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "WNT7A" ]
then
test_code="FUH001"
condition_name="Fuhrmann syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

elif [ $gene_name = "UGT1A1" ]
then
test_code="GIL001 and GIL002"
condition_name="Gilbert's syndrome"
echo $test_code > new/test_code.txt
echo $condition_name > new/condition_name.txt

fi

#-----
#-----
# From the genes name, obtaining the gene description from the Local Gene
Database.

cp
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/summa
ry_$gene_name.txt new/

```

```

#-----
# Taking the ABI file input from the user.

read -p "Enter Name of the Reverse .ab1 file relevant to the $reference
Reference Gene file : " ABI
if [ -f "$ABI" ] && [ "$ABI" != " " ]
then
echo "The entered Reverse ABI filename is $ABI."
else
until [ -f "$ABI" ] && [ "$ABI" != " " ]
do
echo "The ABI file cannot be empty and is required by the MANIPAL System."
echo "Please provide the correct Reverse ABI file relevant to the
$reference Reference Gene file : "
read ABI
done
fi

#-----
# Executing sangerseqR package to get the chromatogram.

R < sangerseqR.R --no-save

echo "                                Preparing FASTA file to use
as reference ..."

sed 's/.*://' geneDB/$reference > $gene_name.fa

#II -->
#-----
# Creating directories for the files created during processing.

mkdir ABI_file fastq phd_dir seq_dir qual_dir

mv $ABI ABI_file/

echo "                                Base Calling ..."

#ttuner -id ABI_file -pd phd_dir -sd seq_dir

phred -id ABI_file/ -sd seq_dir -qd qual_dir -pd phd_dir -trim_fasta -
trim_phd -trim ""

mv seq_dir/*.seq seq_dir/seqs_fasta.fasta

mv qual_dir/*.qual qual_dir/seqs_fasta.qual

#revseq -sequence seq_dir/seqs_fasta.fasta -outseq seq_dir/seqs_fasta.fasta

echo "                                Building FASTQ ..."

```

```

perl fasta_qual2fastq.pl seq_dir/seqs_fasta.fasta qual_dir/seqs_fasta.qual
> fastq/merged.fastq

#III -->

echo "                                " Aligning the Reads ...

bwa index -p ALL $gene_name.fa

bwa mem ALL fastq/merged.fastq > result.sam

echo "                                " Alignment Post-Processing
...

samtools view -Sb result.sam > pipelineMINE.bam

samtools sort pipelineMINE.bam endMINE

samtools index endMINE.bam

samtools faidx $gene_name.fa

echo "                                " Calling the Variants ...

freebayes -f $gene_name.fa -C 1 endMINE.bam > freebayes.vcf

#IV -->

echo "                                " VCF file processing ...

sed '/^##/d' freebayes.vcf > freebayes_GRCh37_tail.vcf

#For FORWARD strand
awk -v s=$position '{$2+=s}1' OFS='\t' freebayes_GRCh37_tail.vcf >
freebayes_GRCh37_FINAL.vcf

bgzip freebayes_GRCh37_FINAL.vcf

#CREATE HEADER FILE for VCF Merging

tabix -r header.txt freebayes_GRCh37_FINAL.vcf.gz >
freebayes_GRCh37_FINAL_header.vcf.gz

gunzip freebayes_GRCh37_FINAL_header.vcf.gz

echo "                                " Annotating the Variants ...

cd gene_humanDB

cp hg19_refGene$gene_name.txt ../humandb
#cp hg19_refGene$gene_name.txt.idx ../humandb

cp hg19_refGeneMrna$gene_name.fa ../humandb

#cp hg19_snp138$gene_name.txt ../humandb

```

```

#cp hg19_snp138$gene_name.txt.idx ../humandb

cp hg19_avsnp142$gene_name.txt ../humandb
cp hg19_avsnp142$gene_name.txt.idx ../humandb

cp hg19_clinvar_20150330$gene_name.txt ../humandb
cp hg19_clinvar_20150330$gene_name.txt.idx ../humandb

cd ..
cd humandb

mv hg19_refGene$gene_name.txt hg19_refGene.txt
#mv hg19_refGene$gene_name.txt.idx hg19_refGene.txt.idx

mv hg19_refGeneMrna$gene_name.fa hg19_refGeneMrna.fa

#mv hg19_snp138$gene_name.txt hg19_snp138.txt
#mv hg19_snp138$gene_name.txt.idx hg19_snp138.txt.idx

mv hg19_avsnp142$gene_name.txt hg19_avsnp142.txt
mv hg19_avsnp142$gene_name.txt.idx hg19_avsnp142.txt.idx

mv hg19_clinvar_20150330$gene_name.txt hg19_clinvar_20150330.txt
mv hg19_clinvar_20150330$gene_name.txt.idx hg19_clinvar_20150330.txt.idx

cd ..

perl convert2annovar.pl freebayes_GRCh37_FINAL_header.vcf -format vcf4 >
annovar.human

perl table_annovar.pl -buildver hg19 annovar.human -protocol
refGene,avsnp142,clinvar_20150330,ljb26_all,popfreq_all_20150413 -operation
g,f,f,f,f humandb/

cd humandb

# rm hg19_snp138.txt hg19_snp138.txt.idx

rm hg19_clinvar_20150330.txt.idx hg19_clinvar_20150330.txt
hg19_avsnp142.txt hg19_avsnp142.txt.idx hg19_refGeneMrna.fa
hg19_refGene.txt

cd ..

mkdir Result

mv annovar.human.hg19_multianno.txt Annovar_Result

mv Annovar_Result Result

cd Result

mv Annovar_Result Annovar_Result_$ABI.txt

cd ..

#-----
# BLAST Alignment

```

```

# II) If gene is on NEGATIVE STRAND!

# A- BLAST for 'FORWARD ABI file'-->

makeblastdb -in geneDB/comp_${gene_name}.fasta -dbtype nucl -out $gene_name

blastn -query seq_dir/seqs_fasta.fasta -db $gene_name | sed '1,30d' | sed -
n '/Query/,/Sbjct/p' | sed '/Query/ i\ ' > Result/$ABI.blastn

#sed 's/| |||@|/g' Result/$ABI.blastn | sed 's/| |||@@|/g' | sed 's/|
|||@@@@|/g' | sed 's/| |||@@@@|/g' | sed 's/| |||@@@@@|/g' | sed
's/| |||@@@@@@|/g' | sed 's/| |||@@@@@@@|/g' >
Result/$ABI.blastn

find . -name "annovar.human*" -exec rm {} \;
find -maxdepth 1 -type f -name "*.log" -delete
find -maxdepth 1 -type f -name "*.bam" -delete
find -maxdepth 1 -type f -name "*.bai" -delete
find -maxdepth 1 -type f -name "*.sam" -delete
find -maxdepth 1 -type f -name "*.vcf" -delete
find -maxdepth 1 -type f -name "*.fa" -delete
find -maxdepth 1 -type f -name "*.fai" -delete
find -maxdepth 1 -type f -name "*.amb" -delete
find -maxdepth 1 -type f -name "*.ann" -delete
find -maxdepth 1 -type f -name "*.bwt" -delete
find -maxdepth 1 -type f -name "*.pac" -delete
find -maxdepth 1 -type f -name "*.sa" -delete
find -maxdepth 1 -type f -name "*.gz" -delete
find -maxdepth 1 -type f -name "*.sh~" -delete
find -maxdepth 1 -type f -name "*.nsq" -delete
find -maxdepth 1 -type f -name "*.nin" -delete
find -maxdepth 1 -type f -name "*.nhr" -delete

cd ABI_file

mv $ABI ../

cd ..

rm -rf ABI_file fastq qual_dir strand.txt

# V -->

echo "                                Result ..."

"

echo "The Annotated Result of the '$ABI' ABI file is present in the Result
folder. "

#-----
-----
-----

# Transferring the variables to reporting scripts and executing it.

echo $ABI > new/ABI.txt

echo $gene_name > new/gene_name.txt

bash annovar_human.sh

```



```

#-----
# Option to repeat the Reverse ABI file processing.

while true; do
    read -p "                                Do you wish to process any
other ABI file? " yn
    case $yn in
        [Yy]* ) bash negative_reverse_MANIPAL.sh; break;;
        [Nn]* ) echo
"~~~~~ Finished
Reverse ABI file processing!
~~~~~"; exit;;
        * ) echo "Please answer with 'yes' or 'no' ";;
    esac
done

```

4.10.2 Secondary analysis

These scripts are mainly concerned with creating the genomic report from the annotated tabular file of ANNOVAR.

4.10.2.1 Extracting essential elements from ANNOVAR output

```

#!/bin/bash

#-----
# MAIN --> To pass the value of the number of significant variants to a
variable.

clinsig=$(cat annovar.human.hg19_clinvar_20150330_dropped | wc -l)

#-----
# To generate POSITIVE or NEGATIVE result.

if [ $clinsig == 0 ]
then
echo "<table bgcolor="LightGreen" border="1" cellpadding="10">
<tr>
    <td width=1500 align="center"><strong>RESULT: Negative - No variants
were identified in the $gene_name gene for causing the diagnosed
condition.</strong></td>
</tr></table>" > new/Result.html

elif [ $clinsig == 1 ]
then
echo "<table bgcolor="IndianRed" border="1" cellpadding="10">
<tr>
    <td width=1500 align="center"><strong>RESULT: Positive - A known
variant causing the diagnosed condition was identified in the $gene_name

```

```

gene.</strong></td>
</tr></table>" > new/Result.html

elif [ $clinsig > 1 ]
then
echo "<table bgcolor="IndianRed" border="1" cellpadding="10">
<tr>
<td width=1500 align="center"><strong>RESULT: Positive - Known variants
causing the diagnosed condition were identified in the $gene_name
gene.</strong></td>
</tr></table>" > new/Result.html

fi

#-----
#-----
#To get clinvar disease name

clinvar_diseases=$(cat annovar.human.hg19_clinvar_20150330_dropped | cut -
d=' ' -f3 | cut -d '|' -f1 | cut -d ';' -f1 | wc -l)

if [ $clinvar_diseases == 0 ]
then
echo "No clinically relevant diseases were identified for the
diagnosed condition." > new/clinvar_diseases.txt
elif [ $clinvar_diseases == 1 ]
then
cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d ';' -f1 | sed -n '1p' | sed 's/_/ /g' >
new/clinvar_diseases_1_1.txt
clinvar_diseases_1_1=$(cat new/clinvar_diseases_1_1.txt)

elif [ $clinvar_diseases == 2 ]
then
cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d ';' -f1 | sed -n '1p' | sed 's/_/ /g' >
new/clinvar_diseases_1_2.txt
cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d ';' -f1 | sed -n '2p' | sed 's/_/ /g' >
new/clinvar_diseases_2_2.txt
clinvar_diseases_1_2=$(cat new/clinvar_diseases_1_2.txt)
clinvar_diseases_2_2=$(cat new/clinvar_diseases_2_2.txt)

elif [ $clinvar_diseases == 3 ]
then
cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d ';' -f1 | sed -n '1p' | sed 's/_/ /g' >
new/clinvar_diseases_1_3.txt
cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d ';' -f1 | sed -n '2p' | sed 's/_/ /g' >
new/clinvar_diseases_2_3.txt
cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d ';' -f1 | sed -n '3p' | sed 's/_/ /g' >
new/clinvar_diseases_3_3.txt
clinvar_diseases_1_3=$(cat new/clinvar_diseases_1_3.txt)
clinvar_diseases_2_3=$(cat new/clinvar_diseases_2_3.txt)
clinvar_diseases_3_3=$(cat new/clinvar_diseases_3_3.txt)

elif [ $clinvar_diseases == 4 ]

```

```

    then
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '1p' | sed 's/_/ /g' >
new/clinvar_diseases_1_4.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '2p' | sed 's/_/ /g' >
new/clinvar_diseases_2_4.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '3p' | sed 's/_/ /g' >
new/clinvar_diseases_3_4.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '4p' | sed 's/_/ /g' >
new/clinvar_diseases_4_4.txt
        clinvar_diseases_1_4=$(cat new/clinvar_diseases_1_4.txt)
        clinvar_diseases_2_4=$(cat new/clinvar_diseases_2_4.txt)
        clinvar_diseases_3_4=$(cat new/clinvar_diseases_3_4.txt)
        clinvar_diseases_4_4=$(cat new/clinvar_diseases_4_4.txt)

elif [ $clinvar_diseases == 5 ]
    then
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '1p' | sed 's/_/ /g' >
new/clinvar_diseases_1_5.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '2p' | sed 's/_/ /g' >
new/clinvar_diseases_2_5.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '3p' | sed 's/_/ /g' >
new/clinvar_diseases_3_5.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '4p' | sed 's/_/ /g' >
new/clinvar_diseases_4_5.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d=' ' -f3 | cut
-d '|' -f1 | cut -d';' -f1 | sed -n '5p' | sed 's/_/ /g' >
new/clinvar_diseases_5_5.txt
        clinvar_diseases_1_5=$(cat new/clinvar_diseases_1_5.txt)
        clinvar_diseases_2_5=$(cat new/clinvar_diseases_2_5.txt)
        clinvar_diseases_3_5=$(cat new/clinvar_diseases_3_5.txt)
        clinvar_diseases_4_5=$(cat new/clinvar_diseases_4_5.txt)
        clinvar_diseases_5_5=$(cat new/clinvar_diseases_5_5.txt)

fi

#-----
#-----
# to get clinvar classification. Whether pathogenic or non-pathogenic.

clinvar_classification=$(cat annovar.human.hg19_clinvar_20150330_dropped |
cut -d' ' -f2 | cut -d=' ' -f2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut
-d';' -f1 | wc -l)

if [ $clinvar_classification == 0 ]
    then
        echo "No clinically significant classification was found for the
diagnosed condition." > new/clinvar_classification.txt

elif [ $clinvar_classification == 1 ]
    then
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f2 |

```

```

cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '1p' > new/clinvar_classification_1_1.txt
    clinvar_classification_1_1=$(cat new/clinvar_classification_1_1.txt)

elif [ $clinvar_classification == 2 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '1p' > new/clinvar_classification_1_2.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '2p' > new/clinvar_classification_2_2.txt
    clinvar_classification_1_2=$(cat new/clinvar_classification_1_2.txt)
    clinvar_classification_2_2=$(cat new/clinvar_classification_2_2.txt)

elif [ $clinvar_classification == 3 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '1p' > new/clinvar_classification_1_3.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '2p' > new/clinvar_classification_2_3.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '3p' > new/clinvar_classification_3_3.txt
    clinvar_classification_1_3=$(cat new/clinvar_classification_1_3.txt)
    clinvar_classification_2_3=$(cat new/clinvar_classification_2_3.txt)
    clinvar_classification_3_3=$(cat new/clinvar_classification_3_3.txt)

elif [ $clinvar_classification == 4 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '1p' > new/clinvar_classification_1_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '2p' > new/clinvar_classification_2_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '3p' > new/clinvar_classification_3_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '4p' > new/clinvar_classification_4_4.txt
    clinvar_classification_1_4=$(cat new/clinvar_classification_1_4.txt)
    clinvar_classification_2_4=$(cat new/clinvar_classification_2_4.txt)
    clinvar_classification_3_4=$(cat new/clinvar_classification_3_4.txt)
    clinvar_classification_4_4=$(cat new/clinvar_classification_4_4.txt)

elif [ $clinvar_classification == 5 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '1p' > new/clinvar_classification_1_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '2p' > new/clinvar_classification_2_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '3p' > new/clinvar_classification_3_5.txt

```

```

        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '4p' > new/clinvar_classification_4_5.txt
        cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f2 |
cut -d'=' -f 2 | cut -d"|" -f1 | sed -e 's/./\U&/' | cut -d';' -f1 | sed -
n '5p' > new/clinvar_classification_5_5.txt
        clinvar_classification_1_5=$(cat new/clinvar_classification_1_5.txt)
        clinvar_classification_2_5=$(cat new/clinvar_classification_2_5.txt)
        clinvar_classification_3_5=$(cat new/clinvar_classification_3_5.txt)
        clinvar_classification_4_5=$(cat new/clinvar_classification_4_5.txt)
        clinvar_classification_5_5=$(cat new/clinvar_classification_5_5.txt)

fi

#-----
#-----
# To get chr

if [ $clinsig == 0 ]
then
    echo "There are no Clinically significant mutations."

elif [ $clinsig == 1 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '1p' > new/chr_1_1.txt

elif [ $clinsig == 2 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '1p' > new/chr_1_2.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '2p' > new/chr_2_2.txt

elif [ $clinsig == 3 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '1p' > new/chr_1_2.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '2p' > new/chr_2_2.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '3p' > new/chr_3_2.txt

elif [ $clinsig == 4 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '1p' > new/chr_1_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '2p' > new/chr_2_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '3p' > new/chr_3_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '4p' > new/chr_4_4.txt

elif [ $clinsig == 5 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |
sed -n '1p' > new/chr_1_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d' ' -f3 |

```

```

sed -n '2p' > new/chr_2_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f3 |
sed -n '3p' > new/chr_3_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f3 |
sed -n '4p' > new/chr_4_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f3 |
sed -n '5p' > new/chr_5_5.txt

fi

#####
#####
#####
# To get gDNA start

if [ $clinsig == 0 ]
then
    echo "There are no Clinically significant mutations."

elif [ $clinsig == 1 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '1p' > new/gDNA_start_1_1.txt

elif [ $clinsig == 2 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '1p' > new/gDNA_start_1_2.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '2p' > new/gDNA_start_2_2.txt

elif [ $clinsig == 3 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '1p' > new/gDNA_start_1_3.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '2p' > new/gDNA_start_2_3.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '3p' > new/gDNA_start_3_3.txt

elif [ $clinsig == 4 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '1p' > new/gDNA_start_1_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '2p' > new/gDNA_start_2_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '3p' > new/gDNA_start_3_4.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '4p' > new/gDNA_start_4_4.txt

elif [ $clinsig == 5 ]
then
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '1p' > new/gDNA_start_1_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '2p' > new/gDNA_start_2_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |
sed -n '3p' > new/gDNA_start_3_5.txt
    cat annovar.human.hg19_clinvar_20150330_dropped | cut -d'      ' -f4 |

```

[illegible]


```

^^^^^^^^^^^^^^^^^^^^
# To combine chr and gDNA start

if [ $clinsig == 0 ]
then
echo "There are no Clinically significant mutations."

elif [ $clinsig == 1 ]
then
cat new/chr_1_1.txt new/gDNA_start_1_1.txt >
new/chr_1_1_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_1_1_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_1_1.txt

elif [ $clinsig == 2 ]
then
cat new/chr_1_2.txt new/gDNA_start_1_2.txt >
new/chr_1_2_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_1_2_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_1_2.txt
cat new/chr_2_2.txt new/gDNA_start_2_2.txt >
new/chr_2_2_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_2_2_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_2_2.txt

elif [ $clinsig == 3 ]
then
cat new/chr_1_3.txt new/gDNA_start_1_3.txt >
new/chr_1_3_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_1_3_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_1_3.txt
cat new/chr_2_3.txt new/gDNA_start_2_3.txt >
new/chr_2_3_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_2_3_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_2_3.txt
cat new/chr_3_3.txt new/gDNA_start_3_3.txt >
new/chr_3_3_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_3_3_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_3_3.txt

elif [ $clinsig == 4 ]
then
cat new/chr_1_4.txt new/gDNA_start_1_4.txt >
new/chr_1_4_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_1_4_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_1_4.txt
cat new/chr_2_4.txt new/gDNA_start_2_4.txt >
new/chr_2_4_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_2_4_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_2_4.txt
cat new/chr_3_4.txt new/gDNA_start_3_4.txt >
new/chr_3_4_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_3_4_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_3_4.txt
cat new/chr_4_4.txt new/gDNA_start_4_4.txt >
new/chr_4_4_gDNA_start.txt
awk 'ORS=NR%2?FS:RS' new/chr_4_4_gDNA_start.txt | sed 's/ /:/g' >
new/chr_gDNA_start_COMBINED_4_4.txt

elif [ $clinsig == 5 ]

```



```

> new/GENOMIC_LOCATION_of_variant_2_2.txt
    GENOMIC_LOCATION_of_variant_1_2=$(cat
new/GENOMIC_LOCATION_of_variant_1_2.txt)
    GENOMIC_LOCATION_of_variant_2_2=$(cat
new/GENOMIC_LOCATION_of_variant_2_2.txt)

elif [ $clinsig == 3 ]
then
    cat new/ref_1_3.txt new/alt_1_3.txt > new/ref_alt_1_3.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_1_3.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_1_3.txt
    cat new/chr_gDNA_start_COMBINED_1_3.txt new/ref_alt_COMBINED_1_3.txt >
new/chr_gDNA_ref_alt_FINAL_1_3.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_1_3.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_1_3.txt
    cat new/ref_2_3.txt new/alt_2_3.txt > new/ref_alt_2_3.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_2_3.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_2_3.txt
    cat new/chr_gDNA_start_COMBINED_2_3.txt new/ref_alt_COMBINED_2_3.txt >
new/chr_gDNA_ref_alt_FINAL_2_3.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_2_3.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_2_3.txt
    cat new/ref_3_3.txt new/alt_3_3.txt > new/ref_alt_3_3.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_3_3.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_3_3.txt
    cat new/chr_gDNA_start_COMBINED_3_3.txt new/ref_alt_COMBINED_3_3.txt >
new/chr_gDNA_ref_alt_FINAL_3_3.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_3_3.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_3_3.txt
    GENOMIC_LOCATION_of_variant_1_3=$(cat
new/GENOMIC_LOCATION_of_variant_1_3.txt)
    GENOMIC_LOCATION_of_variant_2_3=$(cat
new/GENOMIC_LOCATION_of_variant_2_3.txt)
    GENOMIC_LOCATION_of_variant_3_3=$(cat
new/GENOMIC_LOCATION_of_variant_3_3.txt)

elif [ $clinsig == 4 ]
then
    cat new/ref_1_4.txt new/alt_1_4.txt > new/ref_alt_1_4.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_1_4.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_1_4.txt
    cat new/chr_gDNA_start_COMBINED_1_4.txt new/ref_alt_COMBINED_1_4.txt >
new/chr_gDNA_ref_alt_FINAL_1_4.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_1_4.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_1_4.txt
    cat new/ref_2_4.txt new/alt_2_4.txt > new/ref_alt_2_4.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_2_4.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_2_4.txt
    cat new/chr_gDNA_start_COMBINED_2_4.txt new/ref_alt_COMBINED_2_4.txt >
new/chr_gDNA_ref_alt_FINAL_2_4.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_2_4.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_2_4.txt
    cat new/ref_3_4.txt new/alt_3_4.txt > new/ref_alt_3_4.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_3_4.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_3_4.txt
    cat new/chr_gDNA_start_COMBINED_3_4.txt new/ref_alt_COMBINED_3_4.txt >
new/chr_gDNA_ref_alt_FINAL_3_4.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_3_4.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_3_4.txt
    cat new/ref_4_4.txt new/alt_4_4.txt > new/ref_alt_4_4.txt

```

```

    awk 'ORS=NR%2?FS:RS' new/ref_alt_4_4.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_4_4.txt
    cat new/chr_gDNA_start_COMBINED_4_4.txt new/ref_alt_COMBINED_4_4.txt >
new/chr_gDNA_ref_alt_FINAL_4_4.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_4_4.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_4_4.txt
    GENOMIC_LOCATION_of_variant_1_4=$(cat
new/GENOMIC_LOCATION_of_variant_1_4.txt)
    GENOMIC_LOCATION_of_variant_2_4=$(cat
new/GENOMIC_LOCATION_of_variant_2_4.txt)
    GENOMIC_LOCATION_of_variant_3_4=$(cat
new/GENOMIC_LOCATION_of_variant_3_4.txt)
    GENOMIC_LOCATION_of_variant_4_4=$(cat
new/GENOMIC_LOCATION_of_variant_4_4.txt)

elif [ $clinsig == 5 ]
then
    cat new/ref_1_5.txt new/alt_1_5.txt > new/ref_alt_1_5.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_1_5.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_1_5.txt
    cat new/chr_gDNA_start_COMBINED_1_5.txt new/ref_alt_COMBINED_1_5.txt >
new/chr_gDNA_ref_alt_FINAL_1_5.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_1_5.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_1_5.txt
    cat new/ref_2_5.txt new/alt_2_5.txt > new/ref_alt_2_5.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_2_5.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_2_5.txt
    cat new/chr_gDNA_start_COMBINED_2_5.txt new/ref_alt_COMBINED_2_5.txt >
new/chr_gDNA_ref_alt_FINAL_2_5.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_2_5.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_2_5.txt
    cat new/ref_3_5.txt new/alt_3_5.txt > new/ref_alt_3_5.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_3_5.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_3_5.txt
    cat new/chr_gDNA_start_COMBINED_3_5.txt new/ref_alt_COMBINED_3_5.txt >
new/chr_gDNA_ref_alt_FINAL_3_5.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_3_5.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_3_5.txt
    cat new/ref_4_5.txt new/alt_4_5.txt > new/ref_alt_4_5.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_4_5.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_4_5.txt
    cat new/chr_gDNA_start_COMBINED_4_5.txt new/ref_alt_COMBINED_4_5.txt >
new/chr_gDNA_ref_alt_FINAL_4_5.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_4_5.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_4_5.txt
    cat new/ref_5_5.txt new/alt_5_5.txt > new/ref_alt_5_5.txt
    awk 'ORS=NR%2?FS:RS' new/ref_alt_5_5.txt | sed 's/ />/g' >
new/ref_alt_COMBINED_5_5.txt
    cat new/chr_gDNA_start_COMBINED_5_5.txt new/ref_alt_COMBINED_5_5.txt >
new/chr_gDNA_ref_alt_FINAL_5_5.txt
    awk 'ORS=NR%2?FS:RS' new/chr_gDNA_ref_alt_FINAL_5_5.txt | sed 's/ //g'
> new/GENOMIC_LOCATION_of_variant_5_5.txt
    GENOMIC_LOCATION_of_variant_1_5=$(cat
new/GENOMIC_LOCATION_of_variant_1_5.txt)
    GENOMIC_LOCATION_of_variant_2_5=$(cat
new/GENOMIC_LOCATION_of_variant_2_5.txt)
    GENOMIC_LOCATION_of_variant_3_5=$(cat
new/GENOMIC_LOCATION_of_variant_3_5.txt)
    GENOMIC_LOCATION_of_variant_4_5=$(cat
new/GENOMIC_LOCATION_of_variant_4_5.txt)

```

```

GENOMIC_LOCATION_of_variant_5_5=$(cat
new/GENOMIC_LOCATION_of_variant_5_5.txt)

fi

#-----
-----

# To compare the start position of the clinically significant mutation from
clinvar.dropped and the start position from the refGene.exonic file

clinsig_position_1=$(cut -d' ' -f4
annovar.human.hg19_clinvar_20150330_dropped | sed -n '1p')
clinsig_position_2=$(cut -d' ' -f4
annovar.human.hg19_clinvar_20150330_dropped | sed -n '2p')
clinsig_position_3=$(cut -d' ' -f4
annovar.human.hg19_clinvar_20150330_dropped | sed -n '3p')
clinsig_position_4=$(cut -d' ' -f4
annovar.human.hg19_clinvar_20150330_dropped | sed -n '4p')
clinsig_position_5=$(cut -d' ' -f4
annovar.human.hg19_clinvar_20150330_dropped | sed -n '5p')

#-----
-----

# For extracting GENE NAME from annovar.human.refGene.variant_function
file.

if [[ $clinsig == 1 ]]
then
    cat annovar.human.refGene.variant_function | grep
"$clinsig_position_1" | cut -d' ' -f2 > gene_name_1.txt
    gene_name_1=$(cat gene_name_1.txt)

elif [[ $clinsig == 2 ]]
then
    cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f2 > gene_name_1.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_2
| cut -d' ' -f2 > gene_name_2.txt
    gene_name_1=$(cat gene_name_1.txt)
    gene_name_2=$(cat gene_name_2.txt)

elif [[ $clinsig == 3 ]]
then
    cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f2 > gene_name_1.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_2
| cut -d' ' -f2 > gene_name_2.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_3
| cut -d' ' -f2 > gene_name_3.txt
    gene_name_1=$(cat gene_name_1.txt)
    gene_name_2=$(cat gene_name_2.txt)
    gene_name_3=$(cat gene_name_3.txt)

elif [[ $clinsig == 4 ]]
then
    cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f2 > gene_name_1.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_2

```

```

| cut -d' ' -f2 > gene_name_2.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_3
| cut -d' ' -f2 > gene_name_3.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_4
| cut -d' ' -f2 > gene_name_4.txt
    gene_name_1=$(cat gene_name_1.txt)
    gene_name_2=$(cat gene_name_2.txt)
    gene_name_3=$(cat gene_name_3.txt)
    gene_name_4=$(cat gene_name_4.txt)

elif [[ $clinsig == 5 ]]
then
    cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f2 > gene_name_1.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_2
| cut -d' ' -f2 > gene_name_2.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_3
| cut -d' ' -f2 > gene_name_3.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_4
| cut -d' ' -f2 > gene_name_4.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_5
| cut -d' ' -f2 > gene_name_5.txt
    gene_name_1=$(cat gene_name_1.txt)
    gene_name_2=$(cat gene_name_2.txt)
    gene_name_3=$(cat gene_name_3.txt)
    gene_name_4=$(cat gene_name_4.txt)
    gene_name_5=$(cat gene_name_5.txt)

fi

#-----
#-----
#-----
# For extracting VARIANT LOCATION from
annovar.human.refGene.variant_function file.

if [[ $clinsig == 1 ]]
then
    cat annovar.human.refGene.variant_function | grep
"$clinsig_position_1" | cut -d' ' -f1 > variant_location_1.txt
    variant_location_1=$(cat variant_location_1.txt)

elif [[ $clinsig == 2 ]]
then
    cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f1 > variant_location_1.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_2
| cut -d' ' -f1 > variant_location_2.txt
    variant_location_1=$(cat variant_location_1.txt)
    variant_location_2=$(cat variant_location_2.txt)

elif [[ $clinsig == 3 ]]
then
    cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f1 > variant_location_1.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_2
| cut -d' ' -f1 > variant_location_2.txt
    cat annovar.human.refGene.variant_function | grep $clinsig_position_3
| cut -d' ' -f1 > variant_location_3.txt
    variant_location_1=$(cat variant_location_1.txt)

```

```

variant_location_2=$(cat variant_location_2.txt)
variant_location_3=$(cat variant_location_3.txt)

elif [[ $clinsig == 4 ]]
then
cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f1 > variant_location_1.txt
cat annovar.human.refGene.variant_function | grep $clinsig_position_2
| cut -d' ' -f1 > variant_location_2.txt
cat annovar.human.refGene.variant_function | grep $clinsig_position_3
| cut -d' ' -f1 > variant_location_3.txt
cat annovar.human.refGene.variant_function | grep $clinsig_position_4
| cut -d' ' -f1 > variant_location_4.txt
variant_location_1=$(cat variant_location_1.txt)
variant_location_2=$(cat variant_location_2.txt)
variant_location_3=$(cat variant_location_3.txt)
variant_location_4=$(cat variant_location_4.txt)

elif [[ $clinsig == 5 ]]
then
cat annovar.human.refGene.variant_function | grep $clinsig_position_1
| cut -d' ' -f1 > variant_location_1.txt
cat annovar.human.refGene.variant_function | grep $clinsig_position_2
| cut -d' ' -f1 > variant_location_2.txt
cat annovar.human.refGene.variant_function | grep $clinsig_position_3
| cut -d' ' -f1 > variant_location_3.txt
cat annovar.human.refGene.variant_function | grep $clinsig_position_4
| cut -d' ' -f1 > variant_location_4.txt
cat annovar.human.refGene.variant_function | grep $clinsig_position_5
| cut -d' ' -f1 > variant_location_5.txt
variant_location_1=$(cat variant_location_1.txt)
variant_location_2=$(cat variant_location_2.txt)
variant_location_3=$(cat variant_location_3.txt)
variant_location_4=$(cat variant_location_4.txt)
variant_location_5=$(cat variant_location_5.txt)

fi

#-----
#-----
#-----

# To get VARIANT TYPE from annovar.human.refGene.exonic_variant_function
file.

if [[ $clinsig == 1 ]]
then
cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f2 > variant_type_1.txt
variant_type_1=$(cat variant_type_1.txt)

elif [[ $clinsig == 2 ]]
then
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f2 > variant_type_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f2 > variant_type_2.txt
variant_type_1=$(cat variant_type_1.txt)
variant_type_2=$(cat variant_type_2.txt)

elif [[ $clinsig == 3 ]]

```

```

        then
            cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f2 > variant_type_1.txt
            cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f2 > variant_type_2.txt
            cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f2 > variant_type_3.txt
            variant_type_1=$(cat variant_type_1.txt)
            variant_type_2=$(cat variant_type_2.txt)
            variant_type_3=$(cat variant_type_3.txt)

elif [[ $clinsig == 4 ]]
then
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f2 > variant_type_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f2 > variant_type_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f2 > variant_type_3.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f2 > variant_type_4.txt
    variant_type_1=$(cat variant_type_1.txt)
    variant_type_2=$(cat variant_type_2.txt)
    variant_type_3=$(cat variant_type_3.txt)
    variant_type_4=$(cat variant_type_4.txt)

elif [[ $clinsig == 5 ]]
then
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f2 > variant_type_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f2 > variant_type_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f2 > variant_type_3.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f2 > variant_type_4.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_5 | cut -d' ' -f2 > variant_type_5.txt
    variant_type_1=$(cat variant_type_1.txt)
    variant_type_2=$(cat variant_type_2.txt)
    variant_type_3=$(cat variant_type_3.txt)
    variant_type_4=$(cat variant_type_4.txt)
    variant_type_5=$(cat variant_type_5.txt)

fi

#-----
#-----
#-----

# To get dbSNP Identifier from annovar.human.hg19_avsnp142_dropped file.

if [[ $clinsig == 1 ]]
then
    cat annovar.human.hg19_avsnp142_dropped | grep "$clinsig_position_1"
| cut -d' ' -f2 > rsID_1.txt
    rsID_1=$(cat rsID_1.txt)

elif [[ $clinsig == 2 ]]
then
    cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_1 |

```



```

cut -d' ' -f2 > rsID_1.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_2 |
cut -d' ' -f2 > rsID_2.txt
rsID_1=$(cat rsID_1.txt)
rsID_2=$(cat rsID_2.txt)

elif [[ $clinsig == 3 ]]
then
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_1 |
cut -d' ' -f2 > rsID_1.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_2 |
cut -d' ' -f2 > rsID_2.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_3 |
cut -d' ' -f2 > rsID_3.txt
rsID_1=$(cat rsID_1.txt)
rsID_2=$(cat rsID_2.txt)
rsID_3=$(cat rsID_3.txt)

elif [[ $clinsig == 4 ]]
then
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_1 |
cut -d' ' -f2 > rsID_1.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_2 |
cut -d' ' -f2 > rsID_2.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_3 |
cut -d' ' -f2 > rsID_3.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_4 |
cut -d' ' -f2 > rsID_4.txt
rsID_1=$(cat rsID_1.txt)
rsID_2=$(cat rsID_2.txt)
rsID_3=$(cat rsID_3.txt)
rsID_4=$(cat rsID_4.txt)

elif [[ $clinsig == 5 ]]
then
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_1 |
cut -d' ' -f2 > rsID_1.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_2 |
cut -d' ' -f2 > rsID_2.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_3 |
cut -d' ' -f2 > rsID_3.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_4 |
cut -d' ' -f2 > rsID_4.txt
cat annovar.human.hg19_avsnp142_dropped | grep $clinsig_position_5 |
cut -d' ' -f2 > rsID_5.txt
rsID_1=$(cat rsID_1.txt)
rsID_2=$(cat rsID_2.txt)
rsID_3=$(cat rsID_3.txt)
rsID_4=$(cat rsID_4.txt)
rsID_5=$(cat rsID_5.txt)

fi

#-----
#-----
#-----
#To get the MAF score from annovar.human.hg19_popfreq_all_20150413_dropped
file for the clinically significant variants.

if [[ $clinsig == 1 ]]

```

```

        then
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_1"| cut -d' ' -f2 | cut -d',' -f2 > maf_1.txt
        maf_1=$(cat maf_1.txt)

elif [[ $clinsig == 2 ]]
then
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_1"| cut -d' ' -f2 | cut -d',' -f2 > maf_1.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_2"| cut -d' ' -f2 | cut -d',' -f2 > maf_2.txt
        maf_1=$(cat maf_1.txt)
        maf_2=$(cat maf_2.txt)

elif [[ $clinsig == 3 ]]
then
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_1"| cut -d' ' -f2 | cut -d',' -f2 > maf_1.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_2"| cut -d' ' -f2 | cut -d',' -f2 > maf_2.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_3"| cut -d' ' -f2 | cut -d',' -f2 > maf_3.txt
        maf_1=$(cat maf_1.txt)
        maf_2=$(cat maf_2.txt)
        maf_3=$(cat maf_3.txt)

elif [[ $clinsig == 4 ]]
then
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_1"| cut -d' ' -f2 | cut -d',' -f2 > maf_1.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_2"| cut -d' ' -f2 | cut -d',' -f2 > maf_2.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_3"| cut -d' ' -f2 | cut -d',' -f2 > maf_3.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_4"| cut -d' ' -f2 | cut -d',' -f2 > maf_4.txt
        maf_1=$(cat maf_1.txt)
        maf_2=$(cat maf_2.txt)
        maf_3=$(cat maf_3.txt)
        maf_4=$(cat maf_4.txt)

elif [[ $clinsig == 5 ]]
then
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_1"| cut -d' ' -f2 | cut -d',' -f2 > maf_1.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_2"| cut -d' ' -f2 | cut -d',' -f2 > maf_2.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_3"| cut -d' ' -f2 | cut -d',' -f2 > maf_3.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_4"| cut -d' ' -f2 | cut -d',' -f2 > maf_4.txt
        cat annovar.human.hg19_popfreq_all_20150413_dropped |grep
"$clinsig_position_5"| cut -d' ' -f2 | cut -d',' -f2 > maf_5.txt
        maf_1=$(cat maf_1.txt)
        maf_2=$(cat maf_2.txt)
        maf_3=$(cat maf_3.txt)
        maf_4=$(cat maf_4.txt)
        maf_5=$(cat maf_5.txt)

fi

```

```

#-----
#-----
#To get the SIFT score from annovar.human.hg19_ljb26_all_dropped file for
the clinically significant variants.

if [[ $clinsig == 1 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_1.txt
sift_score_1=$(cat sift_score_1.txt)

elif [[ $clinsig == 2 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_2.txt
sift_score_1=$(cat sift_score_1.txt)
sift_score_2=$(cat sift_score_2.txt)

elif [[ $clinsig == 3 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_3.txt
sift_score_1=$(cat sift_score_1.txt)
sift_score_2=$(cat sift_score_2.txt)
sift_score_3=$(cat sift_score_3.txt)

elif [[ $clinsig == 4 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_3.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_4.txt
sift_score_1=$(cat sift_score_1.txt)
sift_score_2=$(cat sift_score_2.txt)
sift_score_3=$(cat sift_score_3.txt)
sift_score_4=$(cat sift_score_4.txt)

elif [[ $clinsig == 5 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_3.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f1 > sift_score_4.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_5"|

```

```

cut -d' ' -f2 | cut -d',' -f1 > sift_score_5.txt
sift_score_1=$(cat sift_score_1.txt)
sift_score_2=$(cat sift_score_2.txt)
sift_score_3=$(cat sift_score_3.txt)
sift_score_4=$(cat sift_score_4.txt)
sift_score_5=$(cat sift_score_5.txt)

fi

#-----
-----

#To get the SIFT predication from annovar.human.hg19_ljb26_all_dropped file
for the clinically significant variants.

if [[ $clinsig == 1 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_1.txt
sift_pred_1=$(cat sift_pred_1.txt)
sift_pred_1=$(
if [[ $sift_pred_1 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_1.txt
elif [[ $sift_pred_1 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_1.txt
fi)

#sift_final_pred_1=$(if [[ $sift_pred_1 == "T" ]]; then sed
's/T/Tolerated/g' sift_pred_1.txt; fi)

elif [[ $clinsig == 2 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_2.txt
sift_pred_1=$(cat sift_pred_1.txt)
sift_pred_1=$(
if [[ $sift_pred_1 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_1.txt
elif [[ $sift_pred_1 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_1.txt
fi)

sift_pred_2=$(cat sift_pred_2.txt)
sift_pred_2=$(
if [[ $sift_pred_2 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_2.txt
elif [[ $sift_pred_2 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_2.txt
fi)

elif [[ $clinsig == 3 ]]
then

```

```

        cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_1.txt
        cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_2.txt
        cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_3.txt
sift_pred_1=$(cat sift_pred_1.txt)
sift_pred_1=$(
if [[ $sift_pred_1 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_1.txt
elif [[ $sift_pred_1 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_1.txt
fi)
sift_pred_2=$(cat sift_pred_2.txt)
sift_pred_2=$(
if [[ $sift_pred_2 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_2.txt
elif [[ $sift_pred_2 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_2.txt
fi)

sift_pred_3=$(cat sift_pred_3.txt)
sift_pred_3=$(
if [[ $sift_pred_3 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_3.txt
elif [[ $sift_pred_3 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_3.txt
fi)

elif [[ $clinsig == 4 ]]
then
        cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_1.txt
        cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_2.txt
        cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_3.txt
        cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_4.txt
sift_pred_1=$(cat sift_pred_1.txt)
sift_pred_1=$(
if [[ $sift_pred_1 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_1.txt
elif [[ $sift_pred_1 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_1.txt
fi)

sift_pred_2=$(cat sift_pred_2.txt)
sift_pred_2=$(
if [[ $sift_pred_2 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_2.txt

```

```

elif [[ $sift_pred_2 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_2.txt
fi)

sift_pred_3=$(cat sift_pred_3.txt)
sift_pred_3=$(
if [[ $sift_pred_3 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_3.txt
elif [[ $sift_pred_3 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_3.txt
fi)

sift_pred_4=$(cat sift_pred_4.txt)
sift_pred_4=$(
if [[ $sift_pred_4 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_4.txt
elif [[ $sift_pred_4 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_4.txt
fi)

elif [[ $clinsig == 5 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_3.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_4.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_5"|
cut -d' ' -f2 | cut -d',' -f2 > sift_pred_5.txt
sift_pred_1=$(cat sift_pred_1.txt)
sift_pred_1=$(
if [[ $sift_pred_1 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_1.txt
elif [[ $sift_pred_1 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_1.txt
fi)

sift_pred_2=$(cat sift_pred_2.txt)
sift_pred_2=$(
if [[ $sift_pred_2 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_2.txt
elif [[ $sift_pred_2 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_2.txt
fi)

sift_pred_3=$(cat sift_pred_3.txt)
sift_pred_3=$(
if [[ $sift_pred_3 == "T" ]]

```

```

then
sed 's/T/Tolerated/g' sift_pred_3.txt
elif [[ $sift_pred_3 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_3.txt
fi)

sift_pred_4=$(cat sift_pred_4.txt)
sift_pred_4=$(
if [[ $sift_pred_4 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_4.txt
elif [[ $sift_pred_4 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_4.txt
fi)

sift_pred_5=$(cat sift_pred_5.txt)
sift_pred_5=$(
if [[ $sift_pred_5 == "T" ]]
then
sed 's/T/Tolerated/g' sift_pred_5.txt
elif [[ $sift_pred_5 == "D" ]]
then
sed 's/D/Deleterious/g' sift_pred_5.txt
fi)

fi

#-----
#-----
#-----
#To get the Polyphen prediction from annovar.human.hg19_ljb26_all_dropped
file for the clinically significant variants.

if [[ $clinsig == 1 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_1.txt
polyphen_pred_1=$(cat polyphen_pred_1.txt)
polyphen_pred_1=$(
if [[ $polyphen_pred_1 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

elif [[ $clinsig == 2 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_2.txt
polyphen_pred_1=$(cat polyphen_pred_1.txt)
polyphen_pred_1=$(

```

```

if [[ $polyphen_pred_1 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

polyphen_pred_2=$(cat polyphen_pred_2.txt)
polyphen_pred_2=$(
if [[ $polyphen_pred_2 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_2.txt
elif [[ $polyphen_pred_2 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_2.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

elif [[ $clinsig == 3 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_3.txt
polyphen_pred_1=$(cat polyphen_pred_1.txt)
polyphen_pred_1=$(
if [[ $polyphen_pred_1 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

polyphen_pred_2=$(cat polyphen_pred_2.txt)
polyphen_pred_2=$(
if [[ $polyphen_pred_2 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_2.txt
elif [[ $polyphen_pred_2 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_2.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

polyphen_pred_3=$(cat polyphen_pred_3.txt)
polyphen_pred_3=$(

```



```

        if [[ $polyphen_pred_3 == "P" ]]
        then
            sed 's/P/Possibly damaging/g' polyphen_pred_3.txt
        elif [[ $polyphen_pred_3 == "D" ]]
        then
            sed 's/D/Probably damaging/g' polyphen_pred_3.txt
        elif [[ $polyphen_pred_1 == "B" ]]
        then
            sed 's/B/Benign/g' polyphen_pred_1.txt
        fi)

elif [[ $clinsig == 4 ]]
then
    cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_1.txt
    cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_2.txt
    cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_3.txt
    cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_4.txt
    polyphen_pred_1=$(cat polyphen_pred_1.txt)
    polyphen_pred_1=$(
        if [[ $polyphen_pred_1 == "P" ]]
        then
            sed 's/P/Possibly damaging/g' polyphen_pred_1.txt
        elif [[ $polyphen_pred_1 == "D" ]]
        then
            sed 's/D/Probably damaging/g' polyphen_pred_1.txt
        elif [[ $polyphen_pred_1 == "B" ]]
        then
            sed 's/B/Benign/g' polyphen_pred_1.txt
        fi)

    polyphen_pred_2=$(cat polyphen_pred_2.txt)
    polyphen_pred_2=$(
        if [[ $polyphen_pred_2 == "P" ]]
        then
            sed 's/P/Possibly damaging/g' polyphen_pred_2.txt
        elif [[ $polyphen_pred_2 == "D" ]]
        then
            sed 's/D/Probably damaging/g' polyphen_pred_2.txt
        elif [[ $polyphen_pred_1 == "B" ]]
        then
            sed 's/B/Benign/g' polyphen_pred_1.txt
        fi)

    polyphen_pred_3=$(cat polyphen_pred_3.txt)
    polyphen_pred_3=$(
        if [[ $polyphen_pred_3 == "P" ]]
        then
            sed 's/P/Possibly damaging/g' polyphen_pred_3.txt
        elif [[ $polyphen_pred_3 == "D" ]]
        then
            sed 's/D/Probably damaging/g' polyphen_pred_3.txt
        elif [[ $polyphen_pred_1 == "B" ]]
        then
            sed 's/B/Benign/g' polyphen_pred_1.txt
        fi)

```

```

polyphen_pred_4=$(cat polyphen_pred_4.txt)
polyphen_pred_4=$(
if [[ $polyphen_pred_4 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_4.txt
elif [[ $polyphen_pred_4 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_4.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

elif [[ $clinsig == 5 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_3.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_4.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_5"|
cut -d' ' -f2 | cut -d',' -f6 > polyphen_pred_5.txt
polyphen_pred_1=$(cat polyphen_pred_1.txt)
polyphen_pred_1=$(
if [[ $polyphen_pred_1 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_1.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

polyphen_pred_2=$(cat polyphen_pred_2.txt)
polyphen_pred_2=$(
if [[ $polyphen_pred_2 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_2.txt
elif [[ $polyphen_pred_2 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_2.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

polyphen_pred_3=$(cat polyphen_pred_3.txt)
polyphen_pred_3=$(
if [[ $polyphen_pred_3 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_3.txt
elif [[ $polyphen_pred_3 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_3.txt
elif [[ $polyphen_pred_1 == "B" ]]

```

```

then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

polyphen_pred_4=$(cat polyphen_pred_4.txt)
polyphen_pred_4=$(
if [[ $polyphen_pred_4 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_4.txt
elif [[ $polyphen_pred_4 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_4.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

polyphen_pred_5=$(cat polyphen_pred_5.txt)
polyphen_pred_5=$(
if [[ $polyphen_pred_5 == "P" ]]
then
sed 's/P/Possibly damaging/g' polyphen_pred_5.txt
elif [[ $polyphen_pred_5 == "D" ]]
then
sed 's/D/Probably damaging/g' polyphen_pred_5.txt
elif [[ $polyphen_pred_1 == "B" ]]
then
sed 's/B/Benign/g' polyphen_pred_1.txt
fi)

fi

#-----
#-----
#-----

#To get the Polyphen score from annovar.human.hg19_ljb26_all_dropped file
for the clinically significant variants.

if [[ $clinsig == 1 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_1.txt
polyphen_score_1=$(cat polyphen_score_1.txt)

elif [[ $clinsig == 2 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_2.txt
polyphen_score_1=$(cat polyphen_score_1.txt)
polyphen_score_2=$(cat polyphen_score_2.txt)

elif [[ $clinsig == 3 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|

```

```

cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_3.txt
polyphen_score_1=$(cat polyphen_score_1.txt)
polyphen_score_2=$(cat polyphen_score_2.txt)
polyphen_score_3=$(cat polyphen_score_3.txt)

elif [[ $clinsig == 4 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_3.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_4.txt
polyphen_score_1=$(cat polyphen_score_1.txt)
polyphen_score_2=$(cat polyphen_score_2.txt)
polyphen_score_3=$(cat polyphen_score_3.txt)
polyphen_score_4=$(cat polyphen_score_4.txt)

elif [[ $clinsig == 5 ]]
then
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_1"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_1.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_2"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_2.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_3"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_3.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_4"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_4.txt
cat annovar.human.hg19_ljb26_all_dropped |grep "$clinsig_position_5"|
cut -d' ' -f2 | cut -d',' -f5 > polyphen_score_5.txt
polyphen_score_1=$(cat polyphen_score_1.txt)
polyphen_score_2=$(cat polyphen_score_2.txt)
polyphen_score_3=$(cat polyphen_score_3.txt)
polyphen_score_4=$(cat polyphen_score_4.txt)
polyphen_score_5=$(cat polyphen_score_5.txt)

fi

#-----
#-----

#For extracting complementary variant location

if [[ $clinsig == 1 ]]
then
cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_1.txt
comp_variant_firstT_1=$(cat comp_variant_firstT_1.txt)

cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_1.txt
comp_variant_secondT_1=$(cat comp_variant_secondT_1.txt)

elif [[ $clinsig == 2 ]]
then
cat annovar.human.refGene.exonic_variant_function | grep

```

```

"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_2.txt
    comp_variant_firstT_1=$(cat comp_variant_firstT_1.txt)
    comp_variant_firstT_2=$(cat comp_variant_firstT_2.txt)

    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_2.txt
    comp_variant_secondT_1=$(cat comp_variant_secondT_1.txt)
    comp_variant_secondT_2=$(cat comp_variant_secondT_2.txt)

elif [[ $clinsig == 3 ]]
then
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_3" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_3.txt
    comp_variant_firstT_1=$(cat comp_variant_firstT_1.txt)
    comp_variant_firstT_2=$(cat comp_variant_firstT_2.txt)
    comp_variant_firstT_3=$(cat comp_variant_firstT_3.txt)

    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_3" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_3.txt
    comp_variant_secondT_1=$(cat comp_variant_secondT_1.txt)
    comp_variant_secondT_2=$(cat comp_variant_secondT_2.txt)
    comp_variant_secondT_3=$(cat comp_variant_secondT_3.txt)

elif [[ $clinsig == 4 ]]
then
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_3" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_3.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_4" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_4.txt
    comp_variant_firstT_1=$(cat comp_variant_firstT_1.txt)

```

```

    comp_variant_firstT_2=$(cat comp_variant_firstT_2.txt)
    comp_variant_firstT_3=$(cat comp_variant_firstT_3.txt)
    comp_variant_firstT_4=$(cat comp_variant_firstT_4.txt)

    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_3" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_3.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_4" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_4.txt
    comp_variant_secondT_1=$(cat comp_variant_secondT_1.txt)
    comp_variant_secondT_2=$(cat comp_variant_secondT_2.txt)
    comp_variant_secondT_3=$(cat comp_variant_secondT_3.txt)
    comp_variant_secondT_4=$(cat comp_variant_secondT_4.txt)

elif [[ $clinsig == 5 ]]
then
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_3" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_3.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_4" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_4.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_5" | cut -d' ' -f3 | cut -d',' -f1 |cut -d':' -f4 >
comp_variant_firstT_5.txt
    comp_variant_firstT_1=$(cat comp_variant_firstT_1.txt)
    comp_variant_firstT_2=$(cat comp_variant_firstT_2.txt)
    comp_variant_firstT_3=$(cat comp_variant_firstT_3.txt)
    comp_variant_firstT_4=$(cat comp_variant_firstT_4.txt)
    comp_variant_firstT_5=$(cat comp_variant_firstT_5.txt)

    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_1" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_2" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_3" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_3.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_4" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_4.txt
    cat annovar.human.refGene.exonic_variant_function | grep
"$clinsig_position_5" | cut -d' ' -f3 | cut -d',' -f2 |cut -d':' -f4 >
comp_variant_secondT_5.txt
    comp_variant_secondT_1=$(cat comp_variant_secondT_1.txt)

```



```

transcript_id_firstT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d'    ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_2.txt

    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d'    ' -f3 |cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d'    ' -f3 |cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_1.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d'    ' -f3 |cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_2.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d'    ' -f3 |cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_2.txt

transcript_id_firstT_1=$(cat transcript_id_firstT_1.txt)
transcript_id_secondT_1=$(cat transcript_id_secondT_1.txt)
transcript_id_firstT_2=$(cat transcript_id_firstT_2.txt)
transcript_id_secondT_2=$(cat transcript_id_secondT_2.txt)

aachange_firstT_1=$(cat aachange_firstT_1.txt)
aachange_secondT_1=$(cat aachange_secondT_1.txt)
aachange_firstT_2=$(cat aachange_firstT_2.txt)
aachange_secondT_2=$(cat aachange_secondT_2.txt)

#*****
*****
*****
if [ ! -z $aachange_firstT_1 ]
then
    bash AA_expand_firstT_1.sh
else
    echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_1 ]
then
    bash AA_expand_secondT_1.sh
else
    echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_firstA_firstT_1.txt new/
aachange_firstA_firstT_1=$(cat new/aachange_firstA_firstT_1.txt)
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_lastA_firstT_1.txt new/
aachange_lastA_firstT_1=$(cat new/aachange_lastA_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]

```

```

then
mv aachange_firstA_secondT_1.txt new/
aachange_firstA_secondT_1=$(cat new/aachange_firstA_secondT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_lastA_secondT_1.txt new/
aachange_lastA_secondT_1=$(cat new/aachange_lastA_secondT_1.txt)

fi

if [ ! -z $transcript_id_firstT_1 ]
then
mv transcript_id_firstT_1.txt new/
transcript_id_firstT_1=$(cat new/transcript_id_firstT_1.txt)
fi

if [ ! -z $transcript_id_secondT_1 ]
then
mv transcript_id_secondT_1.txt new/
transcript_id_secondT_1=$(cat new/transcript_id_secondT_1.txt)
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_firstT_1.txt new/
aachange_firstT_1=$(cat new/aachange_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_secondT_1.txt new/
aachange_secondT_1=$(cat new/aachange_secondT_1.txt)
fi

#####
#####
#####
if [ ! -z $aachange_firstT_2 ]
then
bash AA_expand_firstT_2.sh
else
echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_2 ]
then
bash AA_expand_secondT_2.sh
else
echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_firstA_firstT_2.txt new/
aachange_firstA_firstT_2=$(cat new/aachange_firstA_firstT_2.txt)

```



```

$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_3.txt

cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_3.txt

transcript_id_firstT_1=$(cat transcript_id_firstT_1.txt)
transcript_id_secondT_1=$(cat transcript_id_secondT_1.txt)
transcript_id_firstT_2=$(cat transcript_id_firstT_2.txt)
transcript_id_secondT_2=$(cat transcript_id_secondT_2.txt)
transcript_id_firstT_3=$(cat transcript_id_firstT_3.txt)
transcript_id_secondT_3=$(cat transcript_id_secondT_3.txt)

aachange_firstT_1=$(cat aachange_firstT_1.txt)
aachange_secondT_1=$(cat aachange_secondT_1.txt)
aachange_firstT_2=$(cat aachange_firstT_2.txt)
aachange_secondT_2=$(cat aachange_secondT_2.txt)
aachange_firstT_3=$(cat aachange_firstT_3.txt)
aachange_secondT_3=$(cat aachange_secondT_3.txt)

#*****
#*****
#*****
if [ ! -z $aachange_firstT_1 ]
then
bash AA_expand_firstT_1.sh
else
echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_1 ]
then
bash AA_expand_secondT_1.sh
else

```

```

    echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_firstA_firstT_1.txt new/
aachange_firstA_firstT_1=$(cat new/aachange_firstA_firstT_1.txt)
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_lastA_firstT_1.txt new/
aachange_lastA_firstT_1=$(cat new/aachange_lastA_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_firstA_secondT_1.txt new/
aachange_firstA_secondT_1=$(cat new/aachange_firstA_secondT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_lastA_secondT_1.txt new/
aachange_lastA_secondT_1=$(cat new/aachange_lastA_secondT_1.txt)

fi

if [ ! -z $transcript_id_firstT_1 ]
then
mv transcript_id_firstT_1.txt new/
transcript_id_firstT_1=$(cat new/transcript_id_firstT_1.txt)
fi

if [ ! -z $transcript_id_secondT_1 ]
then
mv transcript_id_secondT_1.txt new/
transcript_id_secondT_1=$(cat new/transcript_id_secondT_1.txt)
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_firstT_1.txt new/
aachange_firstT_1=$(cat new/aachange_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_secondT_1.txt new/
aachange_secondT_1=$(cat new/aachange_secondT_1.txt)
fi

#####
#####
#####
if [ ! -z $aachange_firstT_2 ]
then
bash AA_expand_firstT_2.sh

```

```

        else
        echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_2 ]
then
bash AA_expand_secondT_2.sh
else
echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_firstA_firstT_2.txt new/
aachange_firstA_firstT_2=$(cat new/aachange_firstA_firstT_2.txt)
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_lastA_firstT_2.txt new/
aachange_lastA_firstT_2=$(cat new/aachange_lastA_firstT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
mv aachange_firstA_secondT_2.txt new/
aachange_firstA_secondT_2=$(cat new/aachange_firstA_secondT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
mv aachange_lastA_secondT_2.txt new/
aachange_lastA_secondT_2=$(cat new/aachange_lastA_secondT_2.txt)
fi

if [ ! -z $transcript_id_firstT_2 ]
then
mv transcript_id_firstT_2.txt new/
transcript_id_firstT_2=$(cat new/transcript_id_firstT_2.txt)
fi

if [ ! -z $transcript_id_secondT_2 ]
then
mv transcript_id_secondT_2.txt new/
transcript_id_secondT_2=$(cat new/transcript_id_secondT_2.txt)
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_firstT_2.txt new/
aachange_firstT_2=$(cat new/aachange_firstT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
mv aachange_secondT_2.txt new/
aachange_secondT_2=$(cat new/aachange_secondT_2.txt)

```

```

fi

#*****
#*****
#*****
if [ ! -z $aachange_firstT_3 ]
then
    bash AA_expand_firstT_3.sh
else
    echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_3 ]
then
    bash AA_expand_secondT_3.sh
else
    echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_3 ]
then
    mv aachange_firstA_firstT_3.txt new/
    aachange_firstA_firstT_3=$(cat aachange_firstA_firstT_3.txt)
fi

if [ ! -z $aachange_firstT_3 ]
then
    mv aachange_lastA_firstT_3.txt new/
    aachange_lastA_firstT_3=$(cat aachange_lastA_firstT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
then
    mv aachange_firstA_secondT_3.txt new/
    aachange_firstA_secondT_3=$(cat aachange_firstA_secondT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
then
    mv aachange_lastA_secondT_3.txt new/
    aachange_lastA_secondT_3=$(cat aachange_lastA_secondT_3.txt)
fi

if [ ! -z $transcript_id_firstT_3 ]
then
    mv transcript_id_firstT_3.txt new/
    transcript_id_firstT_3=$(cat transcript_id_firstT_3.txt)
fi

if [ ! -z $transcript_id_secondT_3 ]
then
    mv transcript_id_secondT_3.txt new/
    transcript_id_secondT_3=$(cat transcript_id_secondT_3.txt)
fi

if [ ! -z $aachange_firstT_3 ]
then

```

```

mv aachange_firstT_3.txt new/
aachange_firstT_3=$(cat aachange_firstT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
then
mv aachange_secondT_3.txt new/
aachange_secondT_3=$(cat aachange_secondT_3.txt)
fi

fi

#####

if [[ $clinsig == 4 ]]
then
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_4.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_4.txt

cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >

```



```

aachange_secondT_3.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 |cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_4.txt
    cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 |cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_4.txt

transcript_id_firstT_1=$(cat transcript_id_firstT_1.txt)
transcript_id_secondT_1=$(cat transcript_id_secondT_1.txt)
transcript_id_firstT_2=$(cat transcript_id_firstT_2.txt)
transcript_id_secondT_2=$(cat transcript_id_secondT_2.txt)
transcript_id_firstT_3=$(cat transcript_id_firstT_3.txt)
transcript_id_secondT_3=$(cat transcript_id_secondT_3.txt)
transcript_id_firstT_4=$(cat transcript_id_firstT_4.txt)
transcript_id_secondT_4=$(cat transcript_id_secondT_4.txt)

aachange_firstT_1=$(cat aachange_firstT_1.txt)
aachange_secondT_1=$(cat aachange_secondT_1.txt)
aachange_firstT_2=$(cat aachange_firstT_2.txt)
aachange_secondT_2=$(cat aachange_secondT_2.txt)
aachange_firstT_3=$(cat aachange_firstT_3.txt)
aachange_secondT_3=$(cat aachange_secondT_3.txt)
aachange_firstT_4=$(cat aachange_firstT_4.txt)
aachange_secondT_4=$(cat aachange_secondT_4.txt)

#*****
#*****
#*****
if [ ! -z $aachange_firstT_1 ]
then
    bash AA_expand_firstT_1.sh
else
    echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_1 ]
then
    bash AA_expand_secondT_1.sh
else
    echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_firstA_firstT_1.txt new/
aachange_firstA_firstT_1=$(cat new/aachange_firstA_firstT_1.txt)
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_lastA_firstT_1.txt new/
aachange_lastA_firstT_1=$(cat new/aachange_lastA_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_firstA_secondT_1.txt new/

```

```

aachange_firstA_secondT_1=$(cat new/aachange_firstA_secondT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_lastA_secondT_1.txt new/
aachange_lastA_secondT_1=$(cat new/aachange_lastA_secondT_1.txt)

fi

if [ ! -z $transcript_id_firstT_1 ]
then
mv transcript_id_firstT_1.txt new/
transcript_id_firstT_1=$(cat new/transcript_id_firstT_1.txt)
fi

if [ ! -z $transcript_id_secondT_1 ]
then
mv transcript_id_secondT_1.txt new/
transcript_id_secondT_1=$(cat new/transcript_id_secondT_1.txt)
fi


if [ ! -z $aachange_firstT_1 ]
then
mv aachange_firstT_1.txt new/
aachange_firstT_1=$(cat new/aachange_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_secondT_1.txt new/
aachange_secondT_1=$(cat new/aachange_secondT_1.txt)
fi

#*****
#*****
#*****
if [ ! -z $aachange_firstT_2 ]
then
bash AA_expand_firstT_2.sh
else
echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_2 ]
then
bash AA_expand_secondT_2.sh
else
echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_firstA_firstT_2.txt new/
aachange_firstA_firstT_2=$(cat new/aachange_firstA_firstT_2.txt)
fi

```

```

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_lastA_firstT_2.txt new/
aachange_lastA_firstT_2=$(cat new/aachange_lastA_firstT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
mv aachange_firstA_secondT_2.txt new/
aachange_firstA_secondT_2=$(cat new/aachange_firstA_secondT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
mv aachange_lastA_secondT_2.txt new/
aachange_lastA_secondT_2=$(cat new/aachange_lastA_secondT_2.txt)
fi

if [ ! -z $transcript_id_firstT_2 ]
then
mv transcript_id_firstT_2.txt new/
transcript_id_firstT_2=$(cat new/transcript_id_firstT_2.txt)
fi

if [ ! -z $transcript_id_secondT_2 ]
then
mv transcript_id_secondT_2.txt new/
transcript_id_secondT_2=$(cat new/transcript_id_secondT_2.txt)
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_firstT_2.txt new/
aachange_firstT_2=$(cat new/aachange_firstT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
mv aachange_secondT_2.txt new/
aachange_secondT_2=$(cat new/aachange_secondT_2.txt)
fi

#*****
#*****
#*****
if [ ! -z $aachange_firstT_3 ]
then
bash AA_expand_firstT_3.sh
else
echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_3 ]
then
bash AA_expand_secondT_3.sh
else
echo "There is no Amino Acid change in the Second Transcript for this
Variant."

```

```

fi

if [ ! -z $aachange_firstT_3 ]
then
mv aachange_firstA_firstT_3.txt new/
aachange_firstA_firstT_3=$(cat aachange_firstA_firstT_3.txt)
fi

if [ ! -z $aachange_firstT_3 ]
then
mv aachange_lastA_firstT_3.txt new/
aachange_lastA_firstT_3=$(cat aachange_lastA_firstT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
then
mv aachange_firstA_secondT_3.txt new/
aachange_firstA_secondT_3=$(cat aachange_firstA_secondT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
then
mv aachange_lastA_secondT_3.txt new/
aachange_lastA_secondT_3=$(cat aachange_lastA_secondT_3.txt)
fi

if [ ! -z $transcript_id_firstT_3 ]
then
mv transcript_id_firstT_3.txt new/
transcript_id_firstT_3=$(cat transcript_id_firstT_3.txt)
fi

if [ ! -z $transcript_id_secondT_3 ]
then
mv transcript_id_secondT_3.txt new/
transcript_id_secondT_3=$(cat transcript_id_secondT_3.txt)
fi

if [ ! -z $aachange_firstT_3 ]
then
mv aachange_firstT_3.txt new/
aachange_firstT_3=$(cat aachange_firstT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
then
mv aachange_secondT_3.txt new/
aachange_secondT_3=$(cat aachange_secondT_3.txt)
fi

#*****
#*****
#*****
if [ ! -z $aachange_firstT_4 ]
then
bash AA_expand_firstT_4.sh
else
echo "There is no Amino Acid change in the first Transcript for this
Variant."

```



```

cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_4.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_4.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_5 | cut -d' ' -f3 | cut -d':' -f 2 >
transcript_id_firstT_5.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_5 | cut -d' ' -f3 | cut -d':' -f 6 >
transcript_id_secondT_5.txt

cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_1 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_1.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_2 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_2.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_3 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_3.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_4.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_4 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >
aachange_secondT_4.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_5 | cut -d' ' -f3 | cut -d':' -f5 | cut -d',' -f1 >
aachange_firstT_5.txt
cat annovar.human.refGene.exonic_variant_function | grep
$clinsig_position_5 | cut -d' ' -f3 | cut -d':' -f9 | cut -d',' -f1 >

```

```

aachange_secondT_5.txt

transcript_id_firstT_1=$(cat transcript_id_firstT_1.txt)
transcript_id_secondT_1=$(cat transcript_id_secondT_1.txt)
transcript_id_firstT_2=$(cat transcript_id_firstT_2.txt)
transcript_id_secondT_2=$(cat transcript_id_secondT_2.txt)
transcript_id_firstT_3=$(cat transcript_id_firstT_3.txt)
transcript_id_secondT_3=$(cat transcript_id_secondT_3.txt)
transcript_id_firstT_4=$(cat transcript_id_firstT_4.txt)
transcript_id_secondT_4=$(cat transcript_id_secondT_4.txt)
transcript_id_firstT_5=$(cat transcript_id_firstT_5.txt)
transcript_id_secondT_5=$(cat transcript_id_secondT_5.txt)

aachange_firstT_1=$(cat aachange_firstT_1.txt)
aachange_secondT_1=$(cat aachange_secondT_1.txt)
aachange_firstT_2=$(cat aachange_firstT_2.txt)
aachange_secondT_2=$(cat aachange_secondT_2.txt)
aachange_firstT_3=$(cat aachange_firstT_3.txt)
aachange_secondT_3=$(cat aachange_secondT_3.txt)
aachange_firstT_4=$(cat aachange_firstT_4.txt)
aachange_secondT_4=$(cat aachange_secondT_4.txt)
aachange_firstT_5=$(cat aachange_firstT_5.txt)
aachange_secondT_5=$(cat aachange_secondT_5.txt)
#*****
*****
*****
if [ ! -z $aachange_firstT_1 ]
then
    bash AA_expand_firstT_1.sh
else
    echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_1 ]
then
    bash AA_expand_secondT_1.sh
else
    echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_1 ]
then
    mv aachange_firstA_firstT_1.txt new/
    aachange_firstA_firstT_1=$(cat new/aachange_firstA_firstT_1.txt)
fi

if [ ! -z $aachange_firstT_1 ]
then
    mv aachange_lastA_firstT_1.txt new/
    aachange_lastA_firstT_1=$(cat new/aachange_lastA_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
    mv aachange_firstA_secondT_1.txt new/
    aachange_firstA_secondT_1=$(cat new/aachange_firstA_secondT_1.txt)
fi

```

```

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_lastA_secondT_1.txt new/
aachange_lastA_secondT_1=$(cat new/aachange_lastA_secondT_1.txt)

fi

if [ ! -z $transcript_id_firstT_1 ]
then
mv transcript_id_firstT_1.txt new/
transcript_id_firstT_1=$(cat new/transcript_id_firstT_1.txt)
fi

if [ ! -z $transcript_id_secondT_1 ]
then
mv transcript_id_secondT_1.txt new/
transcript_id_secondT_1=$(cat new/transcript_id_secondT_1.txt)
fi

if [ ! -z $aachange_firstT_1 ]
then
mv aachange_firstT_1.txt new/
aachange_firstT_1=$(cat new/aachange_firstT_1.txt)
fi

if [ ! -z $aachange_secondT_1 ]
then
mv aachange_secondT_1.txt new/
aachange_secondT_1=$(cat new/aachange_secondT_1.txt)
fi

#*****
#*****
#*****
if [ ! -z $aachange_firstT_2 ]
then
bash AA_expand_firstT_2.sh
else
echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_2 ]
then
bash AA_expand_secondT_2.sh
else
echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_firstA_firstT_2.txt new/
aachange_firstA_firstT_2=$(cat new/aachange_firstA_firstT_2.txt)
fi

if [ ! -z $aachange_firstT_2 ]
then
mv aachange_lastA_firstT_2.txt new/

```



```

    aachange_lastA_firstT_2=$(cat new/aachange_lastA_firstT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
    mv aachange_firstA_secondT_2.txt new/
    aachange_firstA_secondT_2=$(cat new/aachange_firstA_secondT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
    mv aachange_lastA_secondT_2.txt new/
    aachange_lastA_secondT_2=$(cat new/aachange_lastA_secondT_2.txt)
fi

if [ ! -z $transcript_id_firstT_2 ]
then
    mv transcript_id_firstT_2.txt new/
    transcript_id_firstT_2=$(cat new/transcript_id_firstT_2.txt)
fi

if [ ! -z $transcript_id_secondT_2 ]
then
    mv transcript_id_secondT_2.txt new/
    transcript_id_secondT_2=$(cat new/transcript_id_secondT_2.txt)
fi

if [ ! -z $aachange_firstT_2 ]
then
    mv aachange_firstT_2.txt new/
    aachange_firstT_2=$(cat new/aachange_firstT_2.txt)
fi

if [ ! -z $aachange_secondT_2 ]
then
    mv aachange_secondT_2.txt new/
    aachange_secondT_2=$(cat new/aachange_secondT_2.txt)
fi

#*****
#*****
#*****
if [ ! -z $aachange_firstT_3 ]
then
    bash AA_expand_firstT_3.sh
else
    echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_3 ]
then
    bash AA_expand_secondT_3.sh
else
    echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_3 ]

```

```

        then
        mv aachange_firstA_firstT_3.txt new/
aachange_firstA_firstT_3=$(cat aachange_firstA_firstT_3.txt)
fi

if [ ! -z $aachange_firstT_3 ]
    then
        mv aachange_lastA_firstT_3.txt new/
aachange_lastA_firstT_3=$(cat aachange_lastA_firstT_3.txt)
fi

        if [ ! -z $aachange_secondT_3 ]
            then
                mv aachange_firstA_secondT_3.txt new/
aachange_firstA_secondT_3=$(cat aachange_firstA_secondT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
    then
        mv aachange_lastA_secondT_3.txt new/
aachange_lastA_secondT_3=$(cat aachange_lastA_secondT_3.txt)
fi

if [ ! -z $transcript_id_firstT_3 ]
    then
        mv transcript_id_firstT_3.txt new/
transcript_id_firstT_3=$(cat transcript_id_firstT_3.txt)
fi

if [ ! -z $transcript_id_secondT_3 ]
    then
        mv transcript_id_secondT_3.txt new/
transcript_id_secondT_3=$(cat transcript_id_secondT_3.txt)
fi

if [ ! -z $aachange_firstT_3 ]
    then
        mv aachange_firstT_3.txt new/
aachange_firstT_3=$(cat aachange_firstT_3.txt)
fi

if [ ! -z $aachange_secondT_3 ]
    then
        mv aachange_secondT_3.txt new/
aachange_secondT_3=$(cat aachange_secondT_3.txt)
fi

#*****
#*****
#*****
if [ ! -z $aachange_firstT_4 ]
    then
        bash AA_expand_firstT_4.sh
    else
        echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_4 ]

```

```

        then
        bash AA_expand_secondT_4.sh
        else
        echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_4 ]
then
mv aachange_firstA_firstT_4.txt new/
aachange_firstA_firstT_4=$(cat aachange_firstA_firstT_4.txt)

fi

if [ ! -z $aachange_firstT_4 ]
then
mv aachange_lastA_firstT_4.txt new/
aachange_lastA_firstT_4=$(cat aachange_lastA_firstT_4.txt)
fi

if [ ! -z $aachange_secondT_4 ]
then
mv aachange_firstA_secondT_4.txt new/
aachange_firstA_secondT_4=$(cat aachange_firstA_secondT_4.txt)
fi

if [ ! -z $aachange_secondT_4 ]
then
mv aachange_lastA_secondT_4.txt new/
aachange_lastA_secondT_4=$(cat aachange_lastA_secondT_4.txt)
fi

if [ ! -z $transcript_id_firstT_4 ]
then
mv transcript_id_firstT_4.txt new/
transcript_id_firstT_4=$(cat transcript_id_firstT_4.txt)
fi

if [ ! -z $transcript_id_secondT_4 ]
then
mv transcript_id_secondT_4.txt new/
transcript_id_secondT_4=$(cat transcript_id_secondT_4.txt)
fi

if [ ! -z $aachange_firstT_4 ]
then
mv aachange_firstT_4.txt new/
aachange_firstT_4=$(cat aachange_firstT_4.txt)
fi

if [ ! -z $aachange_secondT_4 ]
then
mv aachange_secondT_4.txt new/
aachange_secondT_4=$(cat aachange_secondT_4.txt)
fi

#*****
*****
*****

```

```

if [ ! -z $aachange_firstT_5 ]
then
bash AA_expand_firstT_5.sh
else
echo "There is no Amino Acid change in the first Transcript for this
Variant."
fi

if [ ! -z $aachange_secondT_5 ]
then
bash AA_expand_secondT_5.sh
else
echo "There is no Amino Acid change in the Second Transcript for this
Variant."
fi

if [ ! -z $aachange_firstT_5 ]
then
mv aachange_firstA_firstT_5.txt new/
aachange_firstA_firstT_5=$(cat aachange_firstA_firstT_5.txt)
fi

if [ ! -z $aachange_firstT_5 ]
then
mv aachange_lastA_firstT_5.txt new/
aachange_lastA_firstT_5=$(cat aachange_lastA_firstT_5.txt)
fi

if [ ! -z $aachange_secondT_5 ]
then
mv aachange_firstA_secondT_5.txt new/
aachange_firstA_secondT_5=$(cat aachange_firstA_secondT_5.txt)
fi

if [ ! -z $aachange_secondT_5 ]
then
mv aachange_lastA_secondT_5.txt new/
aachange_lastA_secondT_5=$(cat aachange_lastA_secondT_5.txt)
fi

if [ ! -z $transcript_id_firstT_5 ]
then
mv transcript_id_firstT_5.txt new/
transcript_id_firstT_5=$(cat transcript_id_firstT_5.txt)
fi

if [ ! -z $transcript_id_secondT_5 ]
then
mv transcript_id_secondT_5.txt new/
transcript_id_secondT_5=$(cat transcript_id_secondT_5.txt)
fi

if [ ! -z $aachange_firstT_5 ]
then
mv aachange_firstT_5.txt new/
aachange_firstT_5=$(cat aachange_firstT_5.txt)
fi

if [ ! -z $aachange_secondT_5 ]
then

```

```

mv aachange_secondT_5.txt new/
aachange_secondT_5=$(cat aachange_secondT_5.txt)
fi

fi

#-----
-----

# To get the observed and clinvar 1 line summary.

if [ $clinsig == 0 ]
then
echo "No clinically significant variants were observed for the
diagnosed condition." > new/clinvar_observed.txt

elif [ $clinsig == 1 ]
then
echo "1 Clinically significant mutation has been identified for the
diagnosed condition." > new/clinvar_observed.txt
echo "
<table border="1" >
<tr>
<th bgcolor=LightGray>&nbsp;&nbsp;&nbsp;</th>
<th bgcolor="LightGray" width=375 align="center">Gene Name</th>
<th bgcolor="LightGray" width=375 align="center">Disease</th>
<th bgcolor="LightGray" width=375 align="center">Location of the
Variant</th>
<th bgcolor="LightGray" width=375
align="center">Classification</th>
</tr>
<tr>
<td align=center>1.</td>
<td width=375 align="center">$gene_name_1</td>
<td width=375 align="center">$clinvar_diseases_1_1</td>
<td width=375 align="center">$GENOMIC_LOCATION_of_variant_1_1 <br/>
$comp_variant_firstT_1 </td>
<td width=375 align="center">$clinvar_classification_1_1</td>
</tr>
</table><br>&nbsp;&nbsp;&nbsp;<br>" > new/observed.html

elif [ $clinsig == 2 ]
then
echo "2 Clinically significant mutation has been identified for the
diagnosed condition." > new/clinvar_observed.txt
echo "
<table border="1" >
<tr>
<th bgcolor=LightGray>&nbsp;&nbsp;&nbsp;</th>
<th bgcolor="LightGray" width=375 align="center">Gene Name</th>
<th bgcolor="LightGray" width=375 align="center">Disease</th>
<th bgcolor="LightGray" width=375 align="center">Location of the
Variant</th>
<th bgcolor="LightGray" width=375
align="center">Classification</th>
</tr>
<tr>
<td align=center>1.</td>
<td width=375 align="center">$gene_name_1</td>
<td width=375 align="center">$clinvar_diseases_1_2</td>

```

```

        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_1_2 <br/>
$comp_variant_firstT_1 </td>
        <td width=375 align="center">$clinvar_classification_1_2</td>
    </tr>
    <tr>
        <td align=center>2.</td>
        <td width=375 align="center">$gene_name_2</td>
        <td width=375 align="center">$clinvar_diseases_2_2</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_2_2 <br/>
$comp_variant_firstT_2 </td>
        <td width=375 align="center">$clinvar_classification_2_2</td>
    </tr></table><br>&nbsp;<br>" > new/observed.html

elif [ $clinsig == 3 ]
then
    echo "3 Clinically significant mutation has been identified for the
diagnosed condition." > new/clinvar_observed.txt
    echo "
    <table border="1" >
    <tr>
        <th bgcolor=LightGray>&nbsp;</th>
        <th width=375 align="center">Gene Name</th>
        <th width=375 align="center">Disease</th>
        <th width=375 align="center">Location of the Variant</th>
        <th width=375 align="center">Classification</th>
    </tr>
    <tr>
        <td align=center>1.</td>
        <td width=375 align="center">$gene_name_1</td>
        <td width=375 align="center">$clinvar_diseases_1_3</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_1_3 <br/>
$comp_variant_firstT_1 </td>
        <td width=375 align="center">$clinvar_classification_1_3</td>
    </tr>
    <tr>
        <td align=center>2.</td>
        <td width=375 align="center">$gene_name_2</td>
        <td width=375 align="center">$clinvar_diseases_2_3</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_2_3 <br/>
$comp_variant_firstT_2 </td>
        <td width=375 align="center">$clinvar_classification_2_3</td>
    </tr>
    <tr>
        <td align=center>3.</td>
        <td width=375 align="center">$gene_name_3</td>
        <td width=375 align="center">$clinvar_diseases_3_3</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_3_3 <br/>
$comp_variant_firstT_3 </td>
        <td width=375 align="center">$clinvar_classification_3_3</td>
    </tr></table><br>&nbsp;<br>" > new/observed.html

elif [ $clinsig == 4 ]
then
    echo "4 Clinically significant mutation has been identified for the
diagnosed condition." > new/clinvar_observed.txt
    echo "
    <table border="1" >
    <tr>
        <th bgcolor=LightGray>&nbsp;</th>
        <th width=375 align="center">Gene Name</th>

```

```

        <th width=375 align="center">Disease</th>
        <th width=375 align="center">Location of the Variant</th>
        <th width=375 align="center">Classification</th>
    </tr>
    <tr>
    <td align=center>1.</td>
    <td width=375 align="center">$gene_name_1</td>
    <td width=375 align="center">$clinvar_diseases_1_4</td>
    <td width=375 align="center">$GENOMIC_LOCATION_of_variant_1_4 <br/>
$comp_variant_firstT_1 </td>
    <td width=375 align="center">$clinvar_classification_1_4</td>
    </tr>
    <tr>
    <td align=center>2.</td>
    <td width=375 align="center">$gene_name_2</td>
    <td width=375 align="center">$clinvar_diseases_2_4</td>
    <td width=375 align="center">$GENOMIC_LOCATION_of_variant_2_4 <br/>
$comp_variant_firstT_2 </td>
    <td width=375 align="center">$clinvar_classification_2_4</td>
    </tr>
    <tr>
    <td align=center>3.</td>
    <td width=375 align="center">$gene_name_3</td>
    <td width=375 align="center">$clinvar_diseases_3_4</td>
    <td width=375 align="center">$GENOMIC_LOCATION_of_variant_3_4 <br/>
$comp_variant_firstT_3 </td>
    <td width=375 align="center">$clinvar_classification_3_4</td>
    </tr>
    <tr>
    <td align=center>4.</td>
    <td width=375 align="center">$gene_name_4</td>
    <td width=375 align="center">$clinvar_diseases_4_4</td>
    <td width=375 align="center">$GENOMIC_LOCATION_of_variant_4_4 <br/>
$comp_variant_firstT_4 </td>
    <td width=375 align="center">$clinvar_classification_4_4</td>
    </tr></table><br>&nbsp;<br>" > new/observed.html

elif [ $clinsig == 5 ]
then
echo "5 Clinically significant mutation has been identified for the
diagnosed condition." > new/clinvar_observed.txt
echo "
<table border="1" >
<tr>
    <th bgcolor=LightGray>&nbsp;</th>
    <th width=375 align="center">Gene Name</th>
    <th width=375 align="center">Disease</th>
    <th width=375 align="center">Location of the Variant</th>
    <th width=375 align="center">Classification</th>
</tr>
<tr>
<td align=center>1.</td>
<td width=375 align="center">$gene_name_1</td>
<td width=375 align="center">$clinvar_diseases_1_5</td>
<td width=375 align="center">$GENOMIC_LOCATION_of_variant_1_5 <br/>
$comp_variant_firstT_1 </td>
<td width=375 align="center">$clinvar_classification_1_5</td>
</tr>
<tr>
<td align=center>2.</td>

```

```

        <td width=375 align="center">$gene_name_2</td>
        <td width=375 align="center">$clinvar_diseases_2_5</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_2_5 <br/>
$comp_variant_firstT_2 </td>
        <td width=375 align="center">$clinvar_classification_2_5</td>
    </tr>
    <tr>
        <td align=center>3.</td>
        <td width=375 align="center">$gene_name_3</td>
        <td width=375 align="center">$clinvar_diseases_3_5</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_3_5 <br/>
$comp_variant_firstT_3 </td>
        <td width=375 align="center">$clinvar_classification_3_5</td>
    </tr>
    <tr>
        <td align=center>4.</td>
        <td width=375 align="center">$gene_name_4</td>
        <td width=375 align="center">$clinvar_diseases_4_5</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_4_5 <br/>
$comp_variant_firstT_4 </td>
        <td width=375 align="center">$clinvar_classification_4_5</td>
    </tr>
    <tr>
        <td align=center>5.</td>
        <td width=375 align="center">$gene_name_5</td>
        <td width=375 align="center">$clinvar_diseases_5_5</td>
        <td width=375 align="center">$GENOMIC_LOCATION_of_variant_5_5 <br/>
$comp_variant_firstT_5 </td>
        <td width=375 align="center">$clinvar_classification_5_5</td>
    </tr></table><br>&nbsp;<br>" > new/observed.html

fi

#-----
#-----
# To get the DETAILED VARIANT INFORMATION:..

if [ $clinsig == 0 ]
then
    echo "No Clinically significant mutation has been identified for the
diagnosed condition." > new/clinvar_detailed.txt

elif [ $clinsig == 1 ]
then
    echo "Annotations of the Clinically significant mutation." >
new/clinvar_detailed.txt
    echo "
    <table border="1" >

        <tr>
            <th bgcolor=LightGray>&nbsp;</th>
            <th width=375 bgcolor="LightGray" align="center">Gene Name</th>
            <th width=375 bgcolor="LightGray" align="center">Variant
Type</th>
            <th width=375 bgcolor="LightGray" align="center">Variant
Location</th>
            <th width=375 bgcolor="LightGray" align="center">Amino Acid
Change</th>
            <th width=375 bgcolor="LightGray" align="center">dbSNP

```



```

Identifier</th>

</tr>
<tr>
<td align=center>1.</td>
<td width=375 align="center">$gene_name_1 (RefSeq ID:
$transcript_id_firstT_1)</td>
<td width=375 align="center">$variant_type_1</td>
<td width=375 align="center">$variant_location_1</td>
<td width=375 align="center">$aachange_firstT_1</td>
<td width=375 align="center">$rsID_1</td>

</tr></table><br>&nbsp;<br>
<table border=1 >
<caption><strong>MINOR ALLELE FREQUENCY and PREDICTED IMPACT OF
VARIANT:</strong></caption>
<tr>
<th bgcolor=LightGray>&nbsp;</th>
<th width=375 bgcolor="LightGray" align="center">MAF from 1000
Genomes Project</th>
<th width=375 bgcolor="LightGray" align="center">SIFT score and
prediction</th>
<th width=375 bgcolor="LightGray" align="center">Polyphen score
and prediction</th>
</tr>

<tr>
<td align=center>1.</td>
<td width=375 align="center">$maf_1</td>
<td width=375 align="center">$sift_score_1 and $sift_pred_1</td>
<td width=375 align="center">$polyphen_score_1 and
$polyphen_pred_1</td>
</tr> </table> <br>&nbsp;<br>" > new/detailed.html

elif [ $clinsig == 2 ]
then
echo "Annotations of the Clinically significant mutations." >
new/clinvar_detailed.txt
echo "
<table border="1" >

<tr>
<th bgcolor=LightGray>&nbsp;</th>
<th width=375 bgcolor="LightGray" align="center">Gene Name</th>
<th width=375 bgcolor="LightGray" align="center">Variant
Type</th>
<th width=375 bgcolor="LightGray" align="center">Variant
Location</th>
<th width=375 bgcolor="LightGray" align="center">Amino Acid
Change</th>
<th width=375 bgcolor="LightGray" align="center">dbSNP
Identifier</th>
</tr>
<tr>
<td align=center>1.</td>
<td width=375 align="center">$gene_name_1 (RefSeq ID:
$transcript_id_firstT_1)</td>
<td width=375 align="center">$variant_type_1</td>
<td width=375 align="center">$variant_location_1</td>

```

```

        <td width=375 align="center">$aachange_firstT_1</td>
        <td width=375 align="center">$rsID_1</td>

</tr>
<tr>
    <td align=center>2.</td>
    <td width=375 align="center">$gene_name_2 (RefSeq ID:
$transcript_id_firstT_2)</td>
    <td width=375 align="center">$variant_type_2</td>
    <td width=375 align="center">$variant_location_2</td>
    <td width=375 align="center">$aachange_firstT_2</td>
    <td width=375 align="center">$rsID_2</td>
</tr>
</table><br>&nbsp;<br>
    <table border="1" >
        <caption><strong>MINOR ALLELE FREQUENCY and PREDICTED IMPACT OF
VARIANT:</strong></caption>
<tr>
    <th bgcolor=LightGray>&nbsp;</th>
    <th width=375 bgcolor="LightGray" align="center">MAF from 1000
Genomes Project</th>
    <th width=375 bgcolor="LightGray" align="center">SIFT score and
prediction</th>
    <th width=375 bgcolor="LightGray" align="center">Polyphen score
and prediction</th>
</tr>

<tr>
    <td align=center>1.</td>
    <td width=375 align="center">$maf_1</td>
    <td width=375 align="center">$sift_score_1 and $sift_pred_1</td>
    <td width=375 align="center">$polyphen_score_1 and
$polyphen_pred_1</td>
</tr>

<tr>
    <td align=center>2.</td>
    <td width=375 align="center">$maf_2</td>
    <td width=375 align="center">$sift_score_2 and $sift_pred_2</td>
    <td width=375 align="center">$polyphen_score_2 and
$polyphen_pred_2</td>
</tr>
</table> <br>&nbsp;<br>" > new/detailed.html

elif [ $clinsig == 3 ]
then
    echo "Annotations of the Clinically significant mutations." >
new/clinvar_detailed.txt
    echo "
    <table border="1" >

<tr>
    <th bgcolor=LightGray>&nbsp;</th>
    <th width=375 bgcolor="LightGray" align="center">Gene Name</th>
    <th width=375 bgcolor="LightGray" align="center">Variant
Type</th>
    <th width=375 bgcolor="LightGray" align="center">Variant
Location</th>
    <th width=375 bgcolor="LightGray" align="center">Amino Acid

```

```

Change</th>
    <th width=375 bgcolor="LightGray" align="center">dbSNP
Identifier</th>

</tr>
<tr>
    <td align=center>1.</td>
    <td width=375 align="center">$gene_name_1 (RefSeq ID:
$transcript_id_firstT_1)</td>
    <td width=375 align="center">$variant_type_1</td>
    <td width=375 align="center">$variant_location_1</td>
    <td width=375 align="center">$aachange_firstT_1</td>
    <td width=375 align="center">$rsID_1</td>

</tr>
<tr>
    <td align=center>2.</td>
    <td width=375 align="center">$gene_name_2 (RefSeq ID:
$transcript_id_firstT_2)</td>
    <td width=375 align="center">$variant_type_2</td>
    <td width=375 align="center">$variant_location_2</td>
    <td width=375 align="center">$aachange_firstT_2</td>
    <td width=375 align="center">$rsID_2</td>

</tr>
<tr>
    <td align=center>3.</td>
    <td width=375 align="center">$gene_name_3 (RefSeq ID:
$transcript_id_firstT_3)</td>
    <td width=375 align="center">$variant_type_3</td>
    <td width=375 align="center">$variant_location_3</td>
    <td width=375 align="center">$aachange_firstT_3</td>
    <td width=375 align="center">$rsID_3</td>

</tr>
</table><br>&nbsp;<br>

<table border="1" >
    <caption><strong>MINOR ALLELE FREQUENCY and PREDICTED IMPACT OF
VARIANT:</strong></caption>
    <tr>
        <th bgcolor=LightGray>&nbsp;</th>
        <th width=375 bgcolor="LightGray" align="center">MAF from 1000
Genomes Project</th>
        <th width=375 bgcolor="LightGray" align="center">SIFT score and
prediction</th>
        <th width=375 bgcolor="LightGray" align="center">Polyphen score
and prediction</th>
    </tr>

    <tr>
        <td align=center>1.</td>
        <td width=375 align="center">$maf_1</td>
        <td width=375 align="center">$sift_score_1 and $sift_pred_1</td>
        <td width=375 align="center">$polyphen_score_1 and
$polyphen_pred_1</td>
    </tr>

    <tr>
        <td align=center>2.</td>
        <td width=375 align="center">$maf_2</td>
        <td width=375 align="center">$sift_score_2 and $sift_pred_2</td>

```

```

        <td width=375 align="center">$polyphen_score_2 and
        $polyphen_pred_2</td>
    </tr>
    <tr>
        <td align=center>3.</td>
        <td width=375 align="center">$maf_3</td>
        <td width=375 align="center">$sift_score_3 and $sift_pred_3</td>
        <td width=375 align="center">$polyphen_score_3 and
        $polyphen_pred_3</td>
    </tr>
</table> <br>&nbsp;<br>" > new/detailed.html

elif [ $clinsig == 4 ]
then
    echo "Annotations of the Clinically significant mutations." >
    new/clinvar_detailed.txt
    echo "
    <table border="1" >

    <tr>
        <th bgcolor=LightGray>&nbsp;</th>
        <th width=375 bgcolor="LightGray" align="center">Gene Name</th>
        <th width=375 bgcolor="LightGray" align="center">Variant
Type</th>
        <th width=375 bgcolor="LightGray" align="center">Variant
Location</th>
        <th width=375 bgcolor="LightGray" align="center">Amino Acid
Change</th>
        <th width=375 bgcolor="LightGray" align="center">dbSNP
Identifier</th>

    </tr>
    <tr>
        <td align=center>1.</td>
        <td width=375 align="center">$gene_name_1 (RefSeq ID:
$transcript_id_firstT_1)</td>
        <td width=375 align="center">$variant_type_1</td>
        <td width=375 align="center">$variant_location_1</td>
        <td width=375 align="center">$aachange_firstT_1</td>
        <td width=375 align="center">$rsID_1</td>

    </tr>
    <tr>
        <td align=center>2.</td>
        <td width=375 align="center">$gene_name_2 (RefSeq ID:
$transcript_id_firstT_2)</td>
        <td width=375 align="center">$variant_type_2</td>
        <td width=375 align="center">$variant_location_2</td>
        <td width=375 align="center">$aachange_firstT_2</td>
        <td width=375 align="center">$rsID_2</td>

    </tr>
    <tr>
        <td align=center>3.</td>
        <td width=375 align="center">$gene_name_3 (RefSeq ID:
$transcript_id_firstT_3)</td>
        <td width=375 align="center">$variant_type_3</td>
        <td width=375 align="center">$variant_location_3</td>
        <td width=375 align="center">$aachange_firstT_3</td>
        <td width=375 align="center">$rsID_3</td>

    </tr>

```

```

<tr>
  <td align=center>4.</td>
  <td width=375 align="center">$gene_name_4 (RefSeq ID:
$transcript_id_firstT_4)</td>
  <td width=375 align="center">$variant_type_4</td>
  <td width=375 align="center">$variant_location_4</td>
  <td width=375 align="center">$aachange_firstT_4</td>
  <td width=375 align="center">$rsID_4</td>
</tr>
</table><br>&nbsp;<br>

<table border="1" >
  <caption><strong>MINOR ALLELE FREQUENCY and PREDICTED IMPACT OF
VARIANT:</strong></caption>
  <tr>
    <th bgcolor=LightGray>&nbsp;</th>
    <th width=375 bgcolor="LightGray" align="center">MAF from 1000
Genomes Project</th>
    <th width=375 bgcolor="LightGray" align="center">SIFT score and
prediction</th>
    <th width=375 bgcolor="LightGray" align="center">Polyphen score
and prediction</th>
  </tr>

  <tr>
    <td align=center>1.</td>
    <td width=375 align="center">$maf_1</td>
    <td width=375 align="center">$sift_score_1 and $sift_pred_1</td>
    <td width=375 align="center">$polyphen_score_1 and
$polyphen_pred_1</td>
  </tr>

  <tr>
    <td align=center>2.</td>
    <td width=375 align="center">$maf_2</td>
    <td width=375 align="center">$sift_score_2 and $sift_pred_2</td>
    <td width=375 align="center">$polyphen_score_2 and
$polyphen_pred_2</td>
  </tr>

  <tr>
    <td align=center>3.</td>
    <td width=375 align="center">$maf_3</td>
    <td width=375 align="center">$sift_score_3 and $sift_pred_3</td>
    <td width=375 align="center">$polyphen_score_3 and
$polyphen_pred_3</td>
  </tr>

  <tr>
    <td align=center>4.</td>
    <td width=375 align="center">$maf_4</td>
    <td width=375 align="center">$sift_score_4 and $sift_pred_4</td>
    <td width=375 align="center">$polyphen_score_4 and
$polyphen_pred_4</td>
  </tr>
</table> <br>&nbsp;<br>" > new/detailed.html

elif [ $clinsig == 5 ]
then
  echo "Annotations of the Clinically significant mutations." >
new/clinvar_detailed.txt
  echo "

```

```

<table border="1" >

<tr>
    <th bgcolor=LightGray>&nbsp;   </th>
    <th width=375 bgcolor="LightGray" align="center">Gene Name</th>
    <th width=375 bgcolor="LightGray" align="center">Variant
Type</th>
    <th width=375 bgcolor="LightGray" align="center">Variant
Location</th>
    <th width=375 bgcolor="LightGray" align="center">Amino Acid
Change</th>
    <th width=375 bgcolor="LightGray" align="center">dbSNP
Identifier</th>

</tr>
<tr>
    <td align=center>1.</td>
    <td width=375 align="center">$gene_name_1 (RefSeq ID:
$transcript_id_firstT_1)</td>
    <td width=375 align="center">$variant_type_1</td>
    <td width=375 align="center">$variant_location_1</td>
    <td width=375 align="center">$aachange_firstT_1</td>
    <td width=375 align="center">$rsID_1</td>

</tr>
<tr>
    <td align=center>2.</td>
    <td width=375 align="center">$gene_name_2 (RefSeq ID:
$transcript_id_firstT_2)</td>
    <td width=375 align="center">$variant_type_2</td>
    <td width=375 align="center">$variant_location_2</td>
    <td width=375 align="center">$aachange_firstT_2</td>
    <td width=375 align="center">$rsID_2</td>

</tr>
<tr>
    <td align=center>3.</td>
    <td width=375 align="center">$gene_name_3 (RefSeq ID:
$transcript_id_firstT_3)</td>
    <td width=375 align="center">$variant_type_3</td>
    <td width=375 align="center">$variant_location_3</td>
    <td width=375 align="center">$aachange_firstT_3</td>
    <td width=375 align="center">$rsID_3</td>

</tr>
<tr>
    <td align=center>4.</td>
    <td width=375 align="center">$gene_name_4 (RefSeq ID:
$transcript_id_firstT_4)</td>
    <td width=375 align="center">$variant_type_4</td>
    <td width=375 align="center">$variant_location_4</td>
    <td width=375 align="center">$aachange_firstT_4</td>
    <td width=375 align="center">$rsID_4</td>

</tr>
<tr>
    <td align=center>5.</td>
    <td width=375 align="center">$gene_name_5 (RefSeq ID:
$transcript_id_firstT_5)</td>
    <td width=375 align="center">$variant_type_5</td>
    <td width=375 align="center">$variant_location_5</td>
    <td width=375 align="center">$aachange_firstT_5</td>
    <td width=375 align="center">$rsID_5</td>

```

```

</tr>
</table><br>&nbsp;<br>

<table border="1" >
  <caption><strong>MINOR ALLELE FREQUENCY and PREDICTED IMPACT OF
  VARIANT:</strong></caption>
  <tr>
    <th bgcolor=LightGray>&nbsp;</th>
    <th width=375 bgcolor="LightGray" align="center">MAF from 1000
  Genomes Project</th>
    <th width=375 bgcolor="LightGray" align="center">SIFT score and
  prediction</th>
    <th width=375 bgcolor="LightGray" align="center">Polyphen score
  and prediction</th>
  </tr>

  <tr>
    <td align=center>1.</td>
    <td width=375 align="center">$maf_1</td>
    <td width=375 align="center">$sift_score_1 and $sift_pred_1</td>
    <td width=375 align="center">$polyphen_score_1 and
  $polyphen_pred_1</td>
  </tr>

  <tr>
    <td align=center>2.</td>
    <td width=375 align="center">$maf_2</td>
    <td width=375 align="center">$sift_score_2 and $sift_pred_2</td>
    <td width=375 align="center">$polyphen_score_2 and
  $polyphen_pred_2</td>
  </tr>

  <tr>
    <td align=center>3.</td>
    <td width=375 align="center">$maf_3</td>
    <td width=375 align="center">$sift_score_3 and $sift_pred_3</td>
    <td width=375 align="center">$polyphen_score_3 and
  $polyphen_pred_3</td>
  </tr>

  <tr>
    <td align=center>4.</td>
    <td width=375 align="center">$maf_4</td>
    <td width=375 align="center">$sift_score_4 and $sift_pred_4</td>
    <td width=375 align="center">$polyphen_score_4 and
  $polyphen_pred_4</td>
  </tr>

  <tr>
    <td align=center>5.</td>
    <td width=375 align="center">$maf_5</td>
    <td width=375 align="center">$sift_score_5 and $sift_pred_5</td>
    <td width=375 align="center">$polyphen_score_5 and
  $polyphen_pred_5</td>
  </tr>
</table> <br>&nbsp;<br>" > new/detailed.html

fi

#-----
#-----
#-----
# To get the VARIANT INTERPRETATION:

```

```

if [ $clinsig == 0 ]
then
    echo "No Clinically significant mutation has been identified for the
diagnosed condition." > new/variant_interpretation.txt

elif [ $clinsig == 1 ]
then
    echo "<br><br><table border='1' >
<caption><strong>VARIANT INTERPRETATION:</strong></caption>
<tr>
    <td align=center>1.</td>
    <td> <p> The identified variant is located in the <strong>
$variant_location_1 </strong> region and is a <strong> $variant_type_1
</strong> which causes an amino acid change from <strong>
$aachange_firstA_firstT_1 </strong> to <strong> $aachange_lastA_firstT_1
</strong>. </p> </td>
</tr>
</table>" > new/variant_interpretation.txt

elif [ $clinsig == 2 ]
then
    echo "<br><br><table border='1' >
<caption><strong>VARIANT INTERPRETATION:</strong></caption>
<tr>
    <td align=center>1.</td>
    <td> <p> The identified variant is located in the <strong>
$variant_location_1 </strong> region and is a <strong> $variant_type_1
</strong> which causes an amino acid change from <strong>
$aachange_firstA_firstT_1 </strong> to <strong>$aachange_lastA_firstT_1
</strong>. </p> </td>
</tr>
<tr>
    <td align=center>2.</td>
    <td> <p> The identified variant is located in the <strong>
$variant_location_2 </strong> region and is a <strong> $variant_type_2
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_2 </strong> to
<strong>$aachange_lastA_firstT_2 </strong>. </p> </td>
</tr>
</table> " > new/variant_interpretation.txt

elif [ $clinsig == 3 ]
then
    echo "<br><br><table border='1' >
<caption><strong>VARIANT INTERPRETATION:</strong></caption>
<tr>
    <td align=center>1.</td>
    <td> <p> The identified variant is located in the <strong>
$variant_location_1 </strong> region and is a <strong> $variant_type_1
</strong> which causes an amino acid change from <strong>
$aachange_firstA_firstT_1 </strong> to <strong>$aachange_lastA_firstT_1
</strong>. </p> </td>
</tr>
<tr>
    <td align=center>2.</td>
    <td>
        <p> The identified variant is located in the <strong>
$variant_location_2 </strong> region and is a <strong> $variant_type_2

```



```

</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_2 </strong> to
<strong>$aachange_lastA_firstT_2 </strong>. </p> </td>
</tr>
<tr>
  <td align=center>3.</td>
<td>
  <p> The identified variant is located in the <strong>
$variant_location_3 </strong> region and is a <strong> $variant_type_3
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_3 </strong> to
<strong>$aachange_lastA_firstT_3 </strong>. </p> </td>
</tr>
</table>" > new/variant_interpretation.txt

elif [ $clinsig == 4 ]
then
echo "<br>&nbsp;<br><table border="1" >
<caption><strong>VARIANT INTERPRETATION:</strong></caption>
<tr>
  <td align=center>1.</td>
  <td> <p> The identified variant is located in the <strong>
$variant_location_1 </strong> region and is a <strong> $variant_type_1
</strong> which causes an amino acid change from <strong>
$aachange_firstA_firstT_1 </strong> to <strong>$aachange_lastA_firstT_1
</strong>. </p> </td>
</tr>
<tr>
  <td align=center>2.</td>
  <td>
  <p> The identified variant is located in the <strong>
$variant_location_2 </strong> region and is a <strong> $variant_type_2
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_2 </strong> to
<strong>$aachange_lastA_firstT_2 </strong>. </p> </td>
</tr>
<tr>
  <td align=center>3.</td>
  <td>
  <p> The identified variant is located in the <strong>
$variant_location_3 </strong> region and is a <strong> $variant_type_3
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_3 </strong> to
<strong>$aachange_lastA_firstT_3 </strong>. </p> </td>
</tr>
<tr>
  <td align=center>4.</td>
  <td>
  <p> The identified variant is located in the <strong>
$variant_location_4 </strong> region and is a <strong> $variant_type_4
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_4 </strong> to
<strong>$aachange_lastA_firstT_4 </strong>. </p> </td>
</tr>
</table>" > new/variant_interpretation.txt

elif [ $clinsig == 5 ]
then
echo "<br>&nbsp;<br><table border="1" >
<caption><strong>VARIANT INTERPRETATION:</strong></caption>

```

```

<tr>
  <td align=center>1.</td>
  <td> <p> The identified variant is located in the <strong>
$variant_location_1 </strong> region and is a <strong> $variant_type_1
</strong> which causes an amino acid change from <strong>
$aachange_firstA_firstT_1 </strong> to <strong>$aachange_lastA_firstT_1
</strong>. </p> </td>
</tr>
<tr>
  <td align=center>2.</td>
  <td>
    <p> The identified variant is located in the <strong>
$variant_location_2 </strong> region and is a <strong> $variant_type_2
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_2 </strong> to
<strong>$aachange_lastA_firstT_2 </strong>. </p> </td>
</tr>
<tr>
  <td align=center>3.</td>
  <td>
    <p> The identified variant is located in the <strong>
$variant_location_3 </strong> region and is a <strong> $variant_type_3
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_3 </strong> to
<strong>$aachange_lastA_firstT_3 </strong>. </p> </td>
</tr>
<tr>
  <td align=center>4.</td>
  <td>
    <p> The identified variant is located in the <strong>
$variant_location_4 </strong> region and is a <strong> $variant_type_4
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_4 </strong> to
<strong>$aachange_lastA_firstT_4 </strong>. </p> </td>
</tr>
<tr>
  <td align=center>5.</td>
  <td>
    <p> The identified variant is located in the <strong>
$variant_location_5 </strong> region and is a <strong> $variant_type_5
</strong> which causes an amino acid change from
<strong>$aachange_firstA_firstT_5 </strong> to
<strong>$aachange_lastA_firstT_5 </strong>. </p> </td>
</tr>
</table>" > new/variant_interpretation.txt

```

```
fi
```

```

#-----
#-----
#
# To create
and combine ALL files into REPORT!

mv chromatogram_hetero.pdf new/

#-----
#-----
# To obtain gene and ABI variables from MAIN script

```

```

gene_name=$(cat new/gene_name.txt)

ABI=$(cat new/ABI.txt)

cd new

mkdir REPORT

#-----
#-----
# To convert text files into HTML files using AWK scripts!

awk -f clinvar_observed.awk  clinvar_observed.txt > clinvar_observed.html

awk -f clinvar_detailed.awk  clinvar_detailed.txt > clinvar_detailed.html

awk -f condition_name.awk condition_name.txt > condition_name.html
awk -f test_code.awk test_code.txt > test_code.html

awk -f gene_summary.awk
/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/gene_humanDB/summary_$gene_name.txt > gene_summary.html

#-----
#-----
# To convert chromatogram PDF file into HTML file!

pdftohtml -c chromatogram_hetero.pdf chromatogram_hetero.html

#-----
#-----
# To merge all the HTML files!

cat body_head.html test_code.html condition_name.html Result.html
gene_summary.html clinvar_observed.html observed.html clinvar_detailed.html
detailed.html variant_interpretation.txt body_tail.html
chromatogram_title.html chromatogram_hetero-1.html
chromatogram_summary.html > REPORT/MANIPAL_report_$ABI.html

rm *.txt

rm -rf clinvar_observed.html condition_name.html test_code.html
gene_summary.html observed.html detailed.html Result.html
chromatogram_hetero-1.html

cd REPORT

#-----
#-----
# Converting HTML file into PDF file!

html2pdf MANIPAL_report_$ABI.html MANIPAL_report_$ABI.pdf

#-----
#-----

```

```

-----
# Opening the PDF file!

evince MANIPAL_report_${ABI}.pdf

cd ..

cd ..

```

4.10.2.2 Converting 1 letter amino acid abbreviations into full names

```

#!/bin/bash

word=$(cat aachange_firstT_1.txt | cut -d'.' -f2)

firstletter=${word:0:1}

echo $firstletter > aachange_firstA_firstT_1.txt

cat aachange_firstT_1.txt | cut -d'.' -f2 | awk '$0=$NF' FS= >
aachange_lastA_firstT_1.txt

# First row, First AA
aachange_firstA_1=$(cat aachange_firstA_firstT_1.txt)

aa_first_1_A=$(
if [[ $aachange_firstA_1 == "A" ]]
then
sed 's/A/Alanine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_A == "Alanine" ]]; then echo $aa_first_1_A >
aachange_firstA_firstT_1.txt; fi

aa_first_1_C=$(
if [[ $aachange_firstA_1 == "C" ]]
then
sed 's/C/Cysteine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_C == "Cysteine" ]]; then echo $aa_first_1_C >
aachange_firstA_firstT_1.txt; fi

aa_first_1_D=$(
if [[ $aachange_firstA_1 == "D" ]]
then
sed 's/D/Aspartic_Acid/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_D == "Aspartic_Acid" ]]; then echo $aa_first_1_D >
aachange_firstA_firstT_1.txt; fi

aa_first_1_E=$(
if [[ $aachange_firstA_1 == "E" ]]
then

```

```

        sed 's/E/Glutamic_Acid/g' aachange_firstA_firstT_1.txt
    fi)

    if [[ $aa_first_1_E == "Glutamic_Acid" ]]; then echo $aa_first_1_E >
aachange_firstA_firstT_1.txt; fi

    aa_first_1_F=$(
    if [[ $aachange_firstA_1 == "F" ]]
    then
        sed 's/F/Phenylalanine/g' aachange_firstA_firstT_1.txt
    fi)

    if [[ $aa_first_1_F == "Phenylalanine" ]]; then echo $aa_first_1_F >
aachange_firstA_firstT_1.txt; fi

    aa_first_1_G=$(
    if [[ $aachange_firstA_1 == "G" ]]
    then
        sed 's/G/Glycine/g' aachange_firstA_firstT_1.txt
    fi)

    if [[ $aa_first_1_G == "Glycine" ]]; then echo $aa_first_1_G >
aachange_firstA_firstT_1.txt; fi

    aa_first_1_H=$(
    if [[ $aachange_firstA_1 == "H" ]]
    then
        sed 's/H/Histidine/g' aachange_firstA_firstT_1.txt
    fi)

    if [[ $aa_first_1_H == "Histidine" ]]; then echo $aa_first_1_H >
aachange_firstA_firstT_1.txt; fi

    aa_first_1_I=$(
    if [[ $aachange_firstA_1 == "I" ]]
    then
        sed 's/I/Isoleucine/g' aachange_firstA_firstT_1.txt
    fi)

    if [[ $aa_first_1_I == "Isoleucine" ]]; then echo $aa_first_1_I >
aachange_firstA_firstT_1.txt; fi

    aa_first_1_K=$(
    if [[ $aachange_firstA_1 == "K" ]]
    then
        sed 's/K/Lysine/g' aachange_firstA_firstT_1.txt
    fi)

    if [[ $aa_first_1_K == "Lysine" ]]; then echo $aa_first_1_K >
aachange_firstA_firstT_1.txt; fi

    aa_first_1_L=$(
    if [[ $aachange_firstA_1 == "L" ]]
    then
        sed 's/L/Leucine/g' aachange_firstA_firstT_1.txt
    fi)

```

```

if [[ $aa_first_1_L == "Leucine" ]]; then echo $aa_first_1_L >
aachange_firstA_firstT_1.txt; fi

aa_first_1_M=$(
if [[ $aachange_firstA_1 == "M" ]]
then
sed 's/M/Methionine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_M == "Methionine" ]]; then echo $aa_first_1_M >
aachange_firstA_firstT_1.txt; fi

aa_first_1_N=$(
if [[ $aachange_firstA_1 == "N" ]]
then
sed 's/N/Asparagine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_N == "Asparagine" ]]; then echo $aa_first_1_N >
aachange_firstA_firstT_1.txt; fi

aa_first_1_P=$(
if [[ $aachange_firstA_1 == "P" ]]
then
sed 's/P/Proline/g' $AAfirst aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_P == "Proline" ]]; then echo $aa_first_1_P >
aachange_firstA_firstT_1.txt; fi

aa_first_1_Q=$(
if [[ $aachange_firstA_1 == "Q" ]]
then
sed 's/Q/Glutamine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_Q == "Glutamine" ]]; then echo $aa_first_1_Q >
aachange_firstA_firstT_1.txt; fi

aa_first_1_R=$(
if [[ $aachange_firstA_1 == "R" ]]
then
sed 's/R/Arginine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_R == "Arginine" ]]; then echo $aa_first_1_R >
aachange_firstA_firstT_1.txt; fi

aa_first_1_S=$(
if [[ $aachange_firstA_1 == "S" ]]
then
sed 's/S/Serine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_S == "Serine" ]]; then echo $aa_first_1_S >

```

```

aachange_firstA_firstT_1.txt; fi

aa_first_1_T=$(
if [[ $aachange_firstA_1 == "T" ]]
then
sed 's/T/Threonine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_T == "Threonine" ]]; then echo $aa_first_1_T >
aachange_firstA_firstT_1.txt; fi

aa_first_1_V=$(
if [[ $aachange_firstA_1 == "V" ]]
then
sed 's/V/Valine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_V == "Valine" ]]; then echo $aa_first_1_V >
aachange_firstA_firstT_1.txt; fi

aa_first_1_W=$(
if [[ $aachange_firstA_1 == "W" ]]
then
sed 's/W/Tryptophan/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_W == "Tryptophan" ]]; then echo $aa_first_1_W >
aachange_firstA_firstT_1.txt; fi

aa_first_1_Y=$(
if [[ $aachange_firstA_1 == "Y" ]]
then
sed 's/Y/Tyrosine/g' aachange_firstA_firstT_1.txt
fi)

if [[ $aa_first_1_Y == "Tyrosine" ]]; then echo $aa_first_1_Y >
aachange_firstA_firstT_1.txt; fi

# First row, Last AA

aachange_lastA_1=$(cat aachange_lastA_firstT_1.txt)

aa_last_1_A=$(
if [[ $aachange_lastA_1 == "A" ]]
then
sed 's/A/Alanine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_A == "Alanine" ]]; then echo $aa_last_1_A >
aachange_lastA_firstT_1.txt; fi

aa_last_1_C=$(
if [[ $aachange_lastA_1 == "C" ]]
then
sed 's/C/Cysteine/g' aachange_lastA_firstT_1.txt

```

```

fi)

if [[ $aa_last_1_C == "Cysteine" ]]; then echo $aa_last_1_C >
aachange_lastA_firstT_1.txt; fi

aa_last_1_D=$(
if [[ $aachange_lastA_1 == "D" ]]
then
sed 's/D/Aspartic_Acid/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_D == "Aspartic_Acid" ]]; then echo $aa_last_1_D >
aachange_lastA_firstT_1.txt; fi

aa_last_1_E=$(
if [[ $aachange_lastA_1 == "E" ]]
then
sed 's/E/Glutamic_Acid/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_E == "Glutamic_Acid" ]]; then echo $aa_last_1_E >
aachange_lastA_firstT_1.txt; fi

aa_last_1_F=$(
if [[ $aachange_lastA_1 == "F" ]]
then
sed 's/F/Phenylalanine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_F == "Phenylalanine" ]]; then echo $aa_last_1_F >
aachange_lastA_firstT_1.txt; fi

aa_last_1_G=$(
if [[ $aachange_lastA_1 == "G" ]]
then
sed 's/G/Glycine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_G == "Glycine" ]]; then echo $aa_last_1_G >
aachange_lastA_firstT_1.txt; fi

aa_last_1_H=$(
if [[ $aachange_lastA_1 == "H" ]]
then
sed 's/H/Histidine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_H == "Histidine" ]]; then echo $aa_last_1_H >
aachange_lastA_firstT_1.txt; fi

aa_last_1_I=$(
if [[ $aachange_lastA_1 == "I" ]]
then
sed 's/I/Isoleucine/g' aachange_lastA_firstT_1.txt

```



```

fi)

if [[ $aa_last_1_I == "Isoleucine" ]]; then echo $aa_last_1_I >
aachange_lastA_firstT_1.txt; fi

aa_last_1_K=$(
if [[ $aachange_lastA_1 == "K" ]]
then
sed 's/K/Lysine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_K == "Lysine" ]]; then echo $aa_last_1_K >
aachange_lastA_firstT_1.txt; fi

aa_last_1_L=$(
if [[ $aachange_lastA_1 == "L" ]]
then
sed 's/L/Leucine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_L == "Leucine" ]]; then echo $aa_last_1_L >
aachange_lastA_firstT_1.txt; fi

aa_last_1_M=$(
if [[ $aachange_lastA_1 == "M" ]]
then
sed 's/M/Methionine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_M == "Methionine" ]]; then echo $aa_last_1_M >
aachange_lastA_firstT_1.txt; fi

aa_last_1_N=$(
if [[ $aachange_lastA_1 == "N" ]]
then
sed 's/N/Asparagine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_N == "Asparagine" ]]; then echo $aa_last_1_N >
aachange_lastA_firstT_1.txt; fi

aa_last_1_P=$(
if [[ $aachange_lastA_1 == "P" ]]
then
sed 's/P/Proline/g' $AAfirst aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_P == "Proline" ]]; then echo $aa_last_1_P >
aachange_lastA_firstT_1.txt; fi

aa_last_1_Q=$(
if [[ $aachange_lastA_1 == "Q" ]]
then
sed 's/Q/Glutamine/g' aachange_lastA_firstT_1.txt
fi)

```

```

if [[ $aa_last_1_Q == "Glutamine" ]]; then echo $aa_last_1_Q >
aachange_lastA_firstT_1.txt; fi

aa_last_1_R=$(
if [[ $aachange_lastA_1 == "R" ]]
then
sed 's/R/Arginine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_R == "Arginine" ]]; then echo $aa_last_1_R >
aachange_lastA_firstT_1.txt; fi

aa_last_1_S=$(
if [[ $aachange_lastA_1 == "S" ]]
then
sed 's/S/Serine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_S == "Serine" ]]; then echo $aa_last_1_S >
aachange_lastA_firstT_1.txt; fi

aa_last_1_T=$(
if [[ $aachange_lastA_1 == "T" ]]
then
sed 's/T/Threonine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_T == "Threonine" ]]; then echo $aa_last_1_T >
aachange_lastA_firstT_1.txt; fi

aa_last_1_V=$(
if [[ $aachange_lastA_1 == "V" ]]
then
sed 's/V/Valine/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_V == "Valine" ]]; then echo $aa_last_1_V >
aachange_lastA_firstT_1.txt; fi

aa_last_1_W=$(
if [[ $aachange_lastA_1 == "W" ]]
then
sed 's/W/Tryptophan/g' aachange_lastA_firstT_1.txt
fi)

if [[ $aa_last_1_W == "Tryptophan" ]]; then echo $aa_last_1_W >
aachange_lastA_firstT_1.txt; fi

aa_last_1_Y=$(
if [[ $aachange_lastA_1 == "Y" ]]
then
sed 's/Y/Tyrosine/g' aachange_lastA_firstT_1.txt
fi)

```

```
if [[ $aa_last_1_Y == "Tyrosine" ]]; then echo $aa_last_1_Y >
aachange_lastA_firstT_1.txt; fi
```

4.10.2.3 Web page template for generating the genomic report

```
<html>

<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />

<head>

<style type="text/css">
body {background-color:White}
h1   {background-color:White;line-height: 30px;}
hr   {background-color:Black}
p    {text-align:Left}
img  {position: relative; top: 40px;}
table td { border: 1px solid black;}

</style>

</head>

<body bgcolor="LightGray">

<script type="text/javascript">
tday=new
Array("Sunday","Monday","Tuesday","Wednesday","Thursday","Friday","Saturday
");
tmonth=new
Array("January","February","March","April","May","June","July","August","Se
ptember","October","November","December");

function GetClock(){
var d=new Date();
var
nday=d.getDay(),nmonth=d.getMonth(),ndate=d.getDate(),nyear=d.getYear();
if(nyear<1000) nyear+=1900;
var d=new Date();
var nhour=d.getHours(),nmin=d.getMinutes(),nsec=d.getSeconds(),ap;

if(nhour==0){ap=" AM";nhour=12;}
else if(nhour<12){ap=" AM";}
else if(nhour==12){ap=" PM";}
else if(nhour>12){ap=" PM";nhour-=12;}

if(nmin<=9) nmin="0"+nmin;
if(nsec<=9) nsec="0"+nsec;

document.getElementById('clockbox').innerHTML="" +tday[nday]+"",
"+tmonth[nmonth]+" "+ndate+", "+nyear+" "+nhour+": "+nmin+": "+nsec+ap+"";
}

window.onload=function(){
GetClock();
}
</script>
```

```

<img style=' float: left; height: 100px; width: 100px; margin-right: 4px;'
src='/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/logo_genomic_
report.png' />
<h1 align="center"> MANIPAL System Diagnostic Report</h1>
<div align="right">
  <div>
    <div>

      <address >
        <strong>Molecular and Cellular Biology Unit</strong><br
/> School of Life Sciences<br /> Manipal University, Manipal - 576 104<br
/>

        <div>
          Tel: 0820 2922058; 0820 2923501
        </div>
        <div>
          Email: dbt@manipal.edu
        </div>
      </address>
    </div>
  </div>
</div>

<table>

<tr>
  <th bgcolor="LightGray" align="center" rowspan="1" width=1500
align="left">Report Date:</th>
</tr>

<tr>
  <td align="center"><div id="clockbox"></div></td>
</tr>

<tr>
  <th bgcolor="LightGray" width=1500 align="left">Patient Name:</th>
</tr>

<tr>
  <td align="left">&nbsp;</td>
</tr>

<tr>
  <th bgcolor="LightGray" width=1500 align="left">Specimen Type:</th>
</tr>

<tr>
  <td align="left">&nbsp;</td>
</tr>

<tr>
  <th bgcolor="LightGray" width=1500 align="left">Gender:</th>
</tr>

<tr>
  <td align="left">&nbsp;</td>
</tr>

```

```

<tr>
  <th bgcolor="LightGray" width=1500 align="left">Referring
Physician:</th>
</tr>

<tr>
  <td align="left">&nbsp;</td>
</tr>

<tr>
  <th bgcolor="LightGray" width=1500 align="left">Referring
Department:</th>
</tr>

<tr>
  <td align="left">&nbsp;</td>
</tr>

</table>

<hr size="5">

```

Chapter 5

RESULTS

ABI files of 11 diagnostic genes were considered for the current study and successfully processed using the MANIPAL system and clinically significant mutations were identified.

The complete workflow of the MANIPAL system is automated using bash-programming language. The MANIPAL system consists of six major steps. The pipeline begins with the base-calling step and the resultant files are converted into a FASTQ file (**Figure 8**) by using a BioPerl script. This is followed by mapping the FASTQ FILE containing the read with the reference sequence and post processing is performed on the alignment to obtain the BAM file. The final two steps (**Figure 4**) are specific to calling the variants and annotating them.

5.1 Intermediate files

Step 1 results in FASTA and quality files (**Figure 6 and Figure 7**). For Step 2, the FASTA and quality files were used resulting in a FASTQ file (**Figure 8**). The alignment of the FASTQ files produces a SAM file as output (**Figure 9**).

```
>Seq_E04_W93_EX12_F_010.ab1 Reversed: 134 0 134 ABI trimmed
CAGTAGATTATTTAAACACAACCACCATATAGCCCAAGGCATTCAACTTACAGGAAAGT
ATTTCTGAACAACACCAAAATCGATAAAACCGATTACAATCCATACCACCAACGTCAAAG
TGGACATGATGATG
```

Figure 6: FASTA sequence of the input ABI file.

```
>Seq_E04_W93_EX12_F_010.ab1 134 0 134 ABI trimmed
10 10 10 14 18 28 33 27 22 19 10 10 10 10 10 15 18
21 23 25 22 12 11 9 11 14 20 30 22 22 11 12 10 22
14 15 15 19 21 20 20 22 23 32 32 19 16 19 19 20 23
23 16 16 10 10 10 18 19 23 23 30 37 37 37 37 37 30
25 18 16 21 18 21 32 33 47 47 50 42 37 23 22 15 10
10 10 18 23 30 28 28 32 37 33 33 33 33 33 33 37 48
53 57 57 50 57 50 50 50 50 50 50 50 53 50 50 50 37
37 42 41 39 32 32 32 32 37 41 41 42 42 40
```

Figure 7: The quality values corresponding to each bases in the FASTA file.

```
@Seq_E04_W93_EX12_F_010.ab1
CAGTAGATTATTTAAACACAACCACCATATAGCCCAAGGCATTCAACTTACAGGAAAGTATTTCTGAACAACACCAAATCGA
TAAACCGATTACAATCCATACCACCAACGTCAAAGTGGACATGATGATG
+
+++/3=B<74+++++0368:7-,*,/5?77,-+7/00465578AA414458811+++3488?FFFFF?:
31636ABPPSKF870+++38?==AFBBBBBBFQVZZSZSSSSSSSVSSSFFKJHAAAAAFJJKKI
```

Figure 8: FASTQ file containing the FASTA sequence along with the quality values.

```
@SQ      SN:chr13      LN:79377
@PG      ID:bwa      PN:bwa      VN:0.7.8-r455      CL:bwa mem ALL fastq/merged.fastq
Seq_E04_W93_EX12_F_010.ab1      0      chr13      16942      60      134M      *      0      0

CAGTAGATTATTTAAACACAACCACCATATAGCCCAAGGCATTCAACTTACAGGAAAGTATTTCTGAACAACACCAAATCGATAAAA
CCGATTACAATCCATACCACCAACGTCAAAGTGGACATGATGATG      +++/3=B<74+++++0368:7-,*,/5?77,-
+7/00465578AA414458811+++3488?FFFFF?:31636ABPPSKF870+++38?
==AFBBBBBBFQVZZSZSSSSSSSVSSSFFKJHAAAAAFJJKKI      NM:i:2      MD:Z:62C58T12      AS:i:124
XS:i:0
```

Figure 9: The aligned data in SAM format.

5.2 Visualizing Heterozygous peaks in chromatogram

To check for the heterozygous peaks, the chromatogram is obtained (**Figure 10**) from the “sangerseqR” package of Bioconductor. The bases on the first line represent the primary base calling whereas the bases on the second line represent the secondary base calling. The 107th position in the chromatogram is the “A” nucleotide, which is the location of the mutation in the forward ABI file of WISP3 gene. This is a homozygous peak as the primary and secondary base calling at that particular position is similar.

5.3 BLAST Alignment

Visualizing the alignment using Basic Local Alignment Search Tool (BLAST)— A local nucleotide BLAST database is created for the gene being analyzed using the “makeblastdb” command from the BLAST+ suite (v2.2.28, build Jun 3 2013) (Camacho *et al.*, 2008). The trimmed FASTA sequence containing high quality bases that is obtained from Phred is used as the nucleotide query and a BLAST search is performed against the created BLAST database using the “blastn” command. The position of the variant present in the VCF file (**Figure 11 A**), produced after calling the variants, matches with the BLAST alignment (**Figure 11 B**). Also by inspecting the BAM file in the Integrative Genomics Viewer (IGV) (**Figure 12**), the variant’s position is similar to that in the VCF file.

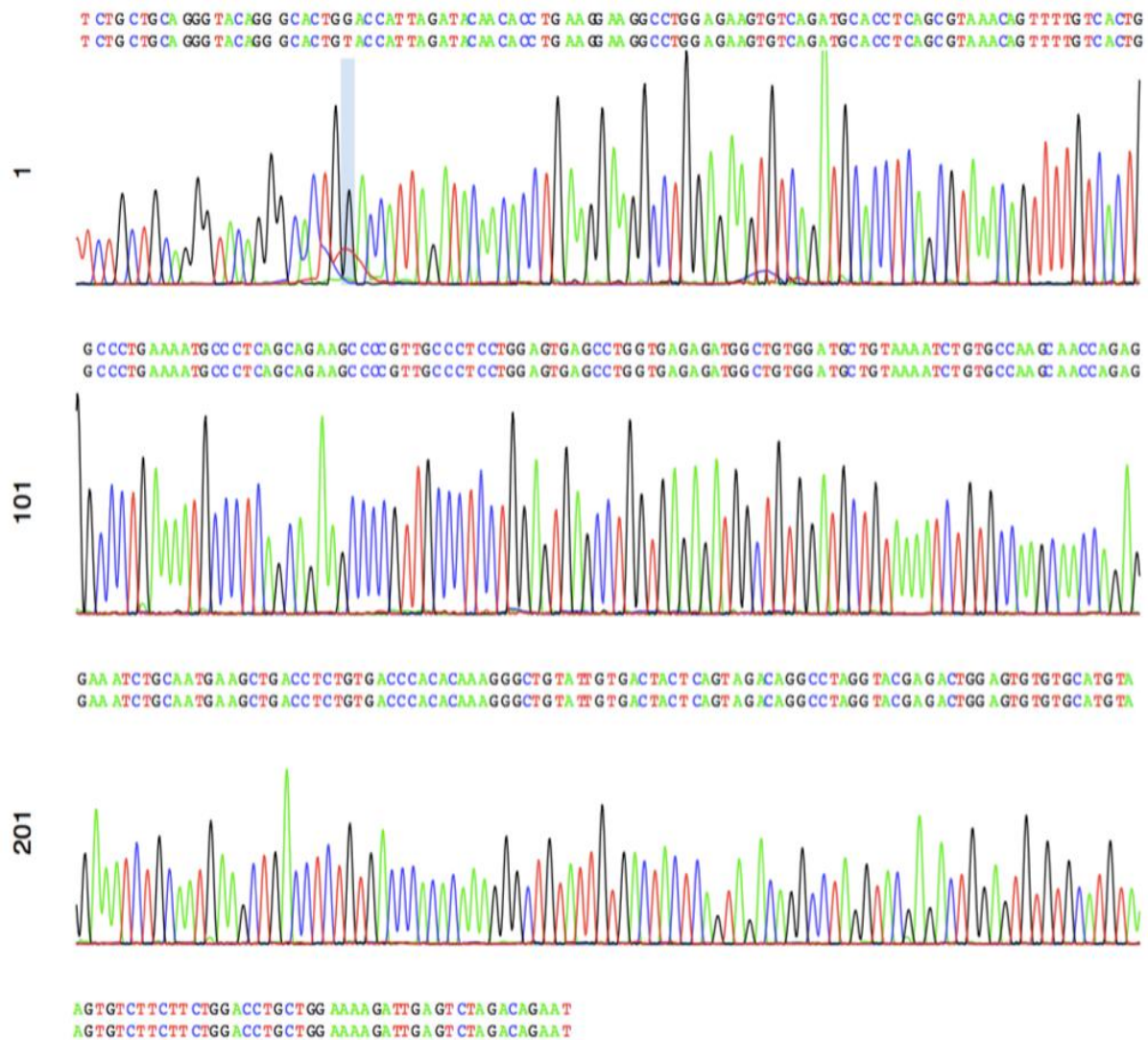


Figure 10: The chromatogram generated by the Bioconductor package called “sangerseqR” highlights the peak that is heterozygous.

A

HBB VCF file

#CHROM	POS	ID	REF	ALT
chr11	1460.		C	G



B

BLAST alignment

```

Query 1      ACGTGACAGCTTGTACAGTGCAGCTCACTCAGTGTGGCAAAGGTGCCCTTGAGGTTGTCC 60
Sbjct 1131   ACGTGACAGCTTGTACAGTGCAGCTCACTCAGTGTGGCAAAGGTGCCCTTGAGGTTGTCC 1190
Query 61     AGGTGAGCCAGGCCATCACTAAAGGCACCGAGCACTTTCTTGCCATGAGCCTTCACCTTA 120
Sbjct 1191   AGGTGAGCCAGGCCATCACTAAAGGCACCGAGCACTTTCTTGCCATGAGCCTTCACCTTA 1250
Query 121    GGGTTGCCCATAAACAGCATCAGGAGTGGACAGATCCCCAAAGGACTCAAAGAACCTCTGG 180
Sbjct 1251   GGGTTGCCCATAAACAGCATCAGGAGTGGACAGATCCCCAAAGGACTCAAAGAACCTCTGG 1310
Query 181    GTCCAAGGGTAGACCACCGAGCAGCTAAGGGTGGGAAAATAGACCAATAGGCAGAGAGAG 240
Sbjct 1311   GTCCAAGGGTAGACCACCGAGCAGCTAAGGGTGGGAAAATAGACCAATAGGCAGAGAGAG 1370
Query 241    TCAGTGCCTATCAGAAACCAAGAGTCTTCTGTCTCCACATGCCCAGTTTCTATTGGT 300
Sbjct 1371   TCAGTGCCTATCAGAAACCAAGAGTCTTCTGTCTCCACATGCCCAGTTTCTATTGGT 1430
Query 301    CTCCTTAAACCTGTCTTGTAACTTGAAGCACTGCCAGGGCCTCACCACCAACTTC 360
Sbjct 1431   CTCCTTAAACCTGTCTTGTAACTTGAAGCACTGCCAGGGCCTCACCACCAACTTC 1490
Query 361    ATCCACGTTACCTTGCCCCACAGGGCAGTAACGGCAGACTTCTCCTCAGGAGTCAGATG 420
Sbjct 1491   ATCCACGTTACCTTGCCCCACAGGGCAGTAACGGCAGACTTCTCCTCAGGAGTCAGATG 1550
Query 421    CACCATGGTGTCTGTTTGAGGTTGCTAGTGAACACAGTTGTGTGAGAAGCAAATGT 476
Sbjct 1551   CACCATGGTGTCTGTTTGAGGTTGCTAGTGAACACAGTTGTGTGAGAAGCAAATGT 1606

```

Figure 11: BLAST Validation

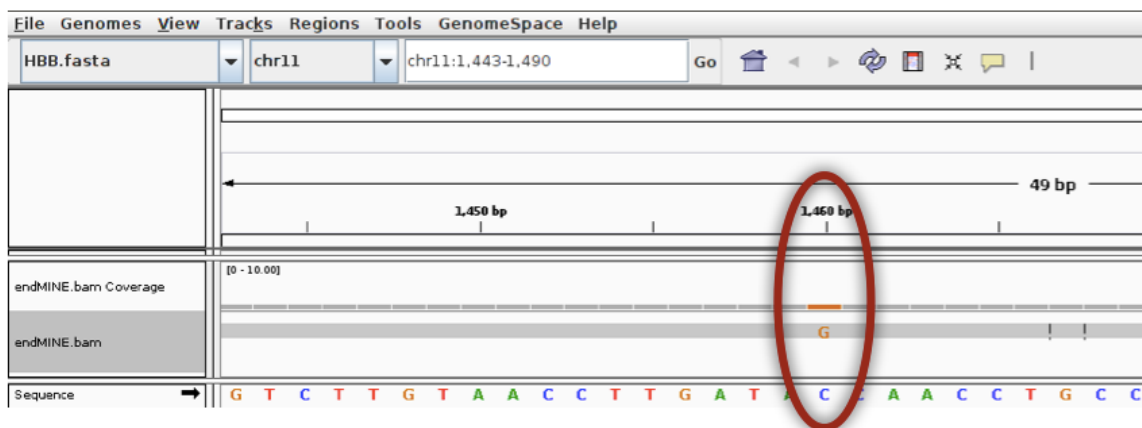


Figure 12: IGV BAM Visualization

5.4 ANNOVAR's annotated output

ANNOVAR produces a tab-delimited file containing all the annotated features of the variants.

A

Gene Name	Chr	Start	End	Ref	Alt	RefGene Function	RefGene Exonic Function	RefGene Amino Acid Change	snp138	ClinVar
WISP3	chr6	112382301	112382301	C	A	exonic	stopgain	WISP3-NM_003880:exon3:c.156C>A:p.C52X	rs121908901	CLNLSIG=pathogenic. CLNDBN=Progressive_pseudorheumatoid_dysplasia. CLNACC=RCV000006753.1 CLNDSDB=MedGen, OMIM, Orphanet. CLNDSDBID=C0432215, 208230, ORPHA1159.

B

Gene Name	Chr	Start	End	Ref	Alt	RefGene Function	RefGene Exonic Function	RefGene Amino Acid Change	snp138	ClinVar
WISP3	chr6	112382301	112382301	C	A	exonic	stopgain	WISP3-NM_003880:exon3:c.156C>A:p.C52X	rs121908901	CLNLSIG=pathogenic. CLNDBN=Progressive_pseudorheumatoid_dysplasia. CLNACC=RCV000006753.1 CLNDSDB=MedGen, OMIM, Orphanet. CLNDSDBID=C0432215, 208230, ORPHA1159.

Figure 13: A. ANNOVAR output for the forward ABI file of WISP3 gene displaying annotated features for the clinically significant mutation. **B.** Annotated result of ANNOVAR for the reverse ABI file of WISP3 gene. “Gene Name” column refers to the diagnostic gene analyzed by the MANIPAL system. “Chr” refers to the chromosomal location whereas the “Start” and “End” refer the genomic start and end co-ordinates of the mutation. The “RefGene Function” column refers to the location of the mutation and the “RefGene Exonic Function” represents the functional consequences of the mutation. The “RefGene Amino Acid Change” column displays the change in the amino acid caused by the mutation. The “rsID” represents the dbSNP identifier for the variant according to the Build 138 of the dbSNP database (<http://www.ncbi.nlm.nih.gov/SNP>). The “clinvar” column represents the annotations according to the ClinVar database (<http://www.ncbi.nlm.nih.gov/clinvar/>) which contains the class of the clinically significant variant, the disease associated with the variant, the ClinVar database accession and versions as well as the variants disease and database name along with the respective identifiers.

- WISP3 Forward ABI file annotation - There is a nucleotide change from C>A at the “112382301” genomic location, and the homozygous mutation causes Cys52-to-ter substitution in exon 3 in the NM_003880 transcript (**Figure 13 A**).
- WISP3 Reverse ABI file annotation - The annotated variant of the reverse ABI file (**Figure 13 B**) matches with the annotated results of the forward ABI file and since the mutation is present in the reverse strand also, the variant called is a true one and not a sequencing error.

5.5 Genomic Report

The genomic report is made up of nine sections, each conveying different information's. The first section deals with the details of the patient; the type of sample that is used; the patient's gender as well as the name and the department of the doctor who has requested for the diagnostic test to be performed. The second section contains the alphanumeric code number of the test being carried out, which helps in tracking the related details easily. Based on the "Test Code", this segment also contains the name of the condition for which the test is done.

The third section is the most crucial part of the report. This directly conveys the message directly to the physician, whether the patient has been tested "Positive" or "Negative" for the test. The working of this depends on the annotation done by ANNOVAR. If there are any annotation of ClinVar database in the ANNOVAR's output file, then the patient has a clinically significant mutation present and is said to be "Positive" for the diagnostic test.

If there are no ClinVar database annotations in the ANNOVAR's output file, then the patient does not have any known clinically significant mutation and is said to be "Negative" for the diagnostic test.

A brief summary of the gene being examined is provided in the fourth section. The "Observed DNA variants" section follows the HGVS nomenclature to report the observed variants. The details are provided in this section only when clinically significant mutations are identified during the analysis. And if present, then this section includes the important elements that communicate the details clearly. It consists of:

1. The gene name that is analyzed.
2. The name of the disease that the variant is known to cause.
3. The nomenclature of the identified variant at the level of genomic as well as complementary DNA and
4. The class under which the variant is classified, signifying the effect of it.

The "Detailed Variant Information" includes specific features of the observed variants. It consists of:

1. The gene name as well as the transcript that is affected by the variant.

2. The class of the variant signifies whether the variant causes any functional effect at the protein level.
3. This tells whether the variant is located in the intronic to exonic or intergenic.
4. This indicates that at position 952 there is an amino acid change from Arginine to Lysine in a transcript called “NM_000053”, the annotation of which is present under the “Gene Name”.
5. If the variant is not novel, then the dbSNP identifier i.e, the rsID of the variant is reported, if it is present in the dbSNP database.

The next section contains the MAF score from 1000 Genomes Project; sift score and its prediction as well as polyphen score and its prediction. A SIFT score of less than 0.05 is said to be “deleterious” whereas a score greater than 0.05 is said to be “tolerated”. In the same way, a polyphen score of less than or equal to 0.446 is said to be “benign”; less than or equal to 0.909 is said to be “possibly damaging” and greater than or equal to 0.909 is said to be “probably damaging”.

The “Variant Interpretation” section is a descriptive explanation of the reported variants for clear understanding.

The last section contains the chromatogram plot generated from the ABI file by the “sangerseqR” package of Bioconductor. This displays the primary as well as secondary base calling and helps in differentiating between the homozygous and heterozygous peaks. When the primary and secondary base calling differs, then the particular heterozygous peak is highlighted in blue for recognition.

MANIPAL System Diagnostic Report



Molecular and Cellular Biology Unit
 School of Life Sciences
 Manipal University, Manipal - 576 104
 Tel: 0820 2922058; 0820 2923501
 Email: dbt@manipal.edu

Report Date:
Friday, July 10, 2015 6:55:55 PM
Patient Name:
Specimen Type:
Gender:
Referring Physician:
Referring Department:

TEST CODE - WIL006
 CONDITION DIAGNOSED - Wilson's Disease

RESULT: Positive - A known variant causing the diagnosed condition was identified in the gene.

Gene Description:

ATP7B gene is a member of the P-type cation transport ATPase family and encodes a protein with several membrane-spanning domains, an ATPase consensus sequence, a hinge domain, a phosphorylation site, and at least 2 putative copper-binding sites. This protein functions as a monomer, exporting copper out of the cells, such as the efflux of hepatic copper into the bile. Alternate transcriptional splice variants, encoding different isoforms with distinct cellular localizations, have been characterized. Mutations in this gene have been associated with Wilson disease (WD). [provided by RefSeq, Jul 2008]

OBSERVED DNA VARIANTS:

1 Clinically significant mutation has been identified for the diagnosed condition.

	Gene Name	Disease	Location of the Variant	Classification
1.	ATP7B	Wilson's disease	chr13:52523808C>T c.2855G>A	Non-pathogenic

DETAILED VARIANT INFORMATION:

Annotations of the Clinically significant mutation.

	Gene Name	Variant Type	Variant Location	Amino Acid Change	dbSNP Identifier
1.	ATP7B (RefSeq ID: NM_000053)	nonsynonymous SNV	exonic	p.R952K	rs732774

MINOR ALLELE FREQUENCY and PREDICTED IMPACT OF VARIANT:

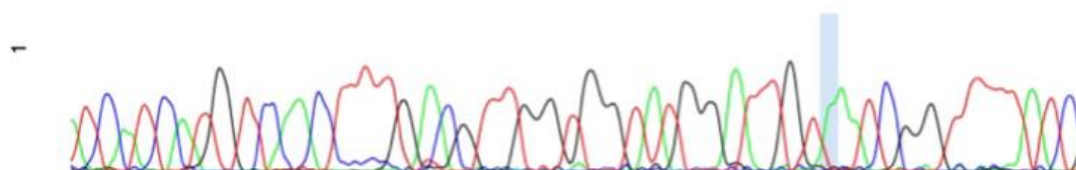
	MAF from 1000 Genomes Project	SIFT score and prediction	Polyphen score and prediction
1.	0.53	0.99 and Tolerated	0.03 and Benign

VARIANT INTERPRETATION:

1. The identified variant is located in the **exonic** region and is a **nonsynonymous SNV** which causes an amino acid change from **Arginine** to **Lysine**.

Chromatogram plot of the ABI file

T C A T C A T G T C A C T T T G A C G T T G G T G G T A T G G A T T G T G A A T C G G T T T A T C
T C A T C A T G T C A C T T T G A C G T T G G T G G T A T G G A T T G T C A A T C G G T T T A T C



The chromatogram was generated using the "sangerseqR" package of Bioconductor. When heterozygous peaks are present, it can be differentiated from the homozygous peaks and appropriate bases are called. In the above plot, the primary and secondary basecalls are similar for the homozygous peaks whereas the primary and secondary basecalls do not match for the heterozygous regions and such peaks are highlighted in blue.

Figure 14: MANIPAL System's genomic report.

5.6 The generated HTML code of the MANIPAL System report.

```
<html>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<head>
<style type="text/css">
body {background-color:White}
h1 {background-color:White;line-height: 30px;}
hr {background-color:Black}
p {text-align:Left}
img {position: relative; top: 40px;}
table td { border: 1px solid black;}
</style>
</head><body bgcolor="LightGray">
<script type="text/javascript">
tday=new
Array("Sunday","Monday","Tuesday","Wednesday","Thursday","Friday","Saturday
");
tmonth=new
Array("January","February","March","April","May","June","July","August","Se
ptember","October","November","December");

function GetClock(){
var d=new Date();
var
nday=d.getDay(),nmonth=d.getMonth(),ndate=d.getDate(),nyear=d.getYear();
```

```

if(nyear<1000) nyear+=1900;
var d=new Date();
var nhour=d.getHours(),nmin=d.getMinutes(),nsec=d.getSeconds(),ap;

if(nhour==0){ap=" AM";nhour=12;}
else if(nhour<12){ap=" AM";}
else if(nhour==12){ap=" PM";}
else if(nhour>12){ap=" PM";nhour-=12;}

if(nmin<=9) nmin="0"+nmin;
if(nsec<=9) nsec="0"+nsec;

document.getElementById('clockbox').innerHTML="" + tday[nday] + ",
"+ tmonth[nmonth] + " " + ndate + ", " + nyear + " " + nhour + ":" + nmin + ":" + nsec + ap + "";
}

window.onload=function(){
GetClock();
}
</script>

<img style=' float: left; height: 100px; width: 100px; margin-right: 4px;'
src='/home/sols/Documents/anil/PIPELINE_NEW_ABI_files/working/logo_genomic_
report.png' />
<h1 align="center"> MANIPAL System Diagnostic Report</h1>
<div align="right">
    <div>
        <div>
            <address >
                <strong>Molecular and Cellular Biology Unit</strong><br
/> School of Life Sciences<br /> Manipal University, Manipal - 576 104<br
/>

                <div>
                    Tel: 0820 2922058; 0820 2923501
                </div>
                <div>
                    Email: dbt@manipal.edu
                </div>
            </address>
        </div>
    </div>
</div>

<table>

<tr>
    <th bgcolor="LightGray" align="center" rowspan="1" width=1500
align="left">Report Date:</th>
</tr>

<tr>
    <td align="center"><div id="clockbox"></div></td>
</tr>

<tr>
    <th bgcolor="LightGray" width=1500 align="left">Patient Name:</th>
</tr>

```


into the bile. Alternate transcriptional splice variants, encoding different isoforms with distinct cellular localizations, have been characterized. Mutations in this gene have been associated with Wilson disease (WD). [provided by RefSeq, Jul 2008]</p>

```
<table ><tr><th width=1500 align=>OBSERVED DNA VARIANTS:</th></tr>
<tr><td width=1500 align=>1 Clinically significant mutation has been
identified for the diagnosed condition.</td></tr></table>
```

```

<table border=1 >
<tr>
  <th bgcolor=LightGray>&nbsp;</th>
  <th bgcolor=LightGray width=375 align=center>Gene Name</th>
  <th bgcolor=LightGray width=375 align=center>Disease</th>
  <th bgcolor=LightGray width=375 align=center>Location of the
Variant</th>
  <th bgcolor=LightGray width=375 align=center>Classification</th>
</tr>
<tr>
<td align=center>1.</td>
<td width=375 align=center>ATP7B</td>
<td width=375 align=center>Wilson's disease</td>
<td width=375 align=center>chr13:52523808C>T <br/>
c.2855G>A </td>
<td width=375 align=center>Non-pathogenic</td>
</tr>
</table><br>&nbsp;<br>
<table ><tr><th width=1500 align=>DETAILED VARIANT INFORMATION:</th></tr>
<tr><td width=1500 align=>Annotations of the Clinically significant
mutation.</td></tr></table>

```

```

<table border=1 >
<tr>
  <th bgcolor=LightGray>&nbsp;</th>
  <th width=375 bgcolor=LightGray align=center>Gene Name</th>
  <th width=375 bgcolor=LightGray align=center>Variant Type</th>
  <th width=375 bgcolor=LightGray align=center>Variant
Location</th>
  <th width=375 bgcolor=LightGray align=center>Amino Acid
Change</th>
  <th width=375 bgcolor=LightGray align=center>dbSNP
Identifier</th>
</tr>
<tr>
<td align=center>1.</td>
<td width=375 align=center>ATP7B (RefSeq ID: NM_000053)</td>
<td width=375 align=center>nonsynonymous SNV</td>
<td width=375 align=center>exonic</td>
<td width=375 align=center>p.R952K</td>
<td width=375 align=center>rs732774</td>
</tr></table><br>&nbsp;<br>
<table border=1 >
<caption><strong>MINOR ALLELE FREQUENCY and PREDICTED IMPACT OF
VARIANT:</strong></caption>
<tr>
  <th bgcolor=LightGray>&nbsp;</th>
  <th width=375 bgcolor=LightGray align=center>MAF from 1000
Genomes Project</th>

```

```

        <th width=375 bgcolor=LightGray align=center>SIFT score and
prediction</th>
        <th width=375 bgcolor=LightGray align=center>Polyphen score and
prediction</th>
</tr>

<tr>
    <td align=center>1.</td>
    <td width=375 align=center>0.53</td>
    <td width=375 align=center>0.99 and Tolerated</td>
    <td width=375 align=center>0.03 and Benign</td>
</tr> </table> <br>&nbsp;<br>
<br>&nbsp;<br><table border=1 >
    <caption><strong>VARIANT INTERPRETATION:</strong></caption>
<tr>
    <td align=center>1.</td>
    <td> <p> The identified variant is located in the <strong> exonic
</strong> region and is a <strong> nonsynonymous SNV </strong> which causes
an amino acid change from <strong> Arginine </strong> to <strong> Lysine
</strong>. </p> </td>
</tr>
</table>
</body>
</html>
<html
<body>
<br>&nbsp;<br>
<br>&nbsp;<br>
<table>

<p><strong>Chromatogram plot of the ABI file</strong></p>
<br>&nbsp;<br>

</table>
</body>

</html>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" lang="" xml:lang="">
<head>
<title>Page 1</title>

<meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
<style type="text/css">
<!--
    p {margin: 0; padding: 0;} .ft00{font-size:16px;font-
family:Helvetica;color:#000000;-moz-transform: matrix(
0, 0, 0, 0, 0, 0);-webkit-transform: matrix(
1, 1, 0, 0, 0, 0);-o-transform: matrix(
-1, 1, 0, 0, 0, 0);-ms-transform: matrix(
-1, 1, 0, 0, 0, 0);-moz-transform-origin: left 75%;-webkit-
transform-origin: left 75%;-o-transform-origin: left 75%;-ms-transform-
origin: left 75%;}
    .ft01{font-size:16px;font-family:Courier;color:#ff0000;}
    .ft02{font-size:16px;font-family:Courier;color:#0000ff;}
    .ft03{font-size:16px;font-family:Courier;color:#00ff00;}
    .ft04{font-size:16px;font-family:Courier;color:#000000;}
-->
</style>
</head>

```

```

<body bgcolor="#A0A0A0" vlink="blue" link="blue">
<div id="page1-div" style="position:relative;width:918px;height:216px;">

<p style="position:absolute;top:100px;left:39px;white-space:nowrap"
class="ft00">1</p>
<p style="position:absolute;top:6px;left:80px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:80px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:95px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:28px;left:95px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:6px;left:112px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:28px;left:112px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:6px;left:128px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:128px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:141px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:28px;left:141px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:6px;left:156px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:28px;left:156px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:6px;left:172px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:172px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:187px;white-space:nowrap"
class="ft04">G</p>
<p style="position:absolute;top:28px;left:187px;white-space:nowrap"
class="ft04">G</p>
<p style="position:absolute;top:6px;left:204px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:204px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:222px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:28px;left:222px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:6px;left:242px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:28px;left:242px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:6px;left:260px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:28px;left:260px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:6px;left:283px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:283px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:296px;white-space:nowrap"
class="ft01">T</p>

```

T

T

G

G

A

A

C

C

G

G

T

T

T

G

G

G

G

T

T

G

G

G

G

T

T

A

A

T

[illegible]

```

<p style="position:absolute;top:28px;left:750px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:767px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:767px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:786px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:786px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:806px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:28px;left:806px;white-space:nowrap"
class="ft03">A</p>
<p style="position:absolute;top:6px;left:821px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:28px;left:821px;white-space:nowrap"
class="ft01">T</p>
<p style="position:absolute;top:6px;left:833px;white-space:nowrap"
class="ft02">C</p>
<p style="position:absolute;top:28px;left:833px;white-space:nowrap"
class="ft02">C</p>
</div>
</body>
</html>
<html>
<body>
<br>&nbsp;<br>
<table>

<p>The chromatogram was generated using the "sangerseqR" package of
Bioconductor. When heterozygous peaks are present, it can be differentiated
from the homozygous peaks and appropriate bases are called. In the above
plot, the primary and secondary basecalls are similar for the homozygous
peaks whereas the primary and secondary basecalls do not match for the
heterozygous regions and such peaks are highlighted in blue. </p>

</table>
</body>

</html>

```

Chapter 6

DISCUSSION

Working with the modern sequencing systems is a multidisciplinary effort. The production of data is mostly laboratory concentrated, but the processing of the sequence data falls into the bioinformatics discipline and hence automation of the process can be done up to some extent. However, the understanding of the results needs close communication between bioinformatics and biology in order to gain maximum understanding of the data.

With the discovery of more and more disease related genes, the bioinformatics resources needs to be kept up-to-date and the content as well as the quality of the genomic report needs to be maintained.

For the working of MANIPAL System, a Local Gene Database (LGdb) of the three databases i.e., RefSeqGene, dbSNP and ClinVar is created for each gene. The advantage of this type of approach over having all the genes data in a single database is that it takes less amount of time for execution and hence saves time significantly during analysis.

Also the alignment of the forward ABI files for the genes located on the positive strand is straight forward whereas for the alignment of the reverse ABI files for the genes located on the positive strand should be reverse complemented and then aligned with the reference.

The forward ABI files for the genes located on the negative strand should be reverse complemented to make it suitable for aligning with the reference while the alignment of the reverse ABI files for the genes located on the negative strand is done with the reference without reverse complementing since the reverse ABI file sequences for the genes located on the negative strand is equivalent to the forward ABI file sequences for the genes located on the positive strand.

Also, presenting the result in an intelligible manner without any complexity and making it clinically interpretable for the physicians would help in making better and efficient clinical decisions.

Chapter 7

CONCLUSIONS

The MANIPAL System is comprised of a series of steps including execution of various command line programs. All the automation is done using shell scripting. The workflow begins with a single ABI file as input and concludes with generating a genomic report.

The ABI files dataset of 11 diagnostic genes (**Table 3**), generated from the Applied BioSystems 3130 Genetic Analyzer, were successfully processed using the MANIPAL System.

Although the system was tested with only 11 genes, it is not limited to only these particular genes. It can be configured to analyse as many genes as possible by simply running the configuration bash scripts, which will create a customized database for the genes and prepare the system for analyzing them.

The framework of the system is feasible with incorporating multiple tools for performing the same analysis. This makes it very easy for integrating new programs, which can do better for a specific step.

MANIPAL system efficiently integrates various open source software's for performing base calling, alignment, variant calling, variant annotation and visualization as well as reporting to accurately discover clinically significant mutations. Automating the entire process of sequence based mutation detection, beginning from analyzing the ABI file obtained from the DNA Sequencer to generating the genomic report, saves a considerable amount of time. Overall, the MANIPAL system succeeds in efficiently reporting the clinically significant mutations in a systematic and clinician-friendly manner.

Chapter 8

REFERENCES

1. Adzhubei IA, Schmidt S, Peshkin L, Ramensky VE, Gerasimova A, Bork P, Kondrashov AS and Sunyaev SR. (2010) A method and server for predicting damaging missense mutations, *Nat Methods* (7):248–9.
2. Aho AV, Kernighan BW and Weinberger PJ. (1988) *The AWK programming language*, Addison-Wesley Longman Publishing Co., Inc, Boston, MA.
3. Altschul SF, Gish W, Miller W, Myers EW and Lipman DJ. (1990) Basic local alignment search tool, *J Mol Biol* (215):403-10.
4. Camacho C, Coulouris G, Avagyan V, Ma N, Papadopoulos J, Bealer K and Madden TL. (2009) BLAST+: architecture and applications, *BMC Bioinformatics* (10):421.
5. Cooper DN, Ball EV and Krawczak M. (1998) The human gene mutation database, *Curr Protoc Bioinformatics, Nucleic Acids Res* (26):285-7.
6. Danecek P, Auton A, Abecasis G, Albers CA, Banks E, DePristo MA, Handsaker RE, Lunter G, Marth GT, Sherry ST, *et al.*, (2011) The variant call format and VCFtools, *Bioinformatics* (27):2156-8.
7. Duzkale H, Shen J, McLaughlin H, Alfares A, Kelly MA, Pugh TJ, Funke BH, Rehm HL and Lebo MS. (2013) A systematic approach to assessing the clinical significance of genetic variants, *Clin Genet* (84):453-63.
8. Ewing B, Hillier L, Wendl MC and Green P. (1998) Base-calling of automated sequencer traces using phred. I. Accuracy assessment, *Genome Res* (8):175-85.
9. Ewing B and Green P. (1998) Base-calling of automated sequencer traces using phred. II. Error probabilities, *Genome Res* (8):186-94.
10. Field D, Tiwari B, Booth T, Houten S, Swan D, Bertrand N and Thurston M. (2006) Open software for biologists: from famine to feast, *Nat Biotechnol* (24):801-3.
11. Garrison E and Marth G. (2012) Haplotype-based variant detection from short-read sequencing, Preprint at arXiv:1207.3907v2 [q-bio.GN].

12. Genetic Home Reference. (2015) How can gene mutations affect health and development? Handbook Help Me Understand Genetics, CreateSpace Independent Publishing Platform, Bethesda.
13. Green RC, Berg JS, Grody WW, Kalia SS, Korf BR, Martin CL, McGuire AL, Nussbaum RL, O'Daniel JM, Ormond KE, *et al.*, (2013) ACMG recommendations for reporting of incidental findings in clinical exome and genome sequencing, *Genet Med* (15):565-74.
14. Hamosh A, Scott AF, Amberger JS, Bocchini CA and McKusick VA. (2005) Online Mendelian Inheritance in Man (OMIM), a knowledgebase of human genes and genetic disorders, *Nucleic Acids Res* (33):D514–D7.
15. Hill JT, Demarest BL, Bisgrove BW, Su YC, Smith M and Yost HJ. (2014). Poly peak parser: Method and software for identification of unknown indels using sanger sequencing of polymerase chain reaction products, *Dev Dyn* (242):1632-6.
16. http://www.appliedbiosystems.com/support/software_community/ABIF_File_Format.pdf accessed on 10/9/2014.
17. Karolchik D, Baertsch R, Diekhans M, Furey TS, Hinrichs A, Lu YT, Roskin KM, Schwartz M, Sugnet CW, Thomas DJ, *et al.*, (2003) The UCSC Genome Browser Database, *Nucleic Acids Res* (31):51-4.
18. Landrum MJ, Lee JM, Riley GR, Jang W, Rubinstein WS, Church DM and Maglott DR. (2014) ClinVar: public archive of relationships among sequence variation and human phenotype, *Nucleic Acids Res* (42):D980-5.
19. Li H, Handsaker B, Wysoker A, Fennell T, Ruan J, Homer N, Marth G, Abecasis G, Durbin R and 1000 Genome Project Data Processing Subgroup. (2009) The Sequence Alignment/Map format and SAMtools, *Bioinformatics* (25):2078-9.
20. Li H. (2013) Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM, Preprint at arXiv:1303.3997v2 [q-bio.GN].
21. Ng PC and Henikoff S. (2003) SIFT: predicting amino acid changes that affect protein function, *Nucleic Acids Res* (31):3812–3814.

22. Pruitt KD, Tatusova T and Maglott DR. (2007) NCBI reference sequences (RefSeq): a curated non-redundant sequence database of genomes, transcripts and proteins, *Nucleic Acids Res* (35):D61-5.
23. R Core Team (2013). R: A language and environment for statistical computing, R Foundation for Statistical Computing, Vienna, Austria.
24. Rice P, Longden I and Bleasby A. (2000) EMBOSS: the European Molecular Biology Open Software Suite, *Trends Genet* (16):276-7.
25. Richards CS, Bale S, Bellissimo DB, Das S, Grody WW, Hegde MR, Lyon E, Ward BE and Molecular Subcommittee of the ACMG Laboratory Quality Assurance Committee. (2008) ACMG recommendations for standards for interpretation and reporting of sequence variations: Revisions 2007, *Genet Med* (10):294-300.
26. Richards S, Aziz N, Bale S, Bick D, Das S, Gastier-Foster J, Grody WW, Hegde M, Lyon E, Spector E, *et al.*, (2015) Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology, *Genet Med* (17):405-24.
27. Stajich JE, Block D, Boulez K, Brenner SE, Chervitz SA, Dagdigian C, Fuellen G, Gilbert JG, Korf I, Lapp H, *et al.*, (2002) The Bioperl toolkit: Perl modules for the life sciences, *Genome Res* (12):1611-8.
28. Sherry ST, Ward MH, Kholodov M, Baker J, Phan L, Smigielski EM and Sirotkin K. (2001) dbSNP: the NCBI database of genetic variation, *Nucleic Acids Res* (29):308-11.
29. Sun K, Yuen YP, Wang H and Sun H. (2014) Online Diagnosis System: a webserver for analysis of Sanger sequencing-based genetic testing data, *Methods* (69):230-6.
30. Vassy JL, McLaughlin HL, MacRae CA, Seidman CE, Lautenbach D, Krier JB, Lane WJ, Kohane IS, Murray MF, McGuire AL, *et al.*, (2015) A one-page summary report of genome sequencing for the healthy adult, *Public Health Genomics* (18):123-9.
31. Wang K, Li M and Hakonarson H. (2010) ANNOVAR: functional annotation of

genetic variants from high-throughput sequencing data, *Nucleic Acids Res* (38):e164.

PROJECT SYNOPSIS

Introduction:

Clinical Genetic testing allows the diagnosis and assessment of individuals risk to a disease. The most common type of genetic variations in humans are the SNP's. The analyzed gene and the details related to these variations affecting the individual's gene can be obtained from the variation databases such as dbSNP, ClinVar, HGMD, LSDB, etc.

Single nucleotide polymorphisms (SNP's) and small insertions and deletions (INDEL's) are common genetic variants that are related to a lot of human genetic conditions. Hence, accurate detection of these variations is very important. These variations can be identified by sequencing techniques such as Sanger and next generation sequencing (NGS). Despite the advent of next-generation sequencing technologies, Sanger sequencing still remains the most popular and robust method of automatic sequencers. It is established on capillary electrophoresis and is less susceptible to errors in sequencing. Its sensitivity and accuracy makes the fluorescence-based sequencing as the 'gold' standard for detecting DNA variations. With the explosion of data generation, there is also a need to automate the data processing. Most of the software solutions presently available require user intervention or provide modules that need advanced informatics skills to allow implementation in pipelines.

Experimental validation for the association of these variants to a disease is very time consuming and labor intensive, there is a need to automate the process of variant analysis, classification and reporting. Hence, to reduce the amount of time consumed during the analysis and reporting of the clinically significant variants of an individual, various methods currently exist to classify the identified SNP's on the basis of their pathogenicity such as SIFT, Polyphen2, etc.

There are many popular programs that perform base calling, alignment, polymorphism identification, polymorphism annotation and variant assessment. These tools need to be configured separately and human interference is needed in all the steps to assess variant. Currently there is no availability of all these tools compiled in one package, and hence we take forward the previous conceptualized and automated system called "MANIPAL: **M**utation detection and **A**nnotation in **N**ucleotide sequences with an **I**ntegrated **P**ipeline for **A**utomation in **L**aboratories" to call, identify and assess the variants. For the purpose of clinical diagnosis, features such as variant classification and report generation can be added to the pre-existing MANIPAL pipeline to make it more clinically efficient and operational.

Aim and Objectives:

Aim: To develop an automated diagnostic framework for analyzing, assessing and reporting the clinically significant variants of individuals from the Sanger sequencing data.

Objectives:

- Prioritization of variants into one of the five clinical categories.
- To automate the retrieval of relevant variant details from databases and literature
- Combine all the variants into a genomic report.
- To visualize chromatogram traces of the input ABI file and the alignment of the reads with the reference genome.
- Design a web interface for the “MANIPAL: **M**utation detection and **A**nnotation in **N**ucleotide sequences with an **I**ntegrated **P**ipeline for **A**utomation in **L**aboratories” system by implementing all the tools of the pipeline for back-end processing.

Methodology:

- **Base Calling-** The input file is an ABI chromatogram trace file, where the bases are called using Phred. The output file is the nucleotide file written in FASTA format and the quality values for each of the called bases in a “.qual” file format. Phred reads each of the ABI trace files, calls the bases and allocates a quality score to each of the called bases
- A Perl script “fa_qual2fastq.pl” combines the file in FASTA format and the corresponding quality values into a FASTQ file.
- **Alignment-** BWA is a read alignment software, which outputs alignment in the SAM format. BWA software then maps the reads in FASTQ format to the reference, resulting in a SAM format output file. The SAM file is then converted into BAM file by a tool named SAMtools.
- **Variant Identification-** FreeBayes tool calls variants with parameter of supporting observations to consider a variant set to 1 and generates the VCF file, which is a text file format that is used to store the DNA sequence variations.

- **Variant Annotation-** Finally, the variants present in the VCF file format are annotated using ANNOVAR's Perl script "table_annovar.pl", and VARANT. ANNOVAR is a tool for the annotation of SNV's (Single Nucleotide Variants) and INDELs (Insertions and Deletions). In order to filter out the variants for the potential mutations in *Homo sapiens* genome which are non-synonymous, it uses six different scores: GERP++, LRT, MutationTaster, PolyPhen, PhyloP conservation and SIFT. VARANT (Variant Annotation Tool) is an open source annotation tool for the human genome and is built using Python programming language.
- **Variant Filtering-** Variant Assessment is done on the basis of genic locations, predicted functional impacts, disease associations in databases and allele frequencies. The standards recommended by the ACMG (American College of Medical Genomics and Genetics) for interpreting and reporting the sequence variations are followed.

Tools/Platform:

The MANIPAL pipeline for the automated data analysis of the Sanger sequence reads is built using the tools, which provide the optimal results. The pipeline was developed with Bash programming language, and tested on Biolinux 7 operating system for ABI-files of "ADRB2" gene generated from ABI 3130 Genetic Analyzer (Applied Biosystems). The tools used are as follows-

1. Phred - A tool for quality and base calling of the data from DNA sequencing instruments.
2. phd2fasta - A program that comes along with Phred, uses the PHD files as input and produces the FASTA files of the reads.
3. "fasta_qual2fastq.pl" Perl script - This combines the fast and quality files to generate the FASTQ file.
4. BWA (Burrows Wheeler Aligner) - It is a read alignment software which produces alignment in SAM file format.
5. SAMtools - It contains modules which can perform conversion of files, statistics of alignment, calling variants and visualizing the assembly.

6. FreeBayes - It is a variation calling tool from the mapped read sequence data which uses BAM file and a reference as an input. The output is generated in a VCF format that describes the INDEL's, SNP's and complex variants in the samples of the input alignments.
7. ANNOVAR - It is a tool for the annotation of SNV's (Single Nucleotide Variants) and INDELs (Insertions and Deletions), by analyzing their functional effect on the genes that they are present in, deducing the cytogenetic bands, describing the functionally important scores, detecting the variants present in the conserved regions or identifying the variants that are present in the 1000 Genome Project and dbSNP.
8. VARANT (Variant Annotation Tool) - It is an extensible and comprehensive an open source annotation tool for the human genome and is built using Python programming language. It is able to use multiple cores/processors for fast annotation and provides immense annotations of the non-coding and UTR regions, transcripts that are affected by the variants, the kind of effect, the potential functional outcomes by including the pathways that these variants may affect and the disease conditions that are known from a variety of different sources like ClinVar, OMIM, dbSNP, along with the gene annotations.
9. CoVEC (Consensus Variant Effect Classification) - It is a tool that integrates different pathogenicity prediction tools and combines results of multiple tools.
10. CAVA (Clinical Annotation of Variants) - It is a variant annotation tool which implements a clinical sequencing nomenclature (CSN).
11. Computational prediction for missense variants - SIFT, Polyphen2, MutationAssessor, MutationTaster.
12. Visualization tools - BioR, TREAT, Artemis, Savant, and IGV.
13. Literature Database - PubMed.
14. Variant Databases - dbSNP, ClinVar, COSMIC, 1000 Genomes Project, HGMD, OMIM.

Expected Results:

Automating the entire process of sequence based variation detection, beginning from analyzing the ABI output file obtained from the DNA Sequencer to generating the genomic report, would save a considerable amount of time. Also, presenting the result in an intelligible

manner without any complexity and making it clinically interpretable for the physicians would help in making better and efficient clinical decisions.

The result of the automated framework would be a genomic report containing a summary of the variants, which may lead to a disease and would be followed by more details of the clinically significant variants.

References:

1. Danecek P, Auton A, Abecasis G, Albers CA, Banks E, DePristo MA, Handsaker RE, Lunter G, Marth GT, Sherry ST, *et al.*, (2011) The variant call format and VCFtools, *Bioinformatics*. (27):2156-8.
2. Duzkale H, Shen J, McLaughlin H, Alfares A, Kelly MA, Pugh TJ, Funke BH, Rehm HL and Lebo MS. (2013) A systematic approach to assessing the clinical significance of genetic variants, *Clin Genet*. (84):453-63.
3. Garrison E and Marth G. (2012) Haplotype-based variant detection from short-read sequencing, *arXiv*. 1207.3907v2.
4. Haga SB, Mills R, Pollak KI, Rehder C, Buchanan AH, Lipkus IM, Crow JH and Datto M. (2014) Developing patient-friendly genetic and genomic test reports: formats to promote patient engagement and understanding, *Genome Med*. (6):58.
5. Kundu K, Rana S, Chellappan A, Sunderam U, Puck JM, Brenner SE, Srinivasan R. (2014) Varant: An Open Source Variant Annotation Tool.
6. Landrum MJ, Lee JM, Riley GR, Jang W, Rubinstein WS, Church DM and Maglott DR. (2014) ClinVar: public archive of relationships among sequence variation and human phenotype, *Nucleic Acids Res*. (42):D980-5.
7. Li H, Handsaker B, Wysoker A, Fennell T, Ruan J, Homer N, Marth G, Abecasis G, Durbin R and 1000 Genome Project Data Processing Subgroup. (2009) The Sequence Alignment/Map format and SAMtools, *Bioinformatics*. (25):2078-9.
8. Li H. (2013) Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv*. 1303.3997v2:1-3.
9. Ng PC and Henikoff S. (2003) SIFT: predicting amino acid changes that affect protein

function, Nucleic Acids Res. (31):3812–3814.

10. Richards CS, Bale S, Bellissimo DB, Das S, Grody WW, Hegde MR, Lyon E, Ward BE and Molecular Subcommittee of the ACMG Laboratory Quality Assurance Committee. (2008) ACMG recommendations for standards for interpretation and reporting of sequence variations: Revisions 2007, Genet Med. (10):294-300.
11. Vassy JL, McLaughlin HL, MacRae CA, Seidman CE, Lautenbach D, Krier JB, Lane WJ, Kohane IS, Murray MF, McGuire AL, *et al.*, (2015) A one-page summary report of genome sequencing for the healthy adult, Public Health Genomics. (18):123-9.
12. Wang K, Li M and Hakonarson H. (2010) ANNOVAR: functional annotation of genetic variants from high-throughput sequencing data, Nucleic Acids Res. (38):e164.